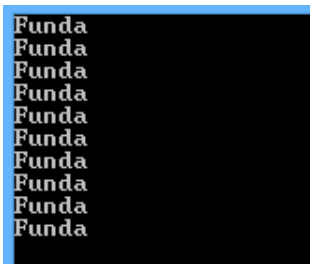
Döngüler - Örnekler

Örnek 1: İsmimizi ekrana 10 defa yazdıran C programı ve ekran çıktısı

```
#include <stdio.h>
// for ile
main()
{
	int i;
	for(i=1; i<=10; i++)
		printf("Funda\n");
}
```




```
--

#include <stdio.h>
// while ile
main()
{
	int i;
	i=1;
	while(i<=10)
	{
		printf("Funda\n");
		i++;
	}
}
---
#include <stdio.h>
//do-while ile
main()
{
	int i;
	i=1;
	do
	{
		printf("Funda\n");
		i++;
	}
	while (i<=10);
}
```

Örnek 2: 1'den 10'a kadar olan sayıları ekranda bastıran program

```
#include <stdio.h>
// for ile
main()
{
	int i;
	for(i=1; i<=10; i++)
		printf("%d\n", i);
}
```

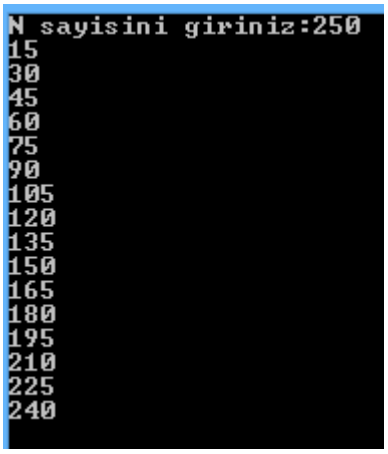


Döngüler - Örnekler

Örnek 3: 1'den N'ye kadar olan sayılar içinde 3'e ve 5'e tam bölünebilen sayıları bulan ve ekranda gösteren C kodu.

```
#include <stdio.h>
//
main()
{
	int i,N;
	printf("N sayisini giriniz:");
	scanf("%d", &N);

	for(i=1;i<=N;i++)
	{
		if(i%3==0 && i%5==0)
			printf("%d\n",i);
	}
}
```

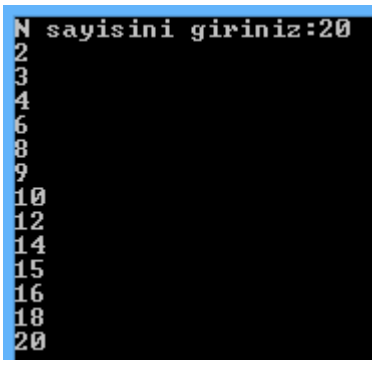


*Not: && (AND) işleci ile iki koşul birbirine bağlanabilir ve bu koşulların **aynı anda** sağlanması şartı vardır. İkisinden en az birisi sağlanmadığı durumda koşul sağlanmaz ve koşul içerisindeki işlemi gerçekleştirmez. Bu örnekte 250 ye kadar olan sayılar içinden 3 ve 5'e aynı anda bölünebilenler ekranda gösterilmiştir.*

Örnek 4: 1'den N'ye kadar olan sayılar içinde 2'e veya 3'e tam bölünebilen sayıları bulan ve ekranda gösteren C kodu.

```
#include <stdio.h>
//
main()
{
	int i,N;
	printf("N sayisini giriniz:");
	scanf("%d", &N);

	for(i=1;i<=N;i++)
	{
		if(i%2==0 || i%3==0)
			printf("%d\n",i);
	}
}
```



*Not: || (OR) işleci ile iki koşul birbirine bağlanabilir ve bu koşulların **aynı anda** sağlanması **gerekmez.** İkisinden en az birisinin sağlanması ile koşul içerisindeki işlem gerçekleşir. Bu örnekte 20'ye kadar olan sayılardan 2 ve 3'ten en az birine bölünebilen sayılar ekranda gösterilmiştir.*

Döngüler - Örnekler

Örnek 5: Dışardan çift sayı girildikçe toplama işlemi yapan, tek sayı girildiği durumda ise girilen çift sayıların toplamını gösteren C kodu

```
#include <stdio.h>

main()
{
   int sayi=0, toplam=0;

   while( sayi % 2 == 0)
   {
            toplam += sayi;

            printf("Bir sayi girin : ");
            scanf("%d",&sayi);
   }
   printf("Döngü sona erdi.");
   printf("Toplam=%d ", toplam);
}
```

```
Bir sayi girin : 30
Bir sayi girin : 20
Bir sayi girin : 5
Dongu sona erdi.Toplam=50
--------------------------
```

Örnek 6: Dışardan girilen bir sayının faktöriyelini hesaplayan programın C kodu

```
#include <stdio.h>

int main()
{
   int i, n, faktor;

   printf("Faktoriyeli hesaplanacak sayi girin : ");
   scanf("%d",&n);

   faktor=1;
   for(i=1; i<=n; i++)
   {
         faktor *= i;
   }

   printf("%d! = %d\n", n, faktor);
}
```

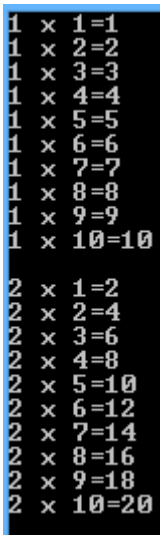
```
Faktoriyeli hesaplanacak sayi girin : 5
5! = 120

--------------------------------
```

Döngüler - Örnekler

Örnek 7: İç içe döngü yapısı ile çarpım tablosunu gerçekleştiren C kodu.

```
#include <stdio.h>
main()
{
    int x,y;
    for (x=1;x<=10;x++)
    {
        for (y=1;y<=10;y++)
        {
            printf("%d x %d=%d",x , y , x*y);
        }
    }
}
```

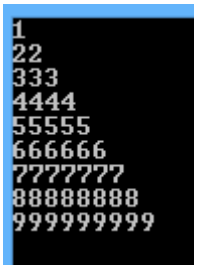


Not: Ekran görüntüsü sadece 2 ye kadar olan çarpım tablosuna aittir. Programı bu şekliyle çalıştırdığınız zaman 10'a kadar olan sonuçları göreceksiniz.

Örnek 8:

```
#include <stdio.h>
main()
{
    int x,y;
    for (x=1;x<10;x++)
    {
        for (y=1;y<=x;y++)
        {
            printf("%d", x);
        }

        printf("\n");
    }
}
```

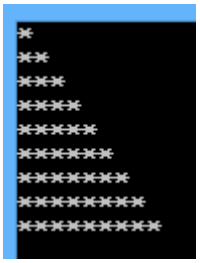


Döngüler - Örnekler

Örnek 9:

```
#include <stdio.h>
main()
{
    int x,y;
    for (x=1;x<10;x++)
    {
        for (y=1;y<=x;y++)
        {
            printf("%c", '*');
        }

        printf("\n");
    }
}
```

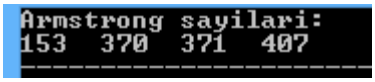


Not: Karakter cinsinden bir değeri görüntülemek için, Örnek 8'deki %d yerine %c yazıp, kullanacağımız karakteri tek tırnak ile almamız gerekmektedir.

Örnek 10: Üç basamaklı, basamaklarının küpleri toplamı kendisine eşit olan tam sayılara ***Armstrong sayısı*** denir. Örneğin: 371 = 3^3 + 7^3 + 1^3.
İç içe döngü yapısı kullanarak 3 basamaklı Armstrong sayılarını bulan programı gerçekleyelim.

```
#include <stdio.h>
main()
{
   int a,b,c, sayi1, sayi2, k=1;
   printf("Armstrong sayilari:\n");
   for(a=1; a<=9; a++)
            for(b=0; b<=9; b++)
                  for(c=0; c<=9; c++)
                  {
                        sayi1 = 100*a + 10*b + c;
                        // sayi1 = abc (üç basamaklı)
                        sayi2 = a*a*a + b*b*b + c*c*c;
                        // sayi2  = a^3+b^3+c^3

                  if( sayi1==sayi2 )
                        printf("%d  ", sayi1);
                  }
}
```



3

Pointers

Pointer, başka bir değişkenin bellekteki adresini işaret eden bir değişkendir. O nedenle, pointere *işaretçi, gösterici* adları verilir. Değişkenler için olduğu gibi, pointerin değeri, işaret ettiği adresteki değerdir. Nasıl ki, değişken bildiriminde, değişkenin veri tipini belirtiyorsak, pointer bildiriminde de, onun işaret edeceği bellek adresine konulabilecek değerin veri tipini belirtmeliyiz. Aslında bütün değişkenler, bellekte birer adres işaret ederler. Dolayısıyla, bütün değişkenler birer pointerdir. Ama, C dilinde pointerlerle *pointer aritmetiği* denilen işlemler yapılır. Bu işlemler, bazı işlerin yapılmasını çok kolaylaştırır. Pointer aritmetiğini biraz sonra açıklayacağız.

Bildirimi sırasında, değişkenin pointer olduğunu belirtmek için, değişken adının önüne yıldız (*) karakteri konulur:

```
int gelir;
int *gelirPtr;
```

bildirimilerinin birincsi *int* değer alan *gelir* adlı bir değişkeni bildiriyor. İkincisi *int* tipi veri içerecek bir bellek adresini işaret eden **gelirPtr* adlı bir pointer bildirimidir. Her ikisi de *int* tipinden olduğu için, istenirse iki bildirim tek bildirim halinde yazılabilir:

```
int gelir, *gelirPtr;
```

Pointer bildiriminde, değişken adının sonuna *Ptr* ekledik. Bunu yapmak zorunlu değildir. Derleyici yalnızca değişkenin önündeki (*) karakterine bakar. *Ptr* eklememizin nedeni, kaynak programı okuyan kişinin pointer değişkenlerini kolay algılaması içindir. Zorunlu olmayan bu işi alışkanlık edinmek yararıdır.

3.1 Pointer'in Yararları

Her veri tipinden pointer tanımlanabilir. O nedenle, bazı kaynaklara göre, pointer, C dilinde türetilmiş (derived) bir veri tipidir. Pointere atanan değerler, değişken adresleridir. Pointer kendisine atanan adresteki değeri gösterir. Dolayısıyla, bellekteki verilerle işlem yapmak için güçlü bir araçtır. Pointerlerin başlıca yararları şöyle sıralanabilir:

1. Arrayler ve veri tabloları üzeride işlemleri kolaylaştırır.

2. Dinamik adres yönetimini sağlar.

3. *Değerle çağrı*'nın asıl olduğu fonksiyonlarda, parametrelere pointerler atanabilir ve böylece *referan ile çağrı* oluşur. Bu oluşumda, parametre değerleri değişebilir ve fonksiyon birden çok değer verebilir. (Bununla ilgili örnekleri birazdan inceleyeceğiz).

4. Pointerler, doğrudan adresleri gösterdiği için, programın koşma hızını artırır.

5. Pointer bağlı listeler, kuyruklar, yığıtlar (stacks) gibi yapılarla işlem yapmayı kolaylaştıran bir araçtır.

3.1.1 Bellek Adresleri

Bilgisayarın CPU'dan sonra en önemli bileşeni anabelleğidir. Buna İngilizcede RAM (Random Access Memory - Seçkili Erişilir Bellek) denilir. Türkçe'de ona *ana bellek* diyoruz. Donanımsal yapıları farklı olan RAM türleri vardır. Ama programcı için, RAM'in donanımsal yapısı önem taşımaz. Hepsi benzer kuralla işlev yapar. Genellikle, bellek adresleri 1 byt'lık (8 bit'lik) gözelerden (hücre) oluşur. Her gözenin pozitif tam sayı ile belirlenen bir adresi vardır. Göze adresleri artan sırada dizilirler. İşletim sistemi, her gözenin adresini bilir. *Seçkili Erişilir Bellek (RAM)* denmesinin nedeni, istenilen gözeye doğrudan erişilebiliyor olmasıdır.

Bilgisayar çalışırken, işletim sisteminin komutları, uygulama programının komutları ve veriler anabellekte tutulur. Uygulama programının gerekli gördüğü komutlar ve veriler anabelleğe yerleştirilir. Bu işi işletim sistemi ve uygulama programı birlikte yaparlar. Anabellek, içine veri yazılan hücrelerden oluşur. Hücrelerin her birisi numarasıyla belirlidir. Buna hücrenin adresi diyoruz. İşletim sistemi ve dolayısıyla derleyici ana bellekteki her hücrenin numarasını (adresini) bilir.

Ana bellekte göze sayısı ne kadar çoksa, orada tutulanlar (veriler, komutlar vb) o kadar çok olur. Bu da programın daha hızlı çalışmasını sağlar. Tabii, RAM büyüklüğü sistemden sisteme değişir. Ama hepsinin bir sınırı vardır. Çok sayıda veri işleyen programlar, RAM'e sığmayan verileri hard disk, flopy disk vb gibi ikincil bir kayıt ortamına yazar ve gerektikçe oradan alır. Oraya süpürme alanı *(swap space)* denilir. Genellikle büyük veri tabanlarında bu iş kaçınılmazdır. İkincil kayıt ortamı ile veri alış-verişi RAM'dekilere göre daha yavaştır.

C dilinde bir değişkenin anabellek adresi, değişkenin önüne konulan & karakteri ile belirtilir. Pointerlerin değerleri de bellek adresleri olduğuna göre * ile & karakterleri birbirlerinin tümleyeni olarak işlev yaparlar. Bunu bir örnekle açıklayalım.

```
int x = 77;

int *p;

p = &x;
```

bildirimlerinden birincisi *int* tipinden x değişkeni tanımlıyor ve ona 77 değerini atıyor. İkincisi *int* tipinden verilere ayrılan bir adresi gösterecek olan *p* pointer değişkenini tanımlıyor. *p* yerine istenilen başka bir ad verilebilir. Bu aşamada *p* pointeri henüz bir adres göstermiyor. Üçüncü deyim, *p* pointer değişkenine x değişkeninin adresini atıyor. Böylece *p* pointeri x 'in adresini işaret etmeye başlıyor; o adreste 77 değeri vardır. Dolayısıyla *p* pointeri 77 değerini işaret etmeye başlıyor. Buna göre,

```
p = &x;

*p = x;
```

atamaları birbirlerine denk olur. Bunu bir örnekle gösterelim:

Program 3.1.

```
1  #include <stdio.h>

   int main( void ) {

     int x, *p;
6
     x = 77;
     p = &x;


11   printf( "\nx degiskeninin degeri   : %d"   , x );
     printf( "\nx degiskeninin adresi   : %d"   , &x );
     printf( "\n");
     printf( "\n*p in degeri            : %d"   , *p );
     printf( "\n  *p = x                :%ld = %ld"   , *p, x );
16   printf( "\n");
```

```
  printf( "\np pointerinin degeri     : %d"   , p );

  printf( "\n  p = &x  (hex)          :%x = %x"   , p, &x );
  printf( "\n  p = &x  (long)         :%ld = %ld"   , p, &x );
}
```

```
/**
  x degiskeninin degeri    : 77
  x degiskeninin adresi    : 2293316

  *p in degeri             : 77
     *p = x                : 77 = 77

  p pointerinin degeri     : 2293316
    p = &x  (hex)          :22fe44 = 22fe44
    p = &x  (long int)     :2293316 = 2293316
*/
```

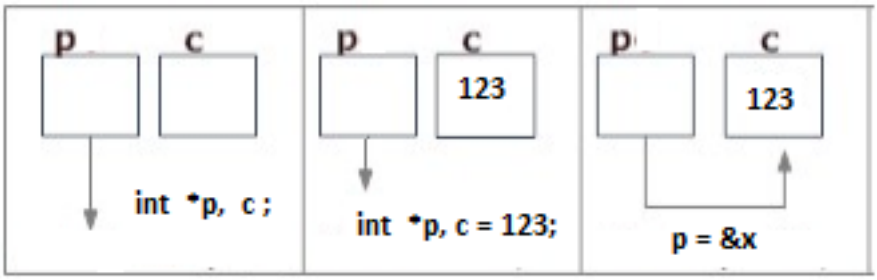
Açıklama:

Çıktının 6. satır $*p = x$ eşitliğini, 9. ve 10 satırları ise $p = \&x$ eşitliğini *hexadecimal* ve *long int* olarak gösteriyor.

Uyarı: Pointer bildiriminde * karakterini iki farklı konumda kullanabiliriz:

1. int *p;
2. int* p;

Bu derste kullandığımız *gcc* derleyicisi bu iki bildirimi eşdeğer sayar. Notasyonlarda birliği sağlamak için biz ilk gösterimi kullanacağız. Program 3.2 ikinci bildirimin de kullanılabileceğini gösteriyor.



Şekil 3.1: Pointer'in İşlevi

Program 3.2.

```c
#include <stdio.h>

int main() {
  int* p;
  int c;

  c=123;
  p = &c;
  printf("c = 123 ise\n");
  printf("c nin adresi:%d\n",&c);
  printf("c nin degeri:%d\n\n",c);
  printf("p pointerinin degeri:%d\n",p);
  printf("p pointerinin gosterdigi deger:%d\n\n",*p);

  return 0;
}
```

```
/**
c = 123 ise
c nin adresi:2293316
c nin degeri:123

p pointerinin degeri:2293316
p pointerinin gosterdigi deger:123
*/
```

3.2 NULL Pointer

NULL, *stdio.h* dosyasında tanımlı sembolik bir sabittir. Türkçe'de *boş*, *hiç* anlamlarına gelir. Bir pointerin hiçbir adresi işaret etmediğini belirtmek için, ona NULL değer atamak iyi bir alışkanlıktır. Aksi halde, pointer çöp (garbage) toplar; yani herhangi bir değer alabilir, dolayısıyla öngörülemeyen bir bellek adresini işaret edebilir. NULL değer atanmış pointerin; yani hiçbir adresi işaret etmeyen pointerin değeri daima 0 olur. Kendisine hiç değer atanmamış, NULL atanmış ya da bir değişken adresi atanmış pointerlerin değerlerini Program 3.3 örneğinden görebiliriz:

Program 3.3.

```c
#include <stdio.h>

int main () {
  int x, *p1, *p2 , *p3;
  x = 123;
  p2 = NULL;
  p3 = &x;

  printf("Deger atanmamis p1 pointerinin degeri : %x\n", p1  );
```

```
    printf("p2 = NULL atanmis pointerin degeri     : %x\n", p2  );
    printf("p3= &x atanmis pointerinin degeri      : %x\n", p3  );
    printf("&x   adresinin degeri                  : %x\n", p3  );

    return 0;
}
```

```
/**
Deger atanmamis p1 pointerinin degeri : 1e
p2 = NULL pointerinin degeri          : 0
p3        pointerinin degeri          : 22fe34
&x   adresinin degeri                 : 22fe34
```

Açıklama:

Çıktının 2. satırında hiç değer atanmamış bir pointerin aldığı değer görülüyor. Bu değer öngörülemez çöp (garbage) değerdir. Pointere NULL ya da bir adres atanmadan önce kullanılması hatalara yol açar.

Çıktının 3. satırı NULL atanmış bir pointerin değerinin 0 olduğunu gösteriyor.

Çıktının 4. ve 5. satırları bir değişken adresi atanmış pointerin değerinin atanan adres olduğunu gösteriyor.

3.3 * ve & Operatörleri

x değişkeninin adresinin &x ile gösterildiğini biliyoruz. Burada & bir operatördür. Bellek adresini göstermek gibi önemli bir işleve sahiptir. Ama bellek adreslerindeki veriler üzerinde işlem yapmaya elvermez. Bellek adreslerindeki verilerle işlem yapacağımız zaman * operatörünü kullanırız. p pointerinin gösterdiği adresteki değer, *p ile belirtilir. Anımsayacağınız gibi, x değişkeninin adresini p pointerine atamak için

```
p = &x;
```

deyimini kullanıyoruz. Bu atamadan sonra

```
*p = x;
```

olur. Bu nedenle & operatörü ile * operatörü birbirlerinin *tümleyen*'leridir, denilir. * operatörü, adresteki veriyi dolaylı gösteriyor. O nedenle * operatörüne dolaylı adres operatörü (indirection operator) denilir.

Bazı problemlerin çözümünde & peratörü çok etkili olur. Program 3.4 takas işleminde & operatörünün kullanılışını gösteriyor.

Program 3.4.

```c
#include <stdio.h>

void swap(int *i, int *j) {
   int t = *i;
   *i = *j;
   *j = t;
}

void main() {
   int a = 23, b = 47;
   printf("Önce a: %d, b: %d\n", a, b);
   swap(&a, &b);
   printf("Sonra a: %d, b: %d\n", a, b);
}
```

```
/**
  Önce : a = 18, b = 67
  Sonra: a = 67, b = 18
```

3.4 Operatörlerin Öncelikleri

İşlemlerde operatörlerin öncelik sıraları vardır. Kod yazarken bunlara dikkat edilmezse mantıksal hatalar doğar. Parantez kulllanarak, işlemlerde öncelik sırasını istediğimiz gibi ayarlayabiliriz.

Ana bellek adresleri artan sırada ardışık unsigned int (pozitif tam sayı) değerler alır. Dolayısıyla p pointeri bir adres gösteririyorsa, p için $++$ ve $--$ operatörlerini *önel* ya da *sonal* olarak kullanabiliriz. Böylece $++p$, $--p$, $p++$ ve $p--$ simgeleri sayısal değişkenlerde bildiğimiz anlamlarına denk olur. Adresler üzerinde

```
++, -, !, *, &
```

operatörleri işlevselsdir. Bunların öncelik sıraları sağdan sola doğrudur. Şimdiye dek gördüğümüz operatörlerin öncelik sıralarını Tablo 3.1'de olduğu gibi, topluca görmek yararlı olabilir.

3.5 Fonksiyonları Referans İle Çağırma

Fonksiyon çağrısının değerle (call by value) ve referansla (call by reference) olmak üzere iki türlü yapılabileceğini söylemiş, referansla yapılan çağrıyı incelenmeyi pointer kavramından sonraya bırakmıştık. Artık referans ile çağrıyı inceleyebiliriz.

Değerle çağrıda, parametre ya da yerel değişken olarak kullanılsalar bile, fonksiyon bloku dışındaki değişken değerlerinin değişmediğini söyle-

Operatörler	*İşlem Sırası Yönü*	*Tür*
() [ ]	soldan sağa	en yüksek öncelik
+ − ++ −− ! ∗ &.	sağdan sola	birli
∗ / %	soldan sağa	çarpımsal
+ -	solda sağa	toplamsal
< <= > >=	solda sağa	ilşkisel
== !=	soldan sağa	eşitlik
&&	sağdan sola	mantıksal VE
||	sağdan sola	mantıksal VEYA
?:	solda sağa	koşullu deyim
= += -= ∗= /= %=	solda sağa	atama
,	solda sağa	virgül, ayraç

Tablo 3.1: Operatör Öncelikleri

miştik. Ancak, bazen fonksiyon bloku dışındaki değişken değerlerini değiştirmek isteyebiliriz. Bunun için referans ile çağrı yöntemi kullanılır.

Ama öncelikle, programın & ve * operatörleri ile ne kadar kolay ve kısa yazıldığını Program 3.5 örneğinde görecek, sonra, Program 3.6 örneğinde *main()* ve *takas()* fonksiyonlarının her adımda yaptığı işleri printf() ile ekrana yazdıracağız.

Program 3.5.

```
#include <stdio.h>

void takas(int *i, int *j) {
   int t = *i;
   *i = *j;
   *j = t;
}

void main() {
   int a = 3, b = 8;
   printf("Önce  a: %d, b: %d\n", a, b);
   takas(&a, &b);
   printf("Sonra a: %d, b: %d\n", a, b);
}
```

```
/**
Önce : a = 3,  b = 8
Sonra: a = 8,  b = 3
*/
```

Referans ile çağrıyı (call by reference) iyi anlayabilmek için Program 3.6 örneğini satır satır incelemek (izleme, trace) yeterli olacaktır.

Program 3.6.

```c
#include <stdio.h>

void takas(int* , int*  );
int main() {

int a = 3, b = 8;
printf("\nTakastan once:");
printf("\na = %d, b = %d", a, b);
printf("\n&a =%d  ve &b = %d"  , &a,&b);
printf("\n");

takas(&a, &b);
printf("\n\nTakastan sonra:");
printf("\na = %d, b = %d", a, b);
printf("\n&a =%d  ve &b = %d"  , &a,&b);

}

void takas(int *x, int *y) {
  int z;
  printf("\n&x =%d  , &y = %d  , &z = %d"  , &x,&y, &z);
  printf("\n *x =%d  , *y = %d  , z= %d"  , *x,*y, z);

  printf("\n\n");
  printf("z = *x  atamasından sonra:");
  z = *x;
  printf("\n&x =%d  , &y = %d  , &z = %d"  , &x,&y, &z);
  printf("\n *x =%d  , *y = %d  , z= %d"  , *x,*y, z);

  printf("\n\n");
  printf("*x = *y  atamasından sonra:");
  *x = *y;
  printf("\n&x =%d  , &y = %d  , &z = %d"  , &x,&y, &z);
  printf("\n *x =%d  , *y = %d  , z= %d"  , *x,*y, z);


  printf("\n\n");
  printf("*y = z  atamasından sonra:");
  *y = z;
  printf("\n&x =%d  , &y = %d  , &z = %d"  , &x,&y, &z);
  printf("\n *x =%d  , *y = %d  , z= %d"  , *x,*y, z);
}
```

```
/**
Takastan once:
a = 3, b = 8
&a =2293324  ve &b = 2293320

&x =2293280  , &y = 2293288  , &z = 2293260
 *x =3  , *y = 8 , z= 0

z = *x  atamasından sonra:
&x =2293280  , &y = 2293288  , &z = 2293260
 *x =3  , *y = 8 , z= 3

*x = *y  atamasından sonra:
&x =2293280  , &y = 2293288  , &z = 2293260
 *x =8  , *y = 8 , z= 3
```

```
*y = z  atamasından sonra:
&x =2293280  , &y = 2293288  , &z = 2293260
 *x =8  , *y = 3 , z= 3

Takastan sonra:
a = 8, b = 3
&a =2293324  ve &b = 2293320
*/
```

Açıklamalar:

main() içindeki 6.satır a ve b değişkenlerine ilk değerleri atıyor. 8. ve 9. satırlar a ile b nin değerlerini ve adreslerini yazıyor. Çıktının 3. ve 4. satırları bunları gösteriyor.

main() içindeki 12. satır *takas()* fonksiyonunu çağırıyor. 3. satırda, *takas()* fonksiyonunun *prototipine* bakarsak, parametrelerin *int* gösteren *pointer* olduğunu görebiliriz. Bu demektir ki, *takas()* fonksiyonu çağrılırken, parametrelere ancak *int* tipinden değişkenlerin adresleri konulabilir. 12.satır bu işi yapıyor. Çıktının 5.-19.satırları *takas()* fonksiyonu tarafından yazılıyor. a ile b nin adresleri ile x ile y nin adreslerinin farklı olduğuna dikkat ediniz. Her fonksiyon, kendi kapsanma alanında (scope), kendi parametreleri ve yerel değişkenleri için gerekli adresleri ayırır. Değişken adları aynı olsa bile, bu adresler, fonksiyonu çağıran bloktaki adreslerden farklıdır.

takas() içindeki 26.satırda z= *x ataması yapılınca, 27. ve 28. satırların yaptığı iş, çıktının 10.ve 11.satırlarında görülüyor. x, y değişkenlerinin adreslerinin ve değerlerinin değişmediğine, ama x in değerinin z ye atandığına; dolayısıyla *x= 3, *y= 8, z=3 olduğuna dikkat ediniz. Takas işleminin ilk adımı burada atılıyor.

takas() içindeki 32.satırda *x= *y ataması yapılınca, 33. ve 34. satırların yaptığı iş, çıktının 14.ve 15.satırlarında görülüyor. x, y değişkenlerinin adreslerinin değişmediğine, ama *x= 8, *y= 8 ve z=3 olduğuna dikkat ediniz. Bu adımda, &x içindeki değer değişmiştir. Takas işleminin ikinci adımı burada atılıyor.

takas() içindeki 39.satırda *y= z ataması yapılınca, 40. ve 41. satırların yaptığı iş, çıktının 18.ve 19.satırlarında görülüyor. x, y değişkenlerinin adreslerinin değişmediğine, ama *x= 8, *y= 3 ve z=3 olduğuna dikkat ediniz. Takas işleminin üçüncü ve son adımı burada atılıyor.

takas() fonksiyonunun işlevi bu adımda bitiyor. Program akışı, *takas()* fonksiyonunu çağıran *main()*'e dönüyor ve kaldığı 12.satırdan devam ediyor.

main() içindeki 14. ve 15.satırlar, a ile b değişkenlerinin değerlerini ve

adreslerini yazdırıyor. adreslerin değişmediğini, ama içlerindeki değerlerin takas edildiğini görüyoruz. Böyle olması doğaldır; çünkü *takas()* fonksiyonu onların içindeki *x ve *y değerlerini takas etti.

Program 3.7.

```
#include <stdio.h>

int kareBul ( int* ) ;
int main( void )
 {
int number = 7;
printf ( "\nSayinin ilk degeri   :%d" , number ) ;

kareBul ( &number ) ;
printf ( "\nSayinin yeni degeri  : %d\n" , number ) ;
}

int kareBul ( int *nPtr ){
*nPtr = *nPtr * *nPtr   ;
return  *nPtr;
}

/**

*/
```

BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 1– Genel Giriş ve Programlamanın Temel Kavramları

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

02/03/2021

Konya

Ders İçeriği

1. Genel Giriş ve Programlamanın Temel Kavramları
2. Algoritma Tasarımı ve Akış Diyagramları
3. C Fonksiyonlarına Giriş ve Değişkenler
4. Operatörler-1
5. Operatörler-2 ve Karşılaştırma
6. Döngüler
7. Diziler, Matrisler
8. Sıralama, Arama
9. Fonksiyonlar
10. İşaretçiler
11. String (Sözce)
12. Matematiksel Fonksiyonlar ve Uygulamalar
13. Dosya İşlemleri
14. Örnek Uygulamalar

Kaynaklar

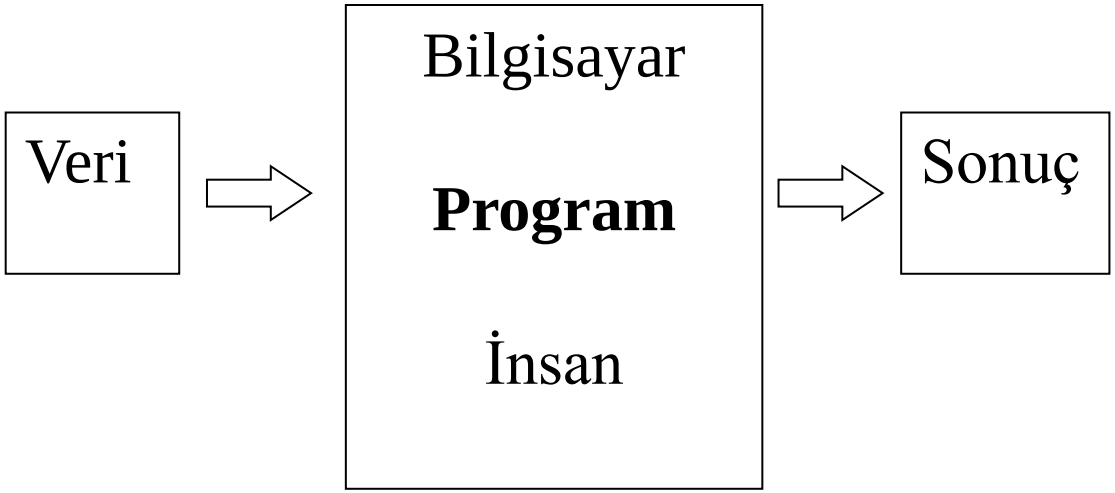
- Programlama Sanatı, Algoritmalar, C Dili Uyarlaması
  Dr. Rifat ÇÖLKESEN, Papatya Yayıncılık
- Her Yönüyle C
  Tevfik KIZILÖREN, Kodlab
- C Programlama Dili
  Dr. Rifat ÇÖLKESEN, Papatya Yayıncılık

- http://www.cagataycebi.com/programming/#c
- http://www1.gantep.edu.tr/~bingul/c/
- http://web.itu.edu.tr/uyar/programlama/c.pdf

Tanımlar

Program:

- Kendi içerisinde bir bütün olan ve belirli bir işi / görevi yerine getiren algoritmik ifade.
- Kod yazılarak ya da donanım tabanlı tasarlanabilir.



Tanımlar

Yazılım:

- Donanım dışında kalan ve programcının kodlama yaparak istenilen bir işin veya görevin yerine getirilmesi için oluşturduğu program/kod ve veri kümesidir.
- Birçok programın, verinin ve dökümanın bir araya gelmesi ile oluşabilir.

Program Kodu:

- Bir işin yapılması için herhangi bir programlama diliyle hazırlanmış algoritmik ifade

Tanımlar

Değişken:

- Verilerin tutulduğu bellek gözlerine verilen simgesel isim.
- Program içinde kullanılacak veriler değişken üzerinde saklanır.
- Örn: Sıcaklık değeri, tekrar sayısı vs…

Diziler:

- Aynı türden verilerin tek bir isim altında sıralı olarak tutulmasını sağlayan değişkendir.
- Bir boyutlu diziler vektör, iki boyutlu diziler matristir.
- Sayılardan oluşur.

Operatör:

- Değişkenler ve veriler üzerinde işlem yapan simgelerdir.
- Örn: Toplama operatörü, karekök operatörü…

Tanımlar

Deyim/İfade (Statements):

- Çeşitli işlemleri gerçekleştiren tekil işlem adımlarıdır.
- Değişken ve operatörlerin birleşmesiyle oluşur.
- Örn: Pırasa yemeğinin pişirilmesi işi => Program

  Pırasaların doğranması => Deyim

Atama Deyimi:

- Değerden değişkene ok ile yapılır.
- Örn N<=5 N değişkenine 5 değerinin atanması.

Tanımlar

Donanım:

- Elektronik elemanlardan oluşan fiziksel birimlerdir.
- Örn: İşlemci, disk, kart…

Bellek: Programa ait kodların ve verilerin saklandığı donanım.

- Genellikle geçici saklama birimidir ve hızlıdır.
- Örn: RAM

Saklama Birimleri:

- Kalıcı saklama birimleridir.
- Örn: HDD, CD…
- Belleke göre daha yavaştır. Hız artışı fiyatı arttırır.

Tanımlar

İşlemci:

- Verileri işleyen, program komutlarını gerçekleştiren birimdir.
- Örn: Intel, AMD…

İşletim Sistemi:

- Bilgisayar ve benzeri cihazlarda çalışan donanım kaynaklarını yöneten ve çeşitli uygulama yazılımları için yaygın servisleri sağlayan bir yazılımlar bütünüdür.
- Bilgisayar ve benzeri cihazların donanımını kullanmamızı sağlayan arayüzdür.
- Örn: Windows, Mac, Linux…

Tanımlar

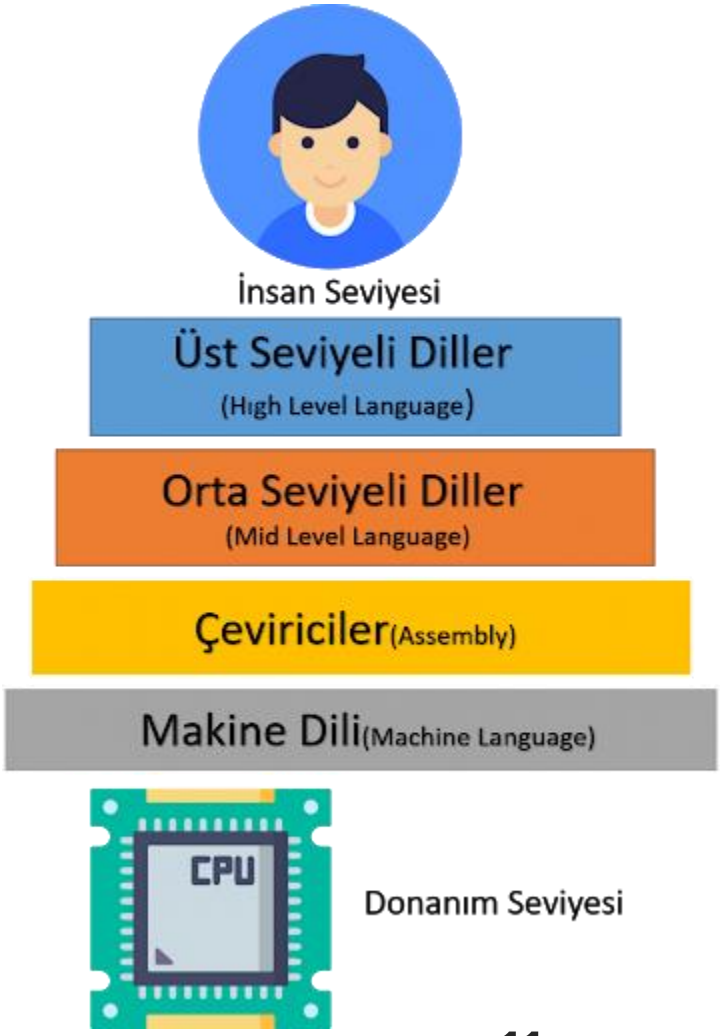
Dosya:

- Sanal ortamdaki verilerin disk üzerinde saklanabilmesi için oluşturulan veri paketidir.
- Hafıza birimi üzerinde her dosyanın başlangıç ve bitiş adresine kadar veriler diske yazılır.
- Silme işleminde ise sadece bu başlangıç adresi yok edilir bu yüzden silme işi çok hızlıdır.
- .jpeg, .doc, .c gibi uzantılar dosyanın içerdiği veri türünü gösterir.

Tanımlar

Programlama Dilleri:

- İnsan ve makine dilleri arasında dönüşümü sağlar.
- Türkçe dili gibi programlama dillerinin de kendi yazım kuralları vardır.
- Özel/dar alanlar için ve daha genel kapsamlı alanlar için tasarlanmış çeşitli diller vardır.
- Orta/üst seviye programlama dillerini kullanırken, üzerinde koşturulacak işlemci hakkında bilgiye ihtiyaç yoktur.
- Assmbly gibi makine diline yakın dilleri kullanmak için işlemci, saklayıcı vs bilgisi gerekir



Tanımlar

C Programlama Dili:

- C dili orta seviye bir programlama dili olarak kabul edilir.
- 1972 yılında Dennis Ritchie tarafından tasarlanarak PDP-11 isimli sistem üzerinde çalıştırılmıştır.
- İnsan dilini makine diline çevirmek için kullanıyoruz.
- Bu dönüşümü gerçekleştirecek bir derleyiciye ihtiyaç duyulmaktadır.

Karakter Tablosu:

- Harf, rakam vb. gibi karakterler ile insan dili ve makine dili arasında dönüşüm yapabilmeyi sağlayan tablodur.
- Dilimizdeki bir metni bilgisayar belleğinde saklayabilmek için dönüşüm yapılır.
- Örn: İnsan Dilinde: BABA <=> ASCII: 65 66 65 66

Tanımlar

Sözce (String):

- İnsan dilindeki cümlenin yazılımdaki karşılığıdır. Anlamlı veya anlamsız olabilir. Karakterlerden oluşur.
- Aritmetik işleme sokulamazlar.
- Sayıya dönüştürülebilirler.

Veri Yapısı:

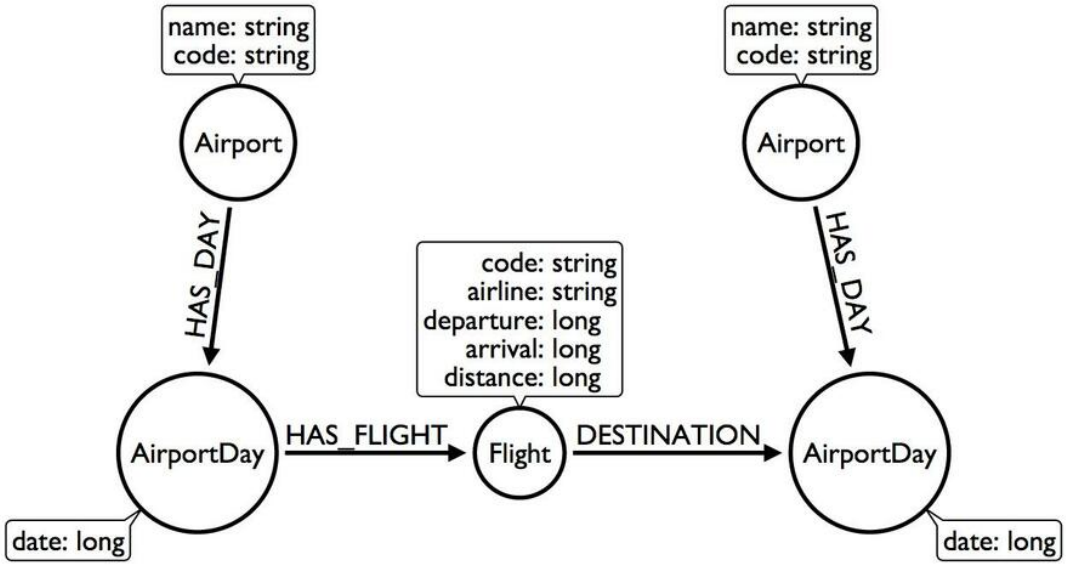
- Verinin bellekte tutulma şeklini ve düzenini gösterir.
- Programlama dillerinde genellikle kullanılan veri yapıları:

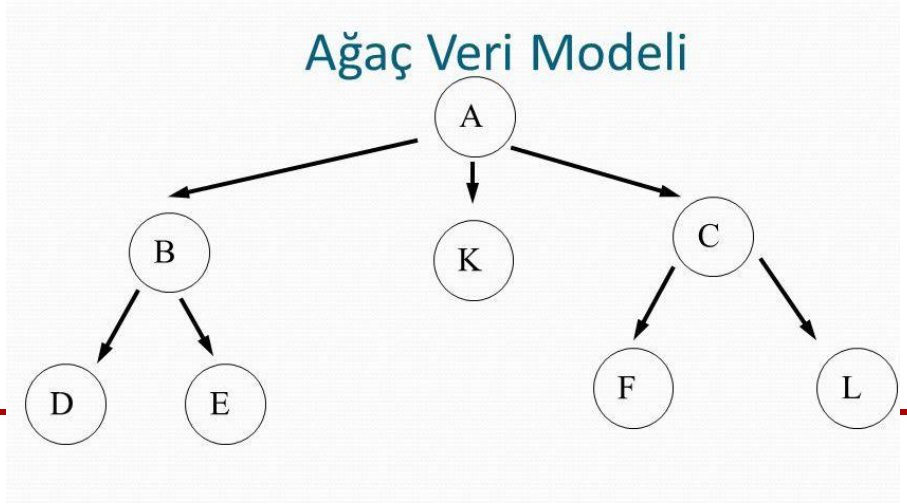
  Tamsayı (integer), Gerçel Sayı (float), karakter (char)…

Tanımlar

Veri Modeli:

- Verilerin birbiriyle ilişkisel veya sırasal durumunu gösterir.
- Problemlerin kolay çözümü için veri modeli yöntemleri kullanılır.





Tanımlar

Problem Çözme:

- Böl ve Yönet Yaklaşımı:
- Büyük problemler küçük parçalara ayrılır. Küçük problemlerin çözümü daha kolaydır.

- Çevrimli/Rekürsif Yaklaşım:
- Algoritma içinde belirli bir kodun tekrarlanmasıdır.
- Çevrimli => for, while gibi döngülerin kullanımı
- Recursive => Kodun tamamının veya kod parçasının tekrarlanması

Sayı Sistemleri ve Bilgi İfadeleri

Bit:

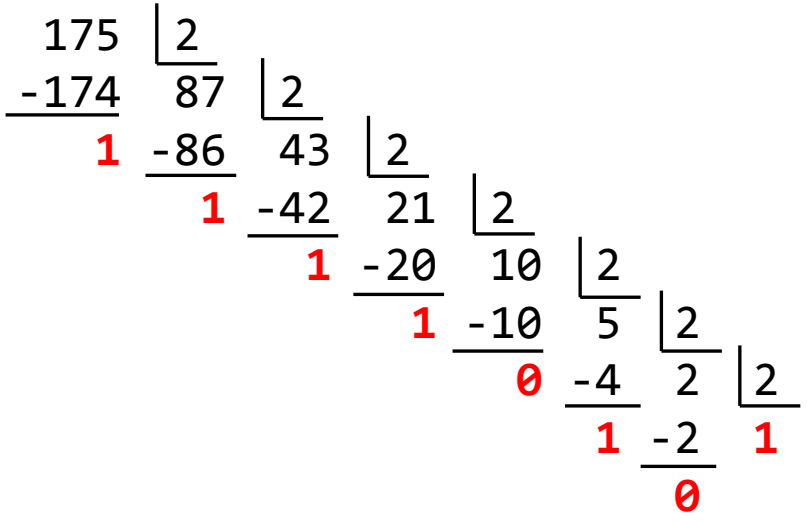
- Elektronik cihazın çıkışında gerilim varsa => mantıksal 1 yani TRUE
  Çıkışındaki gerilim seviyesi 0V ise => mantıksal 0 yani FALSE

- Bu nedenle ikili sayı sistemini temel alarak veri işler ve saklarlar.
- En küçük bellek birimi sadece 0 ve 1 değerlerini alabilen ikili sayı sisteminde bir basamağa denk gelen Bit'tir.
- 1 Bayt (Byte) = 8 Bit
- 1024 Byte = 1 KiloByte (KB)
- 1024 KB = 1 MegaByte (MB)
- 1024 MB = 1 GigaByte (GB)
- 1024 GB = 1 TeraByte (TB)

Sayı Sistemleri ve Bilgi İfadeleri

2'li (Binary) Sayı Sistemi:

- 1 bayt (8 bit) veri 2^8 yani 256 farklı değer alabilir
- $(00000000)_2 = (0)_{10}$ $(11111111)_2 = (255)_{10}$
- Örn: $(175)_{10} = (?)_2$ **10101111**



- Örn: $(10110100)_2 = (?)_{10}$ **180**

Sayı Sistemleri ve Bilgi İfadeleri

Onaltılı (hexadecimal) Sayı Sistemi:

- İkili sayı sisteminin okunabilirliğinin zor olması nedeniyle 16'lı sayı sistemi verilerin temsil edilmesinde sıklıkla kullanılır. 16'lı sayı sistemindeki her basamak 4 biti ifade eder ($2^4 = 16$).

Decimal	Hexadecimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111

Decimal	Hexadecimal	Binary
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Sayı Sistemleri ve Bilgi İfadeleri

- n bit ile temsil edilebilecek en büyük sayı = 2^n-1'dir.

  n = 4 ise 15, n = 8 ise 255, n = 16 ise 65535, …
- Örn: 1,000,000 (1 milyon) sayısını 2'li sayı sisteminde temsil edebilmek için kaç bit gereklidir?

  2^n-1 >= 1,000,000 olması gereklidir. Bu durumda n değeri en az 20 olmalıdır (2^{10} = 1024 olduğuna göre 2^{20} = 1024^2=1,048,576)
- Bilgisayarda sayılar genellikle 1 bayt, 2 bayt, 4 bayt veya 8 bayt gibi büyüklüklerde saklanır. 3 bayt ya da 5 bayt gibi veri büyüklükleri programlama dillerinde yoktur. 1 milyon sayısı için 2 bayt (16 bit) yetmeyeceği için 4 bayt (32 bit) bir veri büyüklüğü seçmeliyiz.

Sayı Sistemleri ve Bilgi İfadeleri

Binary Sayıların Tümleyeni:

- İkili sayı sisteminde bir sayının tümleyenini almak o sayının 0 olan bitlerini 1, 1 olanları da 0'a çevirerek gerçekleştirilir.
- Tersini veya değilini almak da diyebiliriz.
- Sayını kendi ve tersi (tümleyeni) toplanırsa tüm bitler 1 olur
- Örn:

  Sayımız : $(10110100)_2 = (180)_{10}$

  Tümleyeni : $(01001011)_2 = (75)_{10}$

  Toplamları : $(11111111)_2 = (255)_{10}$
- Bulunan tümleyen 1'e göre tümleyendir. Buna 1 eklenirse bulunacak olan 2'ye göre tümleyen olur => $(01001100)_2 = (76)_{10}$

Sayı Sistemleri ve Bilgi İfadeleri

Binary Sayıların Negatifi:

- Farklı program dilleri farklı yöntemler kullanmaktadır.
- C program dilinde negatifini bulmak için o sayının 2'ye tümleyeni hesaplanır.
- Programlama dillerinin büyük çoğunluğu bu yöntemi kullanır.
- Bazıları en soldaki (baştaki) biti işaret biti olarak kullanır.

  (1 => negatif, 0 =>pozitif)
- Bazıları ise 1'e göre tümleyeni negatif kabul eder.

Sayı Sistemleri ve Bilgi İfadeleri

BCD (Binary Coded Decimal):

- Ondalık sayı sistemindeki sayıların her basamağının 4 bit olarak kodlanmasıdır.
- Örn: 4859 sayısının BCD karşılığı nedir?

0100 1000 0101 1001
4 8 5 9

- BCD kullanıldığında 16 bit ile temsil edilebilecek en büyük sayı 9999'dur. Normalde 16 bit ile 2^{16} farklı sayının (65535'e kadar) temsil edilebileceğini biliyoruz. BCD 4 bit ile temsil edilebilecek 16 farklı durumdan 10 tanesini kullanabildiği için böyle bir kayba neden olmaktadır.

Sayı Sistemleri ve Bilgi İfadeleri

ASCII (American Standard Code for Information Interchange):

- Bilgisayarda metin türü verilerin işlenebilmesi veya saklanabilmesi için büyük ve küçük harflerin, noktalama işaretlerinin ve klavyedeki diğer yardımcı karakterlerin ikili kodlar ile ifade edilmesi gereklidir.
- 1963 yılında harflerin, noktalama işaretlerinin ve diğer yardımcı kısaltmaların ikili kodlar ile ifade edildiği standart hale getirilmiş tablo oluşturulmuştur.

ASCII Code Chart

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Sayı Sistemleri ve Bilgi İfadeleri

Genişletilmiş ASCII Kodu:

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Source: www.LookupTables.com

Sayı Sistemleri ve Bilgi İfadeleri

UNICODE:

- 256 karakterlik ASCII kod tablosunda Yunan, Kiril, Arap ve Japon alfabesi gibi farklı alfabelere ait karakterlerin olmadığı (Türkçedeki Ş, ş, Ğ, ğ, İ ve ı karakterleri de yok) dikkatinizi çekmiştir.
- 90'lı yıllara kadar bu alfabelere özgü farklı yazı tipleri (font) geliştirilerek bu soruna çözüm bulunmuş, 1991 yılında yayınlanan ve her elemanın 16-bit bir kod ile temsil edildiği 65536 farklı eleman içerebilen Unicode (Universal Code: Evrensel Kod) ile tüm dillere ait karakterleri, matematiksel sembolleri, vs. içeren ortak bir kod tablosu oluşturulmuştur.

Sayı Sistemleri ve Bilgi İfadeleri

Veri Yapısının Değişimine Göre Kodlamanın Değişimi:

- 0100 0010 0100 0001 0100 0010 0100 0001

4	2	4	1	4	2	4	1

- Yukarıdaki bit dizisi;
  - Sözce (string) ise (ASCII'ye göre): B A B A
  - BCD (Binary Coded Decimal) ise: 4 2 4 1 4 2 4 1
  - 16-bit tam sayı ise: 16961 16961

BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 2– Algoritma Tasarımı ve Akış Diyagramları

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

09/03/2021

Konya

Program Oluşum Süreci

ADIM-1) Problemin Belirlenmesi

ADIM-2) Problem Hakkında Veri Toplanması

ADIM-3) Verilerin Analiz Edilmesi

ADIM-4) Algoritma / Akış Diyagramı / Sözde Kod

ADIM-5) Herhangi bir Programlama Dilinde Kodlama

ADIM-6) Programın Test Edilmesi

ADIM-7) Lisanslama- Paketleme

Algoritma

Verilen bir problemin bilgisayar ortamında çözülmesi için yapılması gereken işlemlerin yapılış sırasının belirlenmesi ve adım adım ortaya koyulmasıdır.



Algoritma sözcüğü Ebu Abdullah Muhammed İbn Musa el Harezmi adındaki Türkistan'lı alimden kaynaklanır.

Bu alim 9. yüzyılda cebir alanındaki **algoritmik** çalışmalarını kitaba dökerek matematiğe çok büyük bir katkı sağlamıştır. "Hisab el-cebir ve el-mukabala kitabı dünyanın ilk cebir kitabı ve aynı zamanda ilk algoritma koleksiyonunu oluşturur.

Latince çevirisi Avrupa'da çok ilgi görür - alimin ismini telaffuz edemeyen Avrupalılar "algorizm" sözcüğünü "Arap sayıları kullanarak aritmetik problemler çözme kuralları" manasında kullanırlar. Bu sözcük daha sonra "algoritma"ya dönüşür ve genel kapsamda kullanılır.

https://tr.wikipedia.org/wiki/Algoritma

Algoritmada Olması Gereken Özellikler:

- **Giriş/Çıkış:** Algoritmanın üzerinde işlem yapması için veri girişi sağlanmalı ve gerçekleştirilen işlemler bir sonuç üretmelidir.

- **Etkinlik:** Gereksiz tanım ve tekrarlar içermemeli ve etkin sonuçlar üretebilmelidir. Gerektiğinde diğer algoritmalar içinde de kullanılabilmelidir.

Algoritmada Olması Gereken Özellikler:

- **Sonluluk:** Belirli sayıda adımdan oluşmalı, başlangıç ve bitiş noktası belli olmalıdır. Açıkta kalan bir durum, düşünülmeyen bir ihtimal kalmamalıdır.

- **Kesinlik:** İşlem sonuçları kesin olmalı ve tekrar yürütüldüğünde aynı sonucu sağlayabilmelidir.

- **Başarım:** Başarımı iyi olacak şekilde tasarlanmalı; olabildiğince bellek gereksinimi ile çalışma süresi dengeli tutulmalıdır.

Örnek:

1-Klavyeden girilen sayıyı oku.

2-Sayının karesini hesapla.

3-Sonucu ekrana yazdır.

Örnek:

1-Klavyeden girilen N sayısını oku.

2-T=0; X=1 tanımla.

3-T=T+X işlemini yap.

4-X değerini 1 arttır.

5-Eğer X<=N ise 3'e git.

6-T değerini yazdır.

Yandaki algoritma ne işe yapar?

*(Klavyeden girilen bir N değerine kadar olan tamsayıları toplayan ve sonucu ekrana yazan bir algoritma)

Sözde kod (Pseudo Code):

- Algoritmanın yarı programlama dili kuralları yarı konuşma dili şeklinde tanımlanmasıdır.

- Sözde kodlar, programlama mantığı altında, «eğer, iken» gibi koşul kelimeleri ve >, =, < gibi ifadeler ile beraber yazılır.

- Bir algoritma, sözde kod ile verilirse daha kolay anlatılabilir ve daha zahmetsiz bir şekilde programlama diline uyarlanabilir.

Sözde kod (Pseudo Code)

- ✓ **Algoritma Adı:** Algoritmayı tanımlayacak anlamlı bir isim verilmelidir.
- ✓ **Yaptığı İş:** Algoritmanın ne iş yapacağını ve algoritmada kullanılacak değişkenlerin tür ve amacını belirten açıklayıcı bilgiler eklenebilir.
- ✓ **İşlem Adımları:** Adımlar işlem sırasına göre numaralandırılmalıdır.
- ✓ **Açıklamalar:** Her bir adımda gerçekleştirilen işlemler kısa notlarla açıklanabilir.

Tanımlayıcılar

Programdaki değişkenleri, sabitleri, kayıt alanlarını, özel bilgi tiplerini vb. adlandırmak için kullanılan kelimelerdir.

- Yerini tuttukları ifadelere çağrışım yapacak şekilde seçilmelidir.
- İngiliz alfabesindeki **A-Z** veya **a-z** arasındaki 26 harf ile **0-9** arası rakamlar kullanılabilir.
- Sembollerden sadece altçizgi «_» kullanılabilir.
- Tanımlayıcı isimleri harfle veya altçizgiyle başlayabilir.
- Tanımlayıcı ismi, rakamla başlayamaz veya sadece rakamlardan oluşamaz.

```
Algoritma BuyuguBul;

/* Bu algoritma iki sayı okur ve büyük olanını bulup ekrana yazdırır. */

1- Oku (Sayı A);              /* Birinci sayıyı oku. */
2- Oku (Sayı B);              /* İkinci sayıyı oku. */
3- Karşılaştır (A>B?)         /* Sayıları karşılaştır. */
        Evet => EB=A;
        Hayır => EB=B;
4- Yaz(EB);                   /* Büyük sayıyı yazdır. */
5- Dur;
```

İki sayının toplamı

Algoritma

1. BAŞLA
2. Birinci sayıyı gir
3. İkinci sayıyı gir
4. İki sayıyı topla
5. Sayıların toplam değerini yaz
6. BİTİR

Sözde Kod

Toplam için T, birinci sayı için X, ikinci sayı için Y seç

1. BAŞLA
2. X değerini OKU
3. Y değerini OKU
4. T = X + Y
5. T değerini YAZ
6. BİTİR

Üçgenin alanını hesaplayan algoritma

1. BAŞLA
2. Taban değerini gir
3. Yükseklik değerini gir
4. Taban ile yüksekliği çarp ve sonucu ikiye böl
5. Çıkan sonucu yaz
6. BİTİR

Sözde kodu nasıl yazarız?

Taban için t, yükseklik için y, alan için A seç

1. BAŞLA
2. t değerini OKU
3. y değerini OKU
4. A = (t * y)/2
5. A değerini YAZ
6. BİTİR

Akış Şeması/Diyagramı:

Yapılacak bir işe ya da yazılacak bir programa ait algoritmanın şekilsel/grafiksel olarak gösterilmesidir.

Bu amaçla dikdörtgen, elips gibi simgesel işaretler kullanılmaktadır.

Akış Şeması Simgeleri



Başla/Bitir

Programın nereden başlayacağını ve nerede sonlanacağını belirtir. Her algoritmada yalnız bir başlangıç noktası olabilir.



Giriş



Klavye

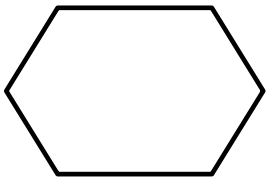
Klavye, kart okuyucu gibi birimler üzerinden programa veri/bilgi aktarmak için kullanılır. Giriş simgesinin içine verilerin saklanacağı değişkenlerin isimleri yazılır. Böylece simgeye bakınca kaç adet veri girileceği de belli olur.

Akış Şeması Simgeleri



Çıkış/Çıktı

Ekrana yazdırmak, yazıcıdan çıktı almak gibi işlemlerde kullanılır. Çıkış simgesinin üstüne nelerin çıkış birimine gönderileceği yazılır.



Döngü

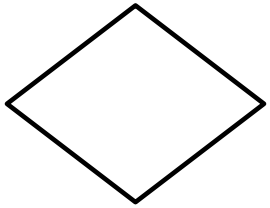
Bir işin ya da bir grup işin tekrarlanması gerektiğinde kullanılır. Koşula bağlı döngüler yazmak da mümkündür. Döngü simgesi içine döngü sayacı, sayaç artırımı gibi değerler açıkça yazılır.



İşlem

Aritmetik işlemler, atama işlemleri ya da deyimler yazılabilir.

Akış Şeması Simgeleri



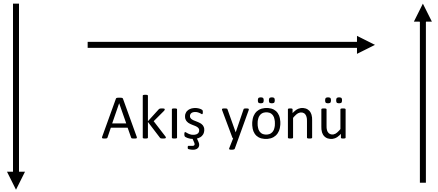
Karar/Karşılaştırma

Koşula bağlı olarak farklı işlemler yapılacağında kullanılır, koşulu sağlama durumuna göre iki farklı yöne dallandırılabilir.



Fonsiyon çağırma

Daha önce yazılmış algoritmaların, o anda kullanılan algoritma içine tamamiyle yazılmadan kullanılmasını sağlar. Simgenin içine çağırılacak fonksiyonun adı ve parametreleri yazılır.



Akış yönü

İşlem bittikten sonra program akışının nereye dallanacağını/yöneleceğini belirtir.

Bir bilgisayar programının geliştirilmesinde kullanılan programlama dili ne olursa olsun bu programların akış şemalarında genel olarak üç basit mantıksal yapı kullanılır:

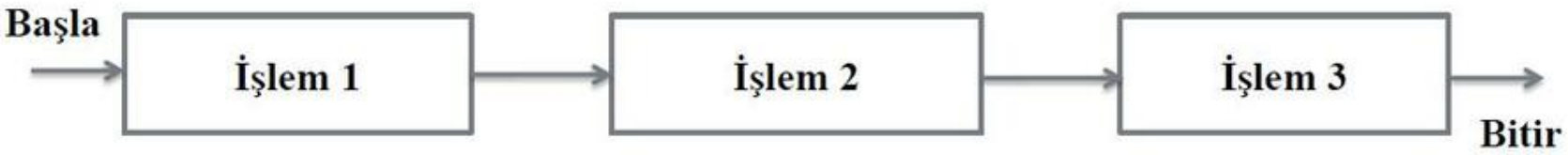
1. SıralıYapı

2. Karar Verme Yapısı

3. Tekrarlı Yapı

Sıralı Yapı

- Sıralı yapı, hazırlanacak programdaki her işlemin mantık sırasına göre nerede yer alması gerektiğini vurgular. Bu yapı sona erinceye kadar ikinci bir işlem başlayamaz.

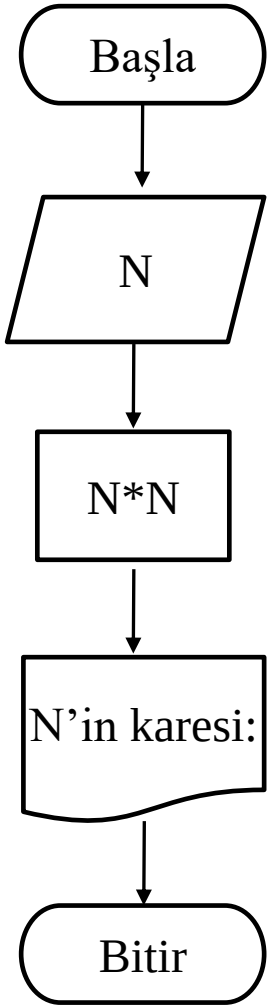


Örnek:

1-Klavyeden girilen sayıyı oku.

2-Sayının karesini hesapla.

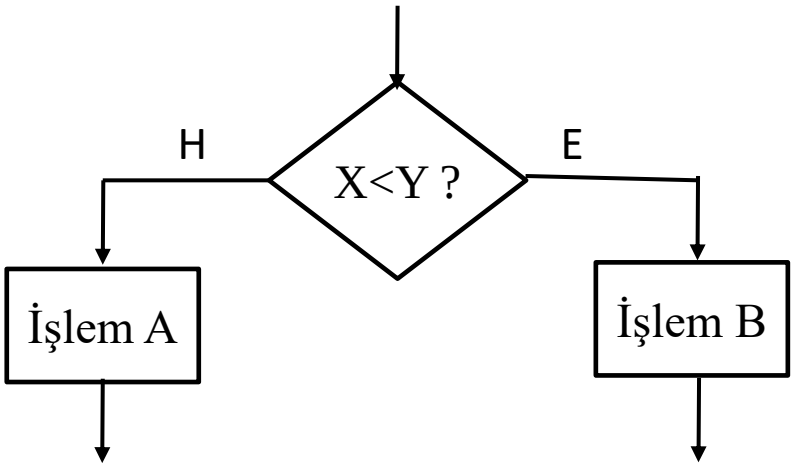
3-Sonucu ekrana yazdır.

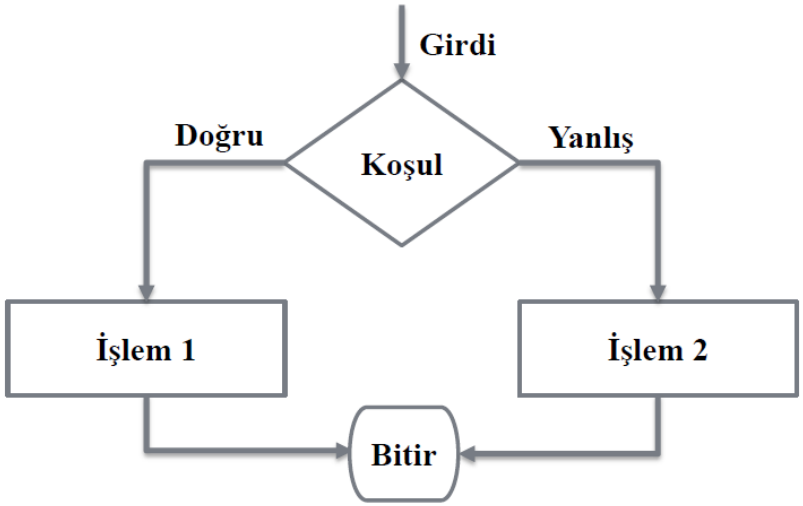


Karar Verme Yapısı / Koşullu İfadeler

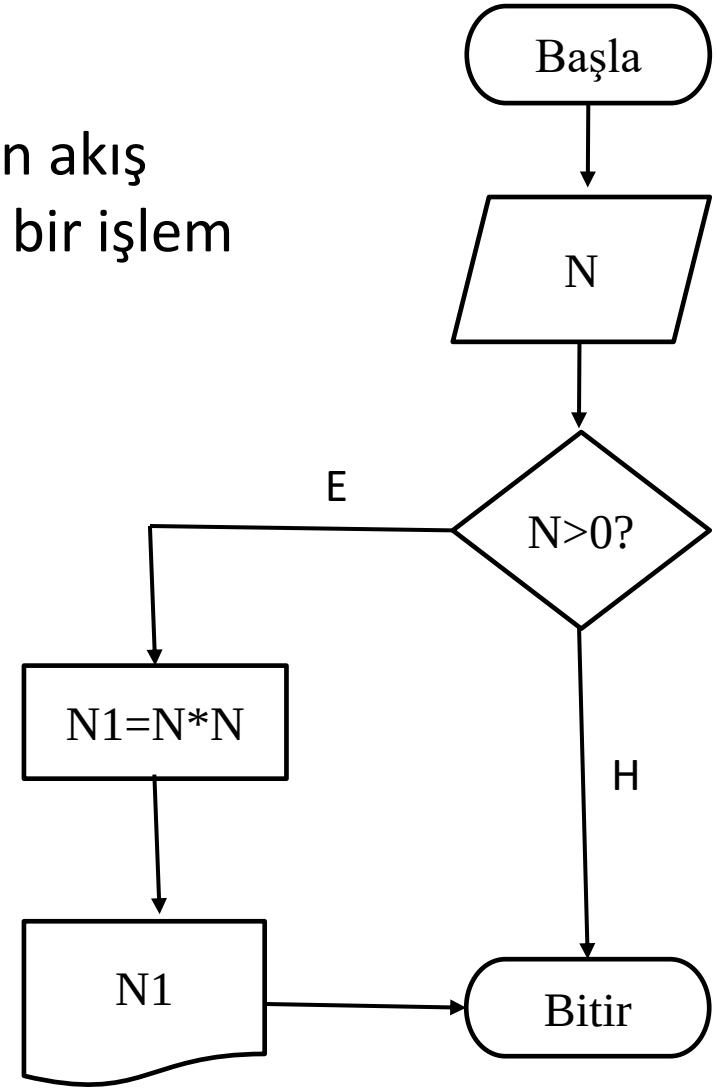
- Algoritmada verilen adımlar genellikle sırayla işlenir. Ancak bazı koşullara göre bu sıralamanın değiştirilmesi gerekebilir. Böyle durumlarda koşullu dallanma kullanılır.

- Baklava dilimi şeklindeki koşullu dallanma simgesinin içine koşul yazılır ve bu koşulun sonucuna göre iki farklı yöne dallanma söz konusu olur.





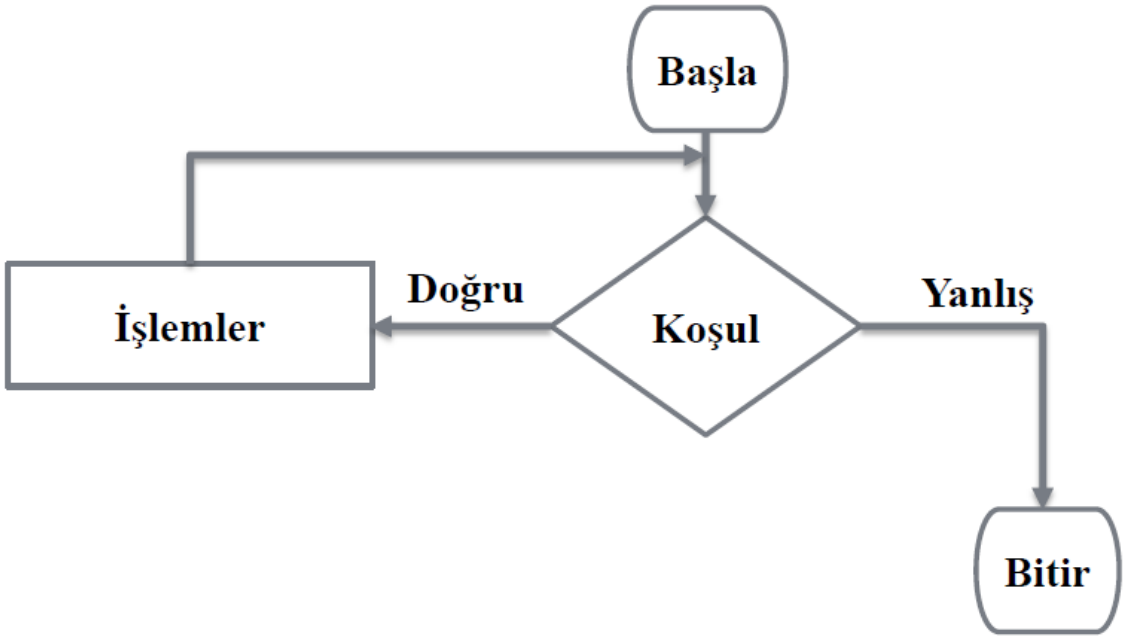
Yanda verilen akış şeması nasıl bir işlem sağlar?



* Klavyeden girilen pozitif sayıların karesini alıp ekrana yazdırır.

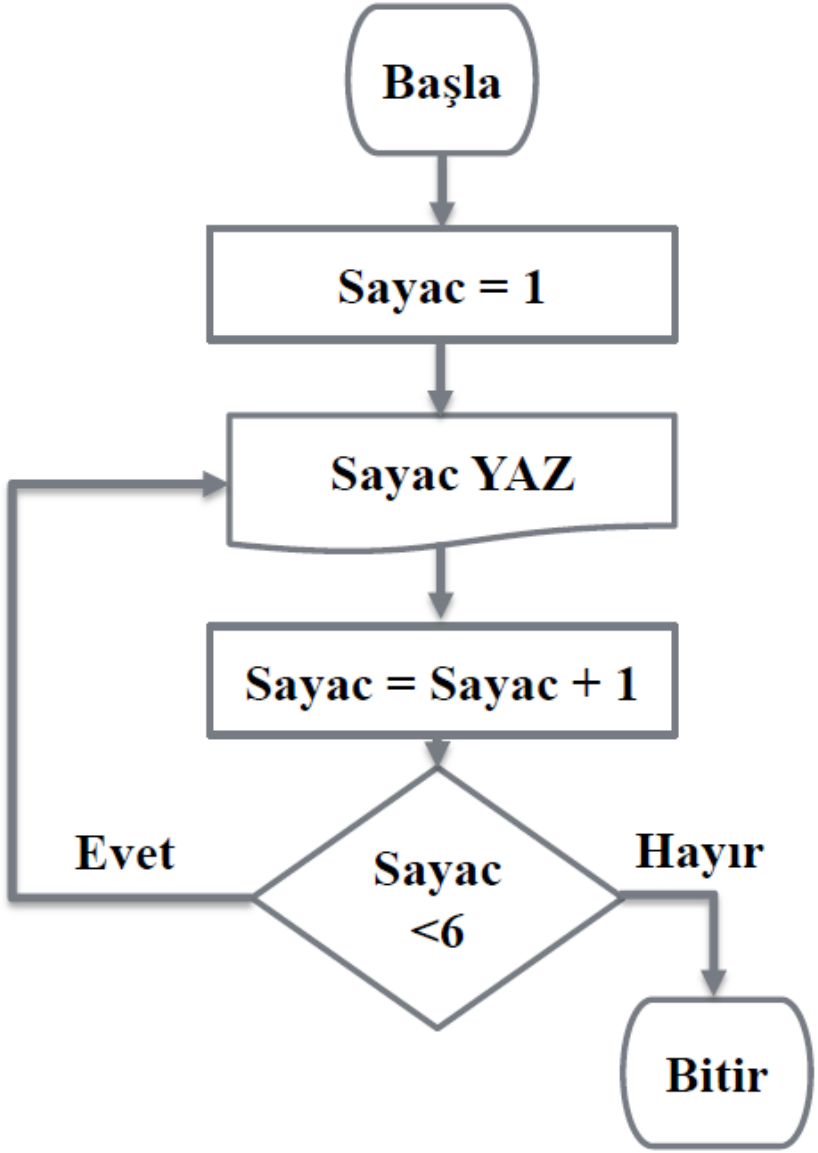
Tekrarlı Yapı / Döngü

- Algoritma içinde, bazı satırlar tekrarlı şekilde işlem görüyorsa, bir döngü söz konusudur. Döngülere belirli bir koşul geçerli olduğu sürece devam eden eylemleri tanımlamak için başvurulur.



ÖRNEK: 1’ den 5’ e kadar sayıların ekrana yazdırılması

1. BAŞLA
2. Sayac = 1
3. Sayac değerini YAZ
4. Sayac = Sayac + 1
5. Eğer Sayac < 6, GİT 3
6. BİTİR



1. BAŞLA
2. Sayac = 1
3. Toplam = 0
4. Eğer Sayac > 10, GİT 8
5. Toplam = Toplam + Sayac
6. Sayac = Sayac + 2
7. GİT 4
8. BİTİR

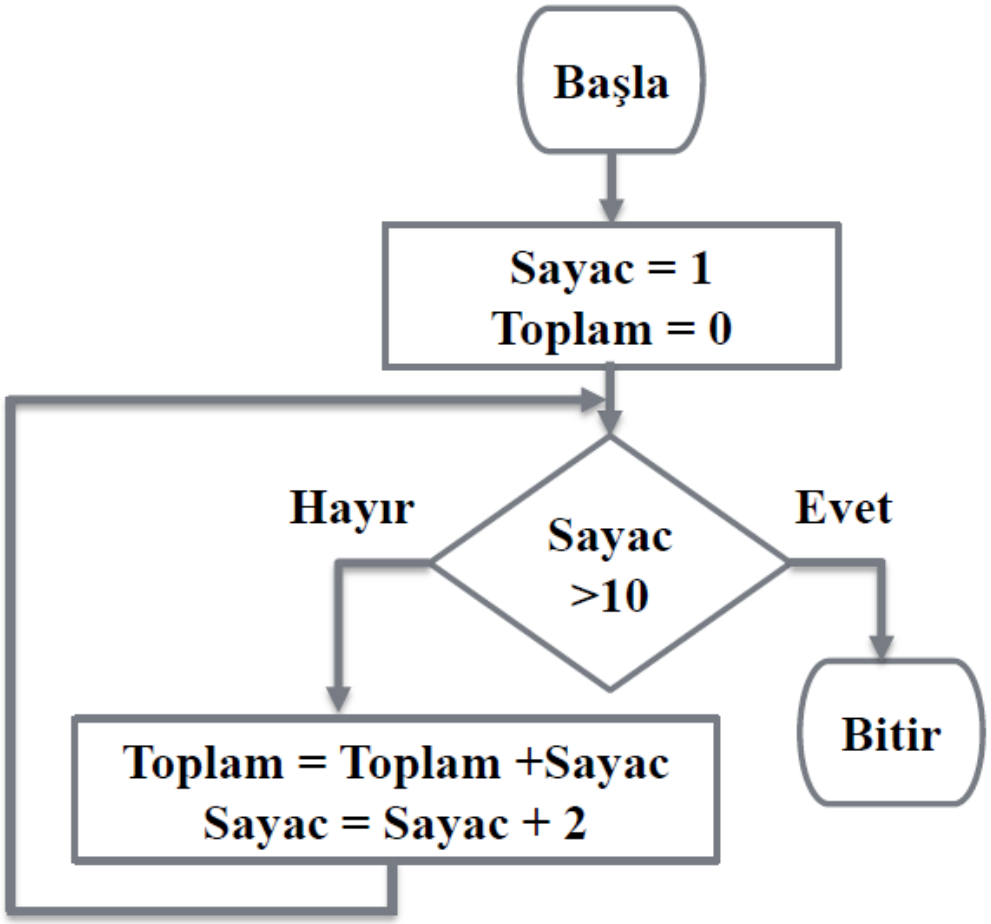
ÖRNEK: 1-10 arasındaki tek sayıları toplayan algoritma

Değişken İzleme Tablosu

Eski Sayac	Eski Toplam	Yeni Toplam	Yeni Sayac
1	0	1	3
3	1	4	5
5	4	9	7
7	9	16	9
9	16	25	11

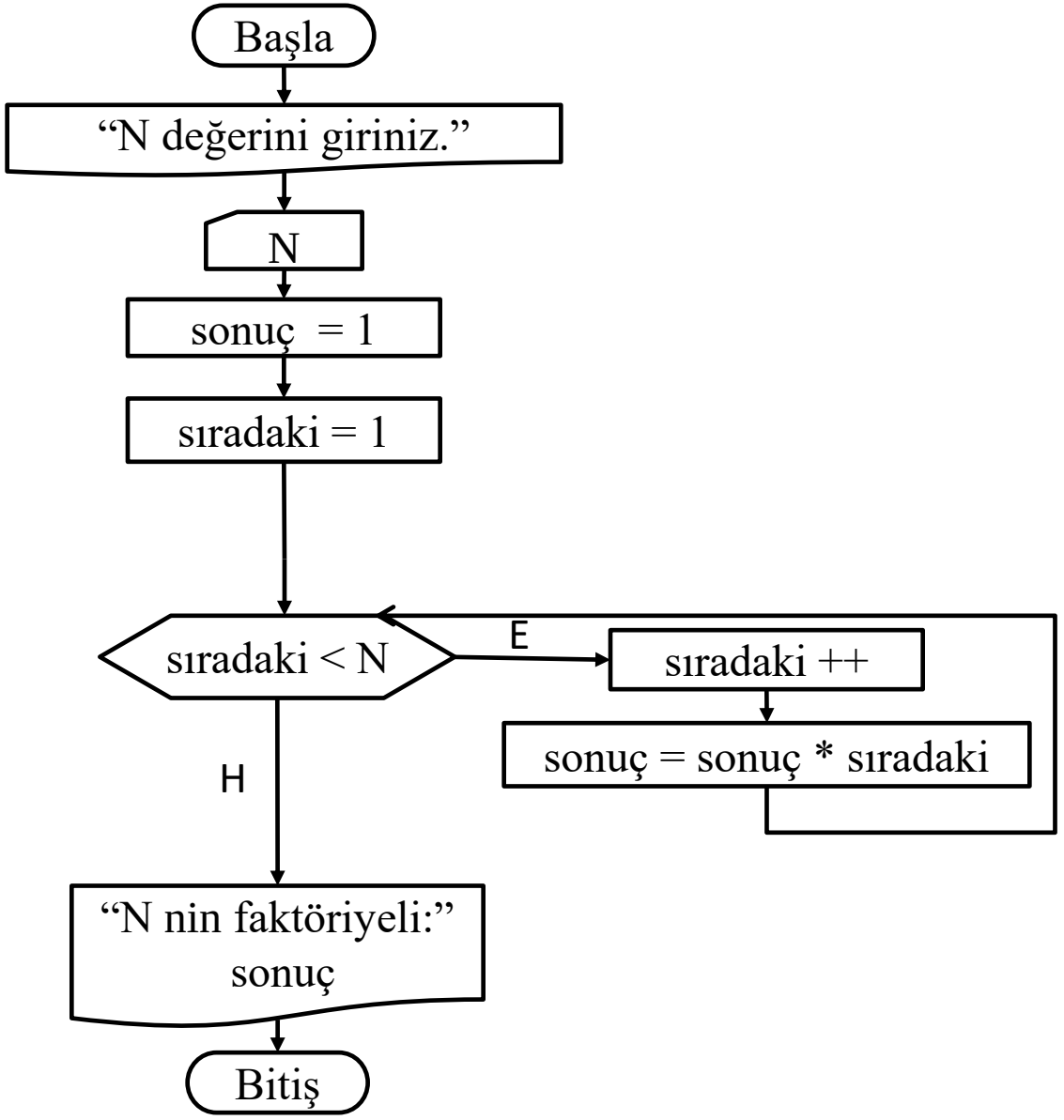
1. BAŞLA
2. Sayac = 1
3. Toplam = 0
4. Eğer Sayac > 10, GİT 8
5. Toplam = Toplam + Sayac
6. Sayac = Sayac + 2
7. GİT 4
8. BİTİR

ÖRNEK: 1-10 arasındaki tek sayıları toplayan algoritma



Klavyeden girilen bir sayının faktöriyelini hesaplama (N! = 1 * 2 * 3 *…* N)

1. Başla.
2. Kullanıcıdan bir sayı girmesini iste.
3. Girilen sayıyı oku.
4. «sonuç» değişkenine 1 ata.
5. «sıradaki» değişkenine 1 ata.
6. «sıradaki» değişkeni N sayısından küçük olduğu sürece aşağıdaki adımları tekrar et:
   1. «sıradaki» değişkenini 1 artır.
   2. «sıradaki» ve «sonuç» değişkenlerini birbiri ile çarp ve bu çarpmadan elde edilen sonucu «sonuç» değişkenine ata.
7. "sonuç" değişkeninin aldığı en son değeri ekrana yazdır.
8. Son.



$$ax^2 + bx + c = 0$$

$$\Delta = b^2 - 4ac$$

$\boldsymbol{\Delta > 0}$

$$x_1 = \frac{-b - \sqrt{\Delta}}{2a}$$

$$x_2 = \frac{-b + \sqrt{\Delta}}{2a}$$

$\boldsymbol{\Delta = 0}$

$$x_1 = x_2 = \frac{-b}{2a}$$

$\boldsymbol{\Delta < 0}$ → Reel kök yoktur.

1. Başla
2. a,b,c katsayılarını gir.
3. $\Delta = b^2 - 4ac$
4. $\Delta > 0$ ise

$$x_1 = (-b - \sqrt{\Delta})/2a$$
$$x_2 = (-b + \sqrt{\Delta})/2a$$

7'ye git.

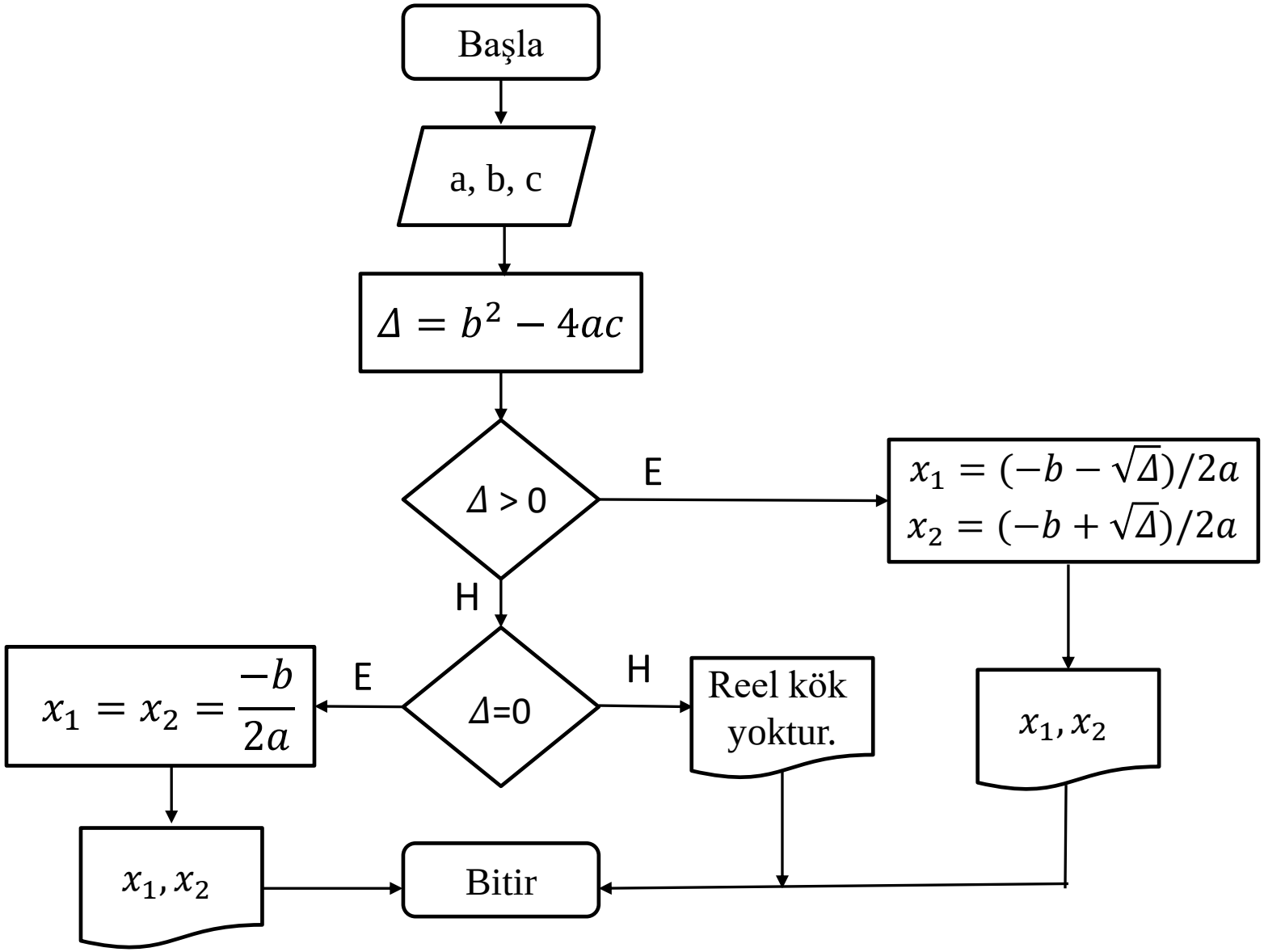
5. $\Delta = 0$ ise

$$x_1 = x_2 = -b/2a$$

7'ye git.

6. «Reel kök yoktur» yaz , 8'e git.
7. x_1 , x_2'yi yaz.
8. Bitir

İkinci dereceden denklemlerin köklerini bulan akış şeması



ÖDEV:

30 kişilik bir sınıftaki öğrencilerin Bilgisayar Programlama dersinden aldıkları vize notlarının tek tek ekrandan girilmesini ve bu notların Konya Teknik Üniversitesi lisans yönergesine göre harf karşılığının sıralı bir şekilde ekrana yazılmasını sağlayan programa ait algoritmayı yazınız ve akış şemasını oluşturunuz.

BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 3– C Fonksiyonlarına Giriş ve Değişkenler

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

16/03/2021

Konya

C Dilinin Özellikleri

- C dili donanımdan bağımsız (machine independent) ve taşınabilirdir (portable).
- Taşınabilirlik, aynı programın farklı donanımlarda ve farklı işletim sistemlerinde işletilebilirliğini ifade eder.
- Bir dilin verimliliği, hızlı fakat fazla yer kaplamayan yazılımlar geliştirmeye olanak sağlaması ile ölçülür.
- C Programlama dili verimli, basit, fakat güçlü yapısından ve bu dilde geliştirilen uygulamaların taşınabilir olmasından dolayı yaygın olarak kullanılmaktadır.

C Programlarının Temel Yapısı

Bir C programı iki temel bölümden oluşur;

- Önişlemci Direktifleri (Preprocessor Directives)
- Ana Fonksiyon (Main Function)

Önişlemci Direktifleri

- Önişlemci direktifleri # işareti ile başlar ve program derlenmeden önce C önişlemcisi tarafından işletilir.
- Her bir önişlemci direktifinin farklı bir görevi vardır.
- **#include** ve **#define** en çok kullanılan direktiflerdir.
- **#include** direktifi program içerisinde kullanılan fonksiyonlar için gerekli kodları programa dahil etmek için kullanılır.

Önişlemci Direktifleri

- Tasarladığımız programda ekrana çıktı yazdırmak istiyorsak; C dilinin standart bir fonksiyonu olan **printf** fonksiyonunu kullanmamız gerekir.
  - **printf(“Örnek Çıktı”);**
- Ancak **printf** fonksiyonunun çalışabilmesi için **<stdio.h>** isimli başlık dosyasına ihtiyacımız olacaktır.
- Bu dosyayı programa dahil etmek için program kodunun en tepesine

#include<stdio.h> komut satırı yazılır.

- C dilinde **.h** uzantılı dosyalara başlık dosyası (**HeaderFile**) adı verilir.
- **stdio.h** başlık dosyası standart giriş çıkış (**S**tandard **i**nput-**o**utput) işlemleri için gerekli kodları içerir.

Bazı Kütüphane Başlık Dosyaları

<ctype.h>	Karakter Fonksiyonları
<locale.h>	Konumlandırma Fonksiyonları
<math.h>	Matematik Fonksiyonları
<setjmp.h>	Atlama Fonksiyonları
<signal.h>	Sinyal Yönetme Fonksiyonları
<stdarg.h>	Değişken argümanları Yönetme Fonksiyonları
<stdio.h>	Standart Giriş/Çıkış Fonksiyonları
<stdlib.h>	Standart Fayda Fonksiyonları
<string.h>	String Yönetme Fonksiyonları
<time.h>	Zaman Fonksiyonları

Bazı Kütüphane Başlık Dosyaları

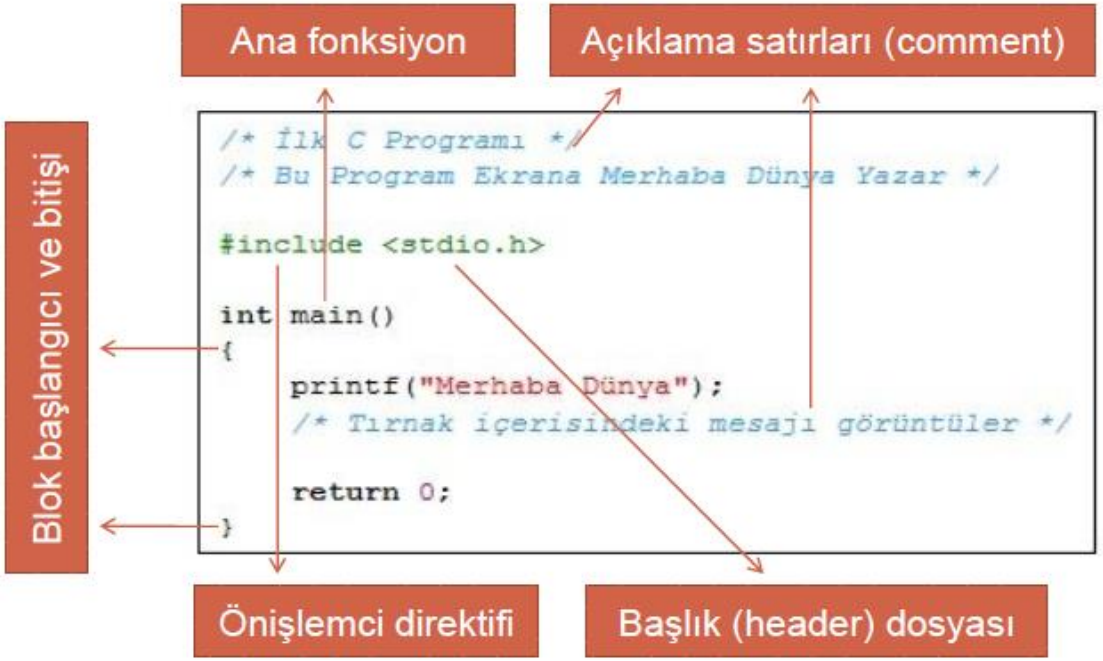
- <stdlib.h>
  - exit: Çalışan programı durdurur
  - qsort: Diziyi sıralar
  - div: Tamsayıyı böler
- <stdio.h>
  - printf(): character, string, float, integer, octal ve hexadecimal türündeki değişkenlerin değerlerini ekrana yazdırır
  - getc(): Dosyadan karakter okur
  - fopen(): Okuma, yazma gibi değişik işlemleri gerçekleştirmek için bir dosyayı açar
- <math.h>
  - sin(): sinüs değerini hesaplar
  - asinh(): hiperbolik arc sinüs değerini hesaplar
  - sqrt(): bir sayının karekök değerini hesaplar

Genel giriş-çıkış kütüphanesi

- Geliştirilen bir programda kullanıcıdan bilgi (veri) almak ve kullanıcıya bilgi göstermek en temel kullanıcı etkileşim işlemleridir.
- C dilinde bütün giriş-çıkış operasyonları giriş ve çıkış fonksiyonları (input-output functions) ile gerçekleştirilir.
- C de kullanılan standart giriş-çıkış fonksiyonları, **printf()** ve **scanf()** dir.
- Bu fonksiyonlar C standart giriş-çıkış başlık dosyasında **stdio.h** bulunurlar.
- Giriş-çıkış fonksiyonlarını program içerisinde kullanabilmek için **stdio.h** başlık dosyasının programa dahil edilmesi gerekir.
- #include<stdio.h>

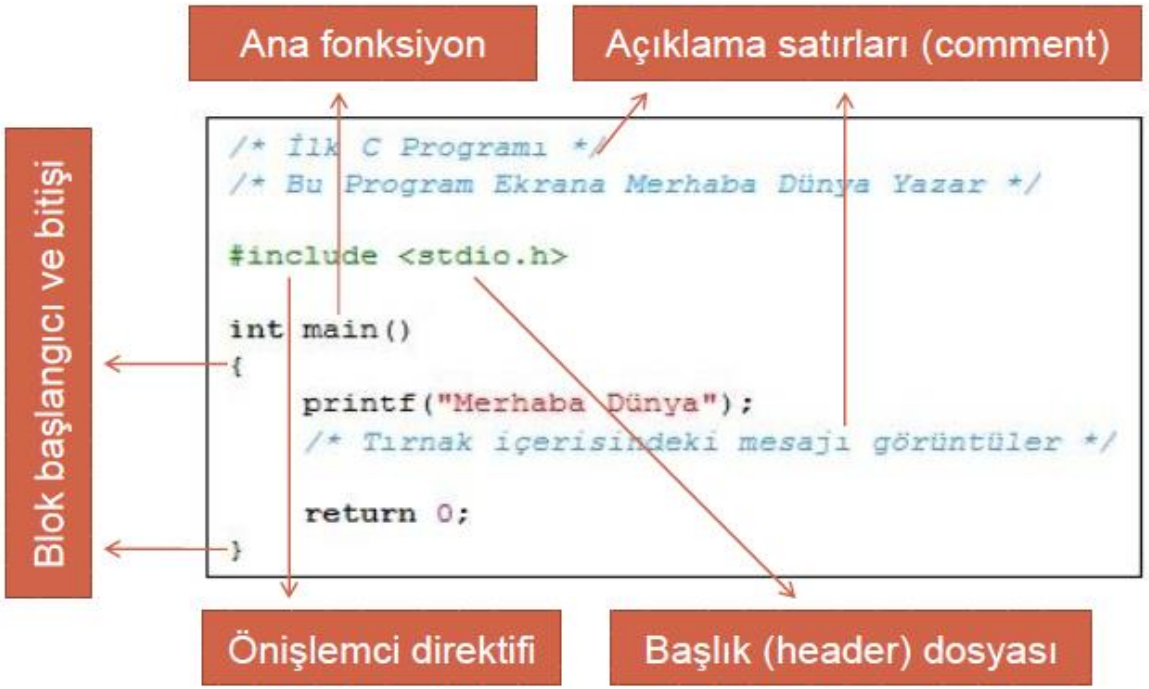
Main Fonksiyonu

- Hemen hemen bütün C programları birden fazla fonksiyondan oluşur.
- main() bütün C programlarında bulunması gereken programın ana fonksiyonu yani gövdesidir.
- İlk çalıştırılacak olan fonksiyondur.



Main Fonksiyonu

- Fonksiyonda çalıştırılacak ifadeler (kod satırları) { } küme parantezleri içinde yazılırlar.
- Her parantez çiftinin oluşturduğu yapılara kod blokları denir.
- Bir kod bloğu içerisinde program içerisinde kullanılacak değişkenler ve gerçekleştirilecek işlemleri yerine getirecek komutlar bulunur.



printf() fonksiyonu

- Değişkenlerin değerlerini, hesaplanan sonuçları yada mesajlar ekranda göstermek için kullanılır.
- printf() fonksiyonu, fonksiyon ismi ve parantezler içindeki parametreler olmak üzere iki kısımdan oluşur.
- printf() fonksiyonu, parametre olarak görüntülenecek bilginin hangi biçimde görüntüleneceğini bildiren çıktı metin formatını ve bu formatın içinde yazdırılacak olan değişkenler listesini alır.
- Genel kullanım formatı aşağıdaki gibidir.
  - printf(“Merhaba Dünya”);
  - printf(çıktı metin formatı, değişken listesi)
  - printf(“Hesaplanan Yaşınız: %d dir”, yas);

printf() fonksiyonu

- printf(“Hesaplanan Yaşınız: %d dir”, yas);
- printf çıktı formatındaki % karakterinin dışındaki tüm karakterleri ekrana yazar.
- % karakterini gördüğünde bunun sağındaki karakteri yazdırılacak değişkenin format karakteri olarak ele alır.

Format Karakterleri

Format Karakterleri	Anlamı
d	Tamsayılar için (onluk sistemde)
i	Tamsayılar için (onluk, sekizlik ve onaltılık sistemde)
lx	Uzun tamsayıları onaltılık (hex) sistemde
ld	Uzun tamsayılar için (onluk sistemde)
o	Tamsayılar için (sekizlik sistemde)
x	Tamsayılar için (onaltılık (hex) sistemde)
X	Tamsayılar için (onaltılık (hex) sistemde büyük harfle)
u	İşaretsiz tamsayılar için
c	Char tipinde değişkenler (karakterler) için
s	Metin (string)
f	Float değişkenler için
E , e	Double ve float türlerini üstel gösterimde yazmak için
lf	Double değişkenler için

printf( ) fonksiyonu

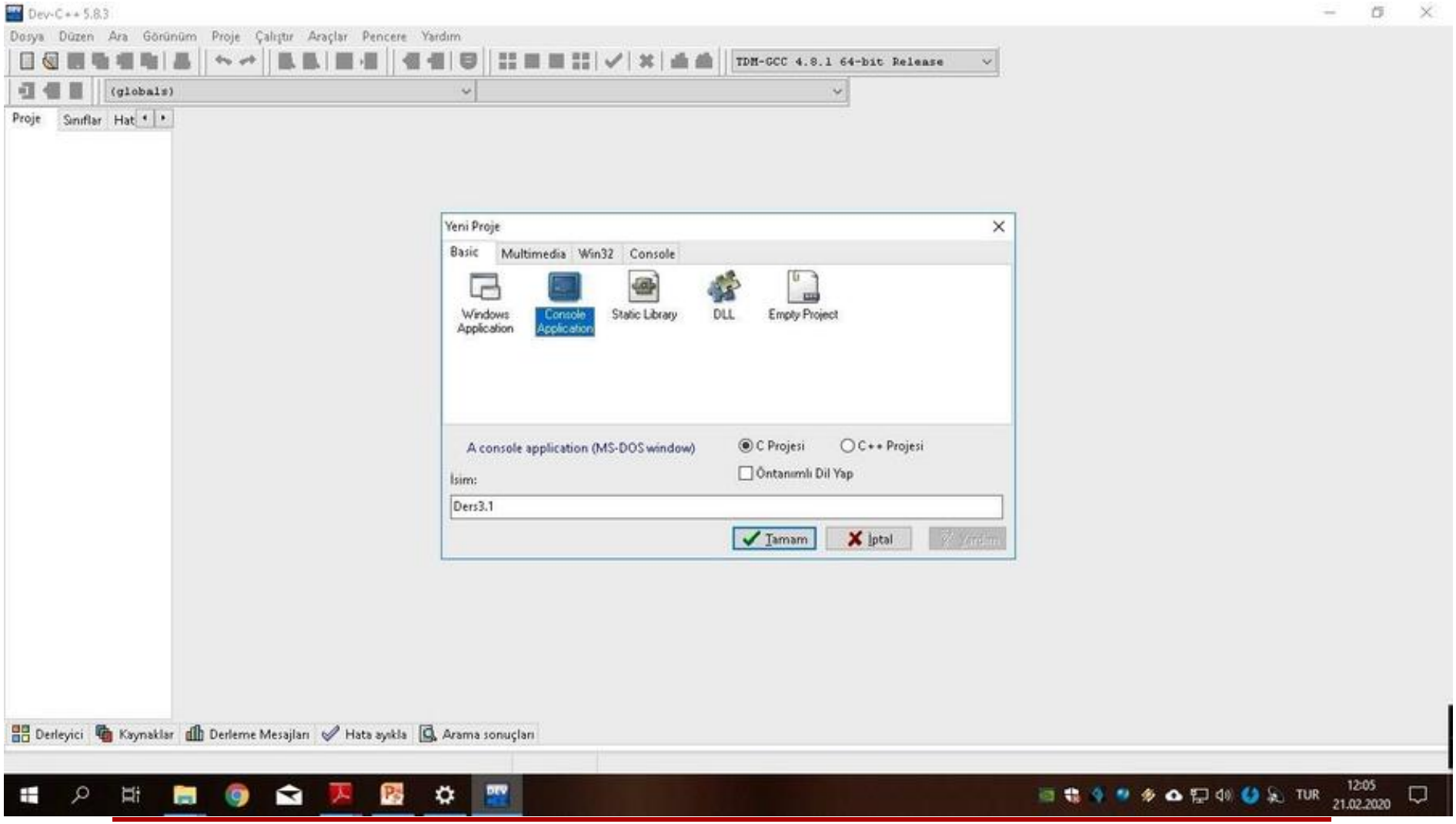
- printf("Hesaplanan \nYaşınız: %d dir", yas);
- printf çıktı formatındaki \ karakterine escape karakteri denir.
- Bu karakterden sonra gelen karakter ise escape serisini ifade eder.
- Örneğin \n ifadesi, çıktı ekranında yeni bir satıra geçilmesi gerektiğini ifade eder.

\0	null karakteri temsil eder (sonlandırıcı karakter)
\n	Yeni satır
\r	Satırbaşı
\t	Yatay sekme
\v	Düşey sekme
\f	İleri besleme
\b	Geri boşluk (space)
\a	Alarm karakteri
\"	Çift tırnak
****	Ters bölü

C dilinin genel yazım kuralları

- C dili birden fazla satırdan oluşan açıklama satırlarına izin verir.
- Bu açıklama satırları programın ne iş yaptığı hakkında bilgi verir.
- Açıklama satırları **/*** ile başlayıp ***/**ile sona erer.
- Derleyici bu satırları çalışma anında dikkate almaz.
- C'de her bir işletilebilir ifade (komut satırı) **;** ile sonlandırılır.
- Bütün anahtar kelimeler ve komutlar küçük harfle yazılır (**#define** hariç. Define ile yapılan tanımlamalarda istenirse büyük harf kullanılabilir).
- C dili büyük-küçük harf duyarlıdır. Yani; "TOPLAM", "toplam" ve "tOpLaM" kelimelerinin hepsi C derleyicisi tarafından ayrı ayrı algılanır.

İlk C Programınız



İlk C Programınız

```
#include <stdio.h>
#include <stdlib.h>

/* run this program using the console pauser or add your own getch, system("pause") or input loop */  => Yorum (Açıklama)

int main()
{
        printf("Merhaba Dunya");
        getch();
        return 0;
}
```

Değişken Tanımlamaları ve Veri Türleri

- Programda kullanılacak bütün değişkenlerin kullanılmadan önce C derleyicisine bildirilmesi gerekir.
- Değişkenleri derleyiciye bildirme işine değişken tanımlama (variable definition) denir.
- Değişken tanımlamak, değişken için bir isim seçmek ve içerisinde ne tür bir veri saklanacağını söylemekle yapılır.
- Her değişkenin ismi, veri türü ve değeri vardır.
- Değişken ismi değişken saklayacak bellek hücresine verilecek isimdir.
- Veri türü saklanan bilginin türüdür (ondalık sayı, tam sayı..).
- Değeri ise değişken içinde saklanan bilginin kendisidir.

Değişken İsimlendirme

- Programda kullanılacak değişken isimleri programcı tarafından tanımlanır.
- C dilinin kendine özgü anahtar sözcükleri, komut veya fonksiyon adları değişken ismi olarak kullanılamaz.
- C dilinin bazı Anahtar Sözcükleri (ISO/IEC 9899:1999):

auto	do	goto	return	union	_Imaginery
break	double	if	short	unsigned	
case	else	inline	signed	void	
char	enum	int	sizeof	volatile	
const	extern	long	static	while	
continue	float	register	struct	_Bool	
default	for	restrict	typedef	_Complex	

Değişken İsimlendirme

- Değişken isimleri içerisinde, a-z ve A-Z arası İngilizce harfleri, 0-9 arası rakamlar ve özel karakter olarak sadece alt çizgi (_) karakteri kullanılabilir.
- Özel karakterler (+,-,! vs.) ve Türkçe karakterler kullanılmaz.
- Maaş, öğrenci, sınıf => bunlar değişken adı olarak kullanılamaz.
- Değişken ismi rakam ile başlayamaz.
- 1.ogrenci => (yanlış) ogrenci1 => (doğru)

Veri türleri

- Veri türü, değişken içerisinde tutulan değerin tipini belirler (tamsayı, ondalık sayı, karakter, vs.).
- C dilinde dört temel veri tipi (türü) vardır.
- **char**, **int**, **float** ve **double**.
- **char:** bir harf, bir rakam veya özel bir simgeyi saklamak için kullanılan veri tipidir.
- **char** tipindeki her bir değer tek tırnak arasına yazılmalıdır
- ('A', 'z', '2', '*', ':', ' ').
- **char** tipinde tanımlanan bir değişkenin tuttuğu değer karakterin ASCII tablosundaki sayısal karşılığıdır.

Veri türleri

- **int:** -32768 ile +32768 (Dos işletim sisteminde) arasındaki tam sayı değerlerini saklamak için kullanılır.
- **float:** ondalıklı olarak verilen sayıları ifade edebilmek için kullanılır.
- **double:** ondalıklı olarak verilen çok küçük ve çok büyük sayıları ifade edebilmek için kullanılır. Float'a göre iki kat daha fazla duyarlılığa sahiptir.

Veri türleri

Tip	Sınırlar			Bellekte alanı
char	-128	...	+127	8 bit
unsigned char	0	...	+255	8 bit
signed char	-128	...	+127	8 bit
int	-32768	...	+32767 veya	16 bit veya
	-2147483648	...	+2147483647	32 bit
unsigned int	0	...	+65535 veya	16 bit veya
	0	...	+4294967294	32 bit
signed int	-32768	...	+32767 veya	16 bit veya
	-2147483648	...	+2147483647	32 bit
short int	-32768	...	+32767	16 bit
unsigned short int	0	...	+65535	16 bit
signed short int	-32768	...	+32767	16 bit
long int	-2147483648	...	+2147483647	32 bit
signed long int	-2147483648	...	+2147483647	32 bit
unsigned long int	0	...	+4294967294	32 bit
float	±3.4E-38	...	±3.4E+38	32 bit
double	±1.7E-308	...	±1.7E+308	64 bit
long double	±1.7E-308	...	±1.7E+308	64 bit veya
	±3.4E-4932	...	±1.1E+4932	80 bit

Değişken tanımlama yerleri, şekilleri ve ilk değer atama

- Standart C'de üç farklı yerde değişken tanımlanabilir.
- fonksiyonların üstünde, blokların { } içerisinde ilk sırada ve fonksiyonlarda parametre olarak.
- İlk değer ataması yapılmayan değişkenlerin değerleri (eğer main fonksiyonunun üstünde tanımlanmışsa) sayısal olanlar 0 diğerleri boş olarak belirlenir, eğer main içinde tanımlanmışsa bellekte rastgele değerler olarak belirlenir.

```
#include<stio.h>

/* Örnek Değişken tanımlamaları */

int main(){

    int a;
    a=1;          ---->  int a=1;  ---->  İlk değer atama

    int x, y, z; /* Aynı satırda birden fazla değişken tanımlanabilir. */

    char m='k';

    double n=5.05;

}
```

ders3.2.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
/* run this program using the c
int a;
float b;
double c;

int main()
{
    a=M_PI;
    printf("\n %d",a);

    b=M_PI;
    printf("\n %f",b);

    c=M_PI;
    printf("\n %lf",c);

    float x=M_PI;
    printf("\n %.12f",x);

    double y=M_PI;
    printf("\n %.12lf",y);

    float z=4.79;
    printf("\n %f",z);

    double m=4.79;
    printf("\n %lf",m);

    return 0;
}
```

- ÖRNEK: Pi Sayısı=3.141592 653589 793…
- float veri türü, virgülden sonra en fazla 6 basamak için kullanılabilir.

```
C:\Users\Kemal\Desktop\C_Kemal_Ders\Kodlar\ders3.2.exe

 3
 3.141593
 3.141593
 3.141592741013
 3.141592653590
 4.790000
 4.790000
--------------------------------
Process exited after 0.01311 seconds with return value 0
Press any key to continue . . .
```

scanf( ) fonksiyonu

- Değişkenlerin içerisine klavyeden değer atamak için kullanılır.
- Fonksiyon ismi ve parametrelerden oluşur.
- Parametre olarak, girilecek değerin hangi formatta olacağını bildiren girdi formatını ve bu formata göre girilecek değişkenler listesini alır.
- scanf fonksiyonunda dışarıdan değer girilecek bütün değişkenlerin başına & işareti konur.
- Bu işaret bellek operatörüdür, değişkenlerin tutulduğu bellek hücresinin adresini okur.
- ÖRN:

```
int t;
{
scanf("%d",&t);
}
```

Örnek

```c
#include <stdio.h>
#include <stdlib.h>

/* run this program using the console pauser or add your own getch, system("pause") or input loop */
int y;
int t,a;

int main()
{
    printf("\nUcgenin yukseklik degerini giriniz");
    scanf("%d",&y);
    printf("\nUcgenin taban uzunlugu degerini giriniz");
    scanf("%d",&t);
    a=(y*t)/2;
    printf("\nUcgenin Alani: %d 'dir",a);
    return 0;
}
```

```
Ucgenin yukseklik degerini giriniz4

Ucgenin taban uzunlugu degerini giriniz6

Ucgenin Alani: 12 'dir
--------------------------------
Process exited after 61.12 seconds with return value 0
Press any key to continue . . . _
```

Program Hataları

- Bug (Hata)
  - Programdaki hatalar
- Debugging (Hata Giderme)
  - Programdaki hataların düzeltilmesi
- Syntax (Sözdizimi) hataları
  - Programlama dili kurallarının ihlal edilmesi
  - Syntax hatalarını derleyici tespit eder. (Her zaman doğru satırı işaret etmeyebilir.)
- Run-time (Yürütme Aşaması) hataları
  - Program çalıştırıldığında bilgisayar tarafından tespit edilen hatalardır.
- Logic (mantık) hataları
  - Program algoritmasındaki hatalardır.
  - Genellikle tespiti zordur. Bilgisayar bu tip hataları bulamaz

Örnek

- Küçük harfi büyük harfe dönüştürme:
- ASCII tablosundan faydalanılabilir.
- Harf değerlerinin arasında sabit fark var.

[*] ders3.3.c

```
1   #include <stdio.h>
2   #include <stdlib.h>
3
4   /* run this program using the console pauser or
5       char harf;
6   int main() {
7
8           printf("\n Bir harf giriniz\n");
9           scanf("%c",&harf);
10          harf= harf-32;
11          printf ("\n Buyuk Harfiniz: %c",harf);
12
13      return 0;
14  }
15
16
```

ter codes 0 - 127)

```
064 @    080 P    096 `    112 p
065 A    081 Q    097 a    113 q
066 B    082 R    098 b    114 r
067 C    083 S    099 c    115 s
078 N    094 ^    110 n    126 ~
079 O    095 _    111 o    127 ⌂
```

C:\Users\Kemal\Desktop\C_Kemal_Ders\Kodlar\ders3.3.exe

```
 Bir harf giriniz
r

 Buyuk Harfiniz: R
--------------------------------
Process exited after 4.825 seconds with retu
Press any key to continue . . .
```

Ödev

- Ödev 1:

  Kullanıcıdan ekrana 4 adet tam sayı girmesini isteyin. Bu sayıların toplamını hesaplayıp, her bir sayının toplama göre yüzde değerini bulup ekrana yazdırın. Yüzdelik değerleri float türü değişken ile tutun ve bulduğunuz küsüratlı 4 değerin toplamının 100 sayısını vermesi gerektiğini unutmayın

- Ödev 2:

  Önceki hafta tasarladığınız algoritmaya uygun olarak 10 öğrencinin notlarını sıralı bir biçimde ekranda harf karşılığı olarak yazdıran program hazırlayınız.

BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 4– Operatörler-1

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

23/03/2021

Konya

Operatörler

Tüm programlama dillerinde bulunan ve kullanılan değişkenlerin değerleri üzerinde matematiksel, mantıksal ya da karşılaştırmaya dayalı işlem yapabilmek için kullanılan dil elemanlarıdır.

C dilinde kullanılan operatörler aşağıdaki gibi sınıflandırılabilir:

- ✓ Atama operatörü
- ✓ Aritmetik operatörler
- ✓ Karşılaştırma operatörleri/ilişkisel operatörler
- ✓ Mantıksal operatörler
- Bit-tabanlı operatörler
- İşaretçi operatörleri
- « sizeof » operatörü

Atama Operatörü (=)

Değişkene değer atamada kullanılan operatördür.

Operatörün sol tarafına değer atanacak değişken, sağ tarafına da bu değişkene atanacak değer yazılmalıdır.

```
int tam_sayi = 32;

int tam_sayi1 = tam_sayi;
```

Aritmetik Operatörler

Operatör	Görevi
+	Toplama
-	Çıkarma
*	Çarpma
/	Bölme
%	Mod Alma
++	1 arttırma

Operatör	Görevi
--	1 eksiltme
+=	Toplama Ataması
-=	Çıkarma Ataması
*=	Çarpma Ataması
/=	Bölme Ataması
%=	Mod Ataması

Mod Operatörü (%)

Bu operatör, solunda bulunan değişkenin/değerin sağında bulunan değişkene/değere bölümünden kalanı hesaplar.

```
Bolunen = 45;
Bolen1 = 15;
Bolen2 = 8;
Kalan1=Bolunen%Bolen1;
Kalan2=Bolunen%Bolen2;
```

Arttırma ve Eksiltme Operatörleri (++, --)

Bu operatörler, uygulandığı değişkenin değerini bir arttırır ya da azaltır.

```
a++;      /* a=a+1 */
a--;      /* a=a-1 */
++a;      /* a=a+1 */
--a;      /* a=a-1 */
```

Bir değişkenin sağında ya da solunda bulunabilir. Eğer önce operatör, sonra değişken yazılmışsa; değişkenin değeri bir artırıldıktan sonra atama işlemi yapılır. Tersi durumda ise atama yapıldıktan sonra değişkenin değeri artırılır ve bu durum atama yapılan değişkeni etkilemez.

```
b=a++;    /* b=a; a=a+1; */
d=++a;    /* a=a+1; d=a; */
```

ÖRNEK:

Aşağıdaki kodun ekran çıktısı nasıl olur? Kağıt üzerinde deneyip elde ettiğiniz sonucu not ediniz, bilgisayarda çalıştırdıktan sonra bulduğunuz sonuçla karşılaştırınız.

```
#include<stdio.h>
int a,b,c,d;
int main()
{ a=5;
b=a++;
c=a;
d=++a;
printf("b=%d dir, c=%d dir, d=%d dir, a=%d dir\n",b,c,d,a);
printf("Simdi ise b=%d dir",--b);
return 0;
}
```

Aritmetik Atama Operatörleri (+=, -=, *=, /=, %=)

Aritmetik işlem ve atamanın aynı anda yapılmasını sağlar.

Topla/Çıkar ve Ata: Solundaki değişkenin değerini, sağındaki değişkenin değeri kadar artırır/azaltır.

Çarp/Böl ve Ata: Solundaki değişkenin değerini sağındaki değişkenin değeri ile çarpar ya da değerine böler ve sonucu solundaki değişkene atar.

Mod Ata: Solundaki değişken değerini sağdaki ifadeye böler ve kalanı solundaki değişkene atar.

```
b-=a;     /* b=b-a; */          d*=a;     /* d=d*a; */

b%=a;     /* b=b%a; */
```

Aşağıdaki kodun ekran çıktısı nasıl olur? Kağıt üzerinde deneyip elde ettiğiniz sonucu not ediniz, bilgisayarda çalıştırdıktan sonra bulduğunuz sonuçla karşılaştırınız.

```c
#include <stdio.h>

main()
{ int a = 45, b = 2;
a +=b ; b+= 3;
printf("a=%d , b=%d\n",a,b);

a -= b; b -= 3;
printf("a=%d , b=%d\n",a,b);

a *= b; b *= 3;
printf("a=%d , b=%d\n",a,b);

a /= b; b /= 5;
printf("a=%d , b=%d\n",--a,b);

return 0;
}
```

Karşılaştırma (İlişkisel) Operatörleri (<, >, <=, >=, ==, !=)

Bu operatörler, sağındaki değişken veya ifade ile solundakini karşılaştırır, kullanılan operatör çeşidine göre **doğru (true, 1)** ya da **yanlış (false, 0)** değerini döndürür.

Ancak C dilinde doğrudan true ve false değerlerini tutacak bir veri tipi tanımlanmadığından sonuçlar 0 ve 1 sayılarıyla ifade edilir. «0'dan farklıysa doğru kabul et» yaklaşımı, çok yaygın bir şekilde kullanılmaktadır.

Aşağıdaki kodun ekran çıktısı nasıl olur? Kağıt üzerinde deneyip elde ettiğiniz sonucu not ediniz, bilgisayarda çalıştırdıktan sonra bulduğunuz sonuçla karşılaştırınız.

```c
#include <stdio.h>
int main()
{
    int a = 5, b = 5, c = 10;
    printf("%d == %d -> %d \n", a, b, a == b);
    printf("%d == %d -> %d \n", a, c, a == c);
    printf("%d > %d -> %d \n", a, b, a > b);
    printf("%d > %d -> %d \n", a, c, a > c);
    printf("%d < %d -> %d \n", a, b, a < b);
    printf("%d < %d -> %d \n", a, c, a < c);
    printf("%d != %d -> %d \n", a, b, a != b);
    printf("%d != %d -> %d \n", a, c, a != c);
    printf("%d >= %d -> %d \n", a, b, a >= b);
    printf("%d >= %d -> %d \n", a, c, a >= c);
    printf("%d <= %d -> %d \n", a, b, a <= b);
    printf("%d <= %d -> %d \n", a, c, a <= c);
    return 0;
}
```

Aritmetik Operatörlerin İşlem Sırası

Açıklama	Operatör	Öncelik
Parantez	(), []	1
Artır*, Azalt*	++, --	2
Çarpma, Bölme, Mod	*, /, %	3
Toplama, Çıkarma	+ , -	4
İlişkisel Operatörler	<, >, <=, >=	5
Eşitlik İnceleme	==, !=	6
Atama	=, +=, -=, *=, /=, %=	7

ÖRNEK:

```
a=10*2+1>=4*4+5==5>5-3*2;
/*a değeri nedir?*/
```

- İ1: 10*2 20
- İ2: 4*4 16
- İ3: 3*2 6
- İ4: İ1+1 21
- İ5: İ2+5 21
- İ6: 5-İ3 -1
- İ7: İ4>=İ5 1
- İ8: 5>İ6 1
- İ9: İ7==İ8 1
- İ10: a=İ9
- Sonuç a=1

Mantıksal Operatörler (!, &&, ||,^)

Genel olarak birden fazla karşılaştırma ifadesinin birleştirilip tek bir koşul ifadesi haline getirilmesinde kullanılır.

Mantıksal Operatörlerin Doğruluk Tablosu							
&&(and)			||(or)			!(not)	
x	y	Sonuç	X	Y	Sonuç	X	sonuç
0	0	**0**	0	0	**0**	0	**1**
0	1	**0**	0	1	**1**	1	**0**
1	0	**0**	1	0	**1**		
1	1	**1**	1	1	**1**		

Mantıksal Operatörler (!, &&, ||,^)

XOR (FARKLI) ^ :

İki değişken birbirinden **farklı**ysa sonuç TRUE

İki değişken birbirinin aynıysa sonuç FALSE,

A	B	A **XOR** B
0	0	0
0	1	1
1	0	1
1	1	0

Aşağıdaki kodun ekran çıktısı nasıl olur? Kağıt üzerinde deneyip elde ettiğiniz sonucu not ediniz, bilgisayarda çalıştırdıktan sonra bulduğunuz sonuçla karşılaştırınız.

```c
#include <stdio.h>
int main()
{
    int a = 5, b = 5, c = 10, result;
    result = (a == b) && (c > b);
    printf("(a == b) && (c > b) -> %d \n", result);
    result = (a == b) && (c < b);
    printf("(a == b) && (c < b) -> %d \n", result);
    result = (a == b) || (c < b);
    printf("(a == b) || (c < b) -> %d \n", result);
    result = (a != b) || (c < b);
    printf("(a != b) || (c < b) -> %d \n", result);
    result = !(a != b);
    printf("!(a != b) -> %d \n", result);
    result = !(a == b);
    printf("!(a == b) -> %d \n", result);
    return 0;
}
```

NOTLAR

- Mantıksal operatörler, aritmetik operatörlerden sonra işletilir.
- Mantıksal operatörlerin kendi içlerinde işlem sırası DEĞİL, VE, YA DA şeklindedir.
- Yazdığınız karmaşık bir ifadede hangi operatörün daha önce çalıştırılacağından emin değilseniz, önce çalıştırılmasını istediğiniz ifadeyi parantez içine almalısınız.

ÖDEV:

Aşağıdaki işlemlerin sonuçlarını bulup, C programı yardımıyla sağlamasını yapınız.

a= 80/5-3*2<2+3*2;

b= 15-7*2>=1==19-9*2;

a && b = ?

a || b = ?

!a && b =?

BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 5– Operatörler-2 ve Karşılaştırma

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

30/03/2021

Konya

Mantıksal Operatörler (!, &&, ||,^)

XOR (FARKLI) ^ :

İki değişken birbirinden **farklı**ysa sonuç TRUE

İki değişken birbirinin aynıysa sonuç FALSE,

A	B	A **XOR** B
0	0	0
0	1	1
1	0	1
1	1	0

XOR Operatörü ^

```
int a=63;                    // 0011 1111 = 63
int b=15;                    // 0000 1111 = 15
int e=a^b;                   // 0011 0000 = 48
Printf("(a^b)=%u", e);
```

Ekran Çıktısı:

```
(a^b)=48
--------------------------------
Process exited after 0.04142 seconds with return value 0
Press any key to continue . . .
```

Complement (Tümleme) Operatörü ~

Uygulandığı değişkenin veya değerinin ikilik tabandaki bitlerini ters çevirir.

0 olan bitleri 1 yapar, 1 olan bitleri 0 yapar.

```c
#include <stdio.h>
#include <stdlib.h>

/* run this program using the console pauser or add your ov

int main() {

     unsigned char ch = 63;          // 0011 1111 = 63
     unsigned char tumleyen = ~ch;   // 1100 0000 = 192
     printf(" %d 'nin tumleyeni: %d ", ch,tumleyen);


    return 0;
}
```

```
 63 'nin tumleyeni: 192
--------------------------------
Process exited after 0.01396 seconds with return value 0
Press any key to continue . . .
```

Bit Tabanlı Sağa/Sola Kaydırma Operatörleri << >>

Bu operatörler solundaki değişken veya değerlerin 2'lik tabandaki karşılıklarını, sağındaki değer kadar sağa veya sola kaydırır.

Sola kaydırma işlemi aslında ilgili değeri 2 katına çıkartır.
Sağa kaydırma işlemi ise ilgili değeri 2'ye böler

Bit Tabanlı Sağa/Sola Kaydırma Operatörleri << >>

```c
#include <stdio.h>
#include <stdlib.h>

/* run this program using the console pauser or add yo

int main() {

    int g=8;        // 0000 1000 = 8
    int saga1kaydir=g>>1;       // 0000 0100 = 4
    int saga2kaydir=g>>2;       // 0000 0010 = 2
    int saga3kaydir=g>>3;       // 0000 0001 = 1
    int saga4kaydir=g>>4;       // 0000 0000 = 0

    printf("(%d >> 1) = %d\n", g, saga1kaydir );
    printf("(%d >> 2) = %d\n", g, saga2kaydir );
    printf("(%d >> 3) = %d\n", g, saga3kaydir );
    printf("(%d >> 4) = %d\n", g, saga4kaydir );

    int sola1kaydir=g<<1;       // 0001 0000 = 16
    int sola2kaydir=g<<2;       // 0010 0000 = 32
    int sola3kaydir=g<<3;       // 0100 0000 = 64
    int sola4kaydir=g<<4;       // 1000 0000 = 128

    printf("(%d << 1) = %d\n", g, sola1kaydir );
    printf("(%d << 2) = %d\n", g, sola2kaydir );
    printf("(%d << 3) = %d\n", g, sola3kaydir );
    printf("(%d << 4) = %d\n", g, sola4kaydir );

    return 0;
}
```

```
C:\Users\Kemal\Desktop\C_Kemal_Ders\Kodlar\ders3\ders3.3.exe
(8 >> 1) = 4
(8 >> 2) = 2
(8 >> 3) = 1
(8 >> 4) = 0
(8 << 1) = 16
(8 << 2) = 32
(8 << 3) = 64
(8 << 4) = 128

--------------------------------
Process exited after 0.01396 seconds with return
Press any key to continue . . .
```

? : Operatörü (Şartlı Atama)

- Bir değişkene şartlı atama yapılmasını sağlar.
- C programlama dilinde 3 argüman ile kullanılan tek operatördür.
- Örnek:

ders5.4.c

```c
#include <stdio.h>
#include <stdlib.h>

/* run this program using the console pauser or add your own getch, system("pause") or input loop *

int main() {

    int x=11, y=5, z=0, z1=0, a=7, b=9;

    z = x > y ? a : b; // x degeri y'den buyukse, z'nin degeri a olsun aksi halde b olsun.

    printf("z: %d\n", z);

    z1 = x < y ? a: b; // x degeri y'den kucukse, z'nin degeri a olsun aksi halde b olsun.

    printf("z1: %d\n", z1);

    return 0;
}
```

```
z: 7
z1: 9

--------------------------------
Process exited after 0.0144 seconds with return value 0
Press any key to continue . . .
```

İşaretçi Operatörü * (Pointer)

- * operatörü çarpma işlemini temsil etmektedir. Bunun dışında işaretçi operatörü (pointer) olarak da kullanılmaktadır.
- İlerleyen haftalarda daha kapsamlı olarak Pointer konusunda anlatılacaktır.

Sizeof Operatörü

- Bu operatör bize, bir veri tipinin bellekte kaç bayt yer kapladığını gösterir.
- Bir veri tipinin kapladığı alan negatif olamayacağından bu işlem sonucundaki çıktı her zaman pozitif yani işaretsiz tamsayı döndürür (unsigned int).
- printf fonksiyonu ile ekrana çıktı yazdırılırken %lu biçim niteleyicisi kullanılır.
- Örnek:

```
int ornekSayi = 5;
unsigned int kacBayt = sizeof (ornekSayi);
printf ("Bellekte kapladigi alan: %lu bayttir", kacBayt);
```

Koşul (Şart) İfadeleri

- Koşul ifadelerinin ve akış kontrollerinin programlama dillerinde ve uygulama altyapısında her zaman önemli bir yeri vardır.
- Yazdığımız uygulamanın hangi koşulda nasıl davranacağını belirterek bir akış kontrolü oluşturabilmek için koşul ifadelerinin iyi bilinmesi gereklidir.
- Tüm programlama dillerinde olduğu gibi en temel koşul ifadesi «if-else» bloklarıdır.

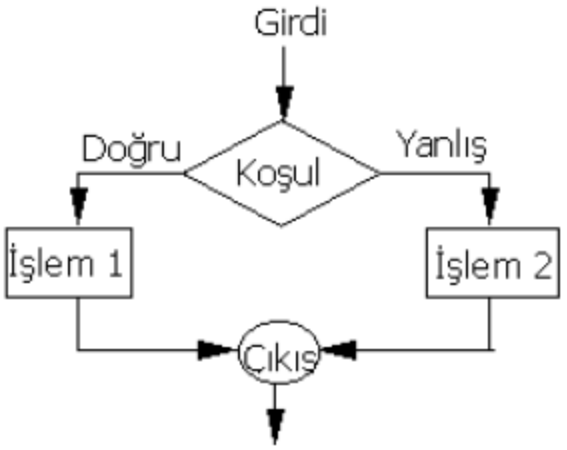
Koşul (Şart) İfadeleri

- Sonucu DOĞRU veya YANLIŞ olan herhangi bir ifadeye "koşul" denir.
- DOĞRU => 1 ile gösterilir.
- YANLIŞ => 0 ile gösterilir.
- Örnek:
- if(a==b)
- printf("İki sayı birbirine eşittir \n");
- **(a==b)** ifadesi bir koşuldur.
- Koşullar karşılaştırma operatörleri ve mantıksal operatörler kullanılarak oluşturulurlar.

If - Else

- if ve else tek bir karşılaştırma deyimi olup else kullanımı isteğe bağlıdır.
- Eğer bu koşul olumlu ise if den sonraki bölüm yürütülür ve else den sonraki bölüm atlanır.
- Koşul olumsuz ise if den sonraki küme atlanır ve eğer varsa, else den sonraki kümedeki işlemler gerçekleştirilir.

```
if (koşul)
   { Doğru ise İşlem1}
else
   { Yanlış ise İşlem2}
```



If - Else

Örnek 1

```
if (sayi < 0) {
   printf("Negatif sayı girdiniz\n");
}
else {
  printf("Sıfır veya pozitif bir sayı girdiniz\n");
}
```

Örnek 2

```
if (sayi < 0)
   printf("Negatif sayı girdiniz\n");
else
  printf("Sıfır veya pozitif bir sayı girdiniz\n");
```

Not: *Her iki örnek aynı işlemi gerçekleştirmektedir. Tek fark birinci örnekte { } sembollerinin kullanılmasıdır. Tek satır deyim içeren bloklarda, { } sembollerinin kullanılmasına gerek yoktur.*

İç içe if – else blokları

- if-else yapıları içerisine başka if-else yapıları yerleştirerek birden fazla koşul aynı anda test edilebilir.
- **Örnek:**

```
if (sayi < 0)
  printf("Negatif sayı girdiniz\n");
else
    if (sayi > 0)
        printf("Pozitif bir sayı  girdiniz\n");
    else
       printf("Sıfır sayısını girdiniz\n");
```

İç içe if – else blokları

- Diğer bir kullanım şekli aşağıdaki gibidir:

```
if(koşul_1) {
  deyimler; //(küme_1)
 }
 else if(koşul_2) {
  deyimler; //(küme_2)
 }
 ....
 ....
 else if(koşul_n-1) {
  deyimler; //(küme_n-1)
 }
 else {
  deyimler; //(küme_n)
  }
```

Girilen Notun Harf Karşılığını Bulma

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int ogrenci1;
    printf ("Notu giriniz:\n");
    scanf("%d",&ogrenci1);

    if(ogrenci1<=24)
    printf ("Ogrencinin Harf Notu FF");
    else if(ogrenci1>=25 && ogrenci1<=34)
    printf ("Ogrencinin Harf Notu FD");
    else if(ogrenci1>=35 && ogrenci1<=39)
    printf ("Ogrencinin Harf Notu DD");
    else if(ogrenci1>=40 && ogrenci1<=49)
    printf ("Ogrencinin Harf Notu DC");
    else if(ogrenci1>=50 && ogrenci1<=57)
    printf ("Ogrencinin Harf Notu CC");
    else if(ogrenci1>=58 && ogrenci1<=64)
    printf ("Ogrencinin Harf Notu CB");
    else if(ogrenci1>=65 && ogrenci1<=73)
    printf ("Ogrencinin Harf Notu BB");
    else if(ogrenci1>=74 && ogrenci1<=81)
    printf ("Ogrencinin Harf Notu BA");
    else if(ogrenci1>=82 && ogrenci1<=100)
    printf ("Ogrencinin Harf Notu AA");
    else
    printf ("Uygun Deger Girmediniz");
    return 0;
}
```

Not Aralıkları	Harf Notu
82 – 100	AA
74 – 81	BA
65 – 73	BB
58 – 64	CB
50 – 57	CC
40 – 49	DC
35 – 39	DD
25 – 34	FD
0 – 24	FF

if – else

- C programlamada, koşulların mantıksal değerlendirmelerinde, beklenilen değere ulaşıldığında koşulun tamamı değerlendirilmeden koşuldan çıkılır.
- VEYA kullanımında koşulun ilk kısmı DOĞRU (true) ise koşulun tamamı DOĞRU olacağı için koşulun ikinci kısmı değerlendirilmez.
- VE kullanımında koşulun ilk kısmı YANLIŞ (false) ise koşulun tamamı YANLIŞ olacağı için koşulun ikinci kısmı değerlendirilmez.
- **Örnek:** if ((a>b) && (a<c)) eğer (a>b) YANLIŞ ise (a<c) değerlendirilmez.
- **Örnek:** if ((a>b) || (a<c)) eğer (a>b) DOĞRU ise (a<c) değerlendirilmez.

Örnek

1. Program Parçası

```
if (x < 5)
if (y > 5)
printf("aaaa\n");
else
printf("bbbb\n");
printf("cccc\n");
```

Soru 1: x=4 ve y=6 iken çıktı ne olur?

```
aaaa
cccc
```

Soru 2: x=6 ve y=4 iken çıktı ne olur?

```
cccc
```

2. Program Parçası

```
if (x < 5) {
if (y > 5)
printf("aaaa\n");
}
else {
printf("bbbb\n");
printf("cccc\n");
}
```

```
aaaa
```

```
bbbb
cccc
```

If – Else Kullanım Hataları

Hata-1

```
if sayi==10              //Söz dizimi hatası
   printf ("Girilen sayi 10\n");
```

- Koşul ifadesinde ( ) sembolleri kullanılmadığı için "syntax error" hatası alınır (Söz dizimi hatası).

Hata-2

```
if (sayi==10);              //Boş ifade
   printf ("Girilen sayi 10\n");
```

- Koşul ifadesinden sonra ; sembolü kullanılmıştır. C derleyicisi için boş bir ifadedir. Sonraki satır bu satırdan bağımsızdır.

If – Else Kullanım Hataları

Hata-3

```
if (10<=sayi<=50)                    //Mantıksal  Hata
```

- Sonucu her zaman doğru olan mantıksal bir hatadır.
- Koşul ifadesi soldan sağa doğru değerlendirilir. **sayi** değişkenin değeri 10 ile karşılaştırılır.
  - 10'dan küçükse false yani 0 sonucu döner,
  - 10'dan büyükse true yani 1 sonucu döner,
  - Her iki durum da (0 veya 1 sonucu) dönen değer 50'den küçük olacağı için bu ifade her zaman doğru sonuç verir.
- Bunun yerine ((sayi>=10) && (sayi<=50)) koşul ifadesi kullanılmalıdır.

If – Else Kullanım Hataları

Hata 4

```
if (sayi=10)                    //Mantıksal Hata
```

- Sonucu her zaman doğru olan mantıksal bir hatadır.
  - Koşul ifadesinde karşılaştırma operatörü == yerine, atama operatörü = kullanılmıştır.
  - Atama sonucunda 10 değeri oluşur.
  - 10 değeri 0'dan büyük olduğu için sonuç "true" dönecektir ve bu da yanlış bir sonuç üretecektir.
- Atama operatörü yerine, == karşılaştırma operatörü kullanılmalıdır.

Switch – Case Yapısı

- Bir değişkenin aldığı değerlere bağlı olarak, program akışındaki belirli işlemlerin yaptırılmasını sağlar.
- Aslında bu tür işlemleri **if** deyimi yardımıyla yerine getirebiliriz.
- Ancak bazı uygulamalarda **if** deyiminin kullanılması programın karmaşıklığını artırabilir ve okunabilirliğini azaltabilir.
- Eğer bir değişkenin değeri belirli sabitlerle karşılaştırılacak ve bunun sonucunda farklı işlemler yapılacak ise **if** deyimi yerine **switch** deyimi kullanmamız daha sağlıklı olacaktır.
- Sadece char ve int veri türleri için kullanılabilir.

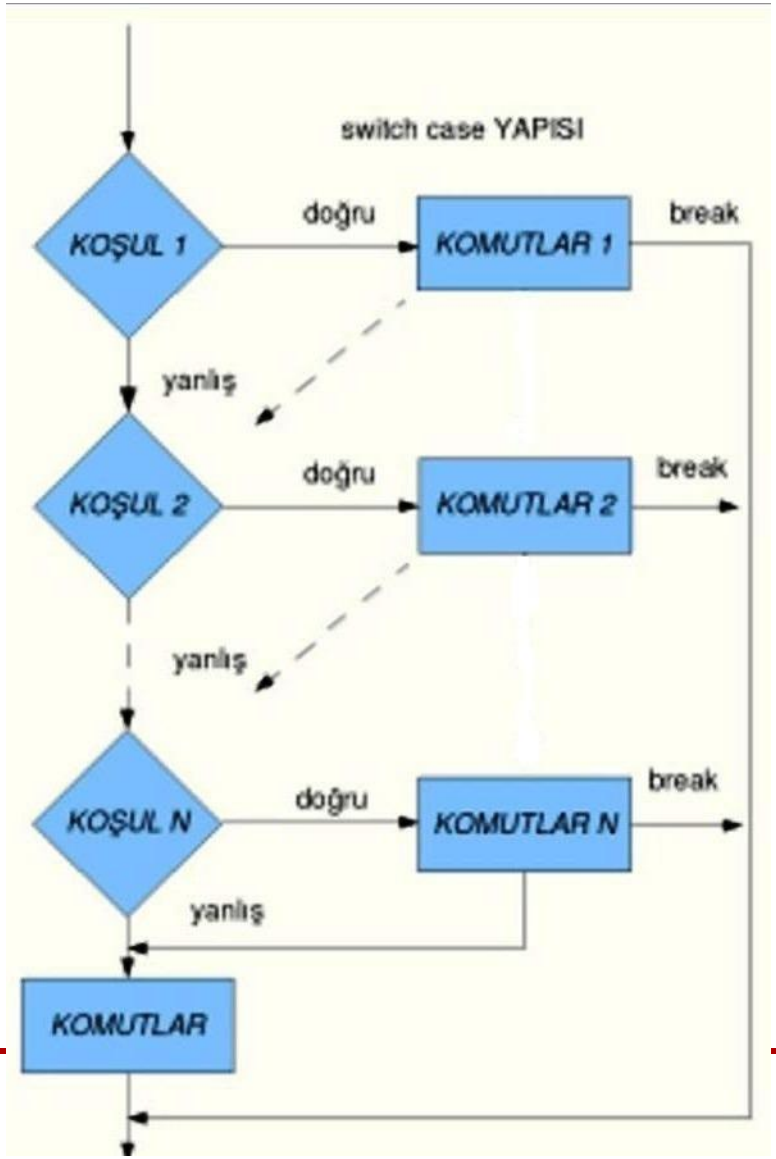
Switch – Case Yapısı

```
switch (değişken) {
   case sabit1:
        deyim1
        break;
   case sabit2:
        deyim2
        break;
        …
   case sabitn:
        deyimn
        break;
   default:
         deyimd;
   }
```

değişken değeri ***sabit*$_1$** değerine eşitse **deyim$_1$** çalıştırılır ve **break** kullanılarak bloktan çıkış yapılır.

değişken değeri herhangi bir sabite eşit değilse **default** sabitinin deyimi ***deyim*$_d$** çalışır.

Switch – Case Akış Şeması



Örnek (Karşılaştırma)

- Girilen il plaka koduna göre ekrana ilgili şehir ismi yazdırılacaktır.
  - ANKARA
  - İSTANBUL
  - İZMİR
  - MANİSA
- Bunların dışında bir plaka kodu girilirse «TANIMSIZ PLAKA KODU» şeklinde hata mesajı verilecektir.

Örnek (Karşılaştırma)

```c
int main()
{
    int plaka_kodu;
    printf("Ilin plaka kodunu giriniz: ");
    scanf("%d",&plaka_kodu);

    switch (plaka_kodu){
        case 6:
            printf("ANKARA\n");
            break;
        case 34:
            printf("ISTANBUL\n");
            break;
        case 35:
            printf("IZMIR\n");
            break;
        case 45:
            printf("MANISA\n");
            break;
        default:
            printf("TANIMSIZ PLAKA KODU\n");
    }
    return 0;
}
```

```c
int main()
{
    int plaka_kodu;
    printf("Ilin plaka kodunu giriniz: ");
    scanf("%d",&plaka_kodu);

    if (plaka_kodu==6)
        printf("ANKARA\n");
    else if (plaka_kodu==34)
        printf("ISTANBUL\n");
    else if (plaka_kodu==35)
        printf("IZMIR\n");
    else if (plaka_kodu==45)
        printf("MANISA\n");
    else
        printf("TANIMSIZ PLAKA KODU\n");

    return 0;
}
```

Switch-Case Kullanım Hataları

- break deyiminin kullanılmasının unutulması.
- switch-case bloğundaki { } sembollerinin kullanımlarının unutulması.
- default deyiminin ve içeriğinin kullanımının unutulması.
- Kontrol edilen değişken ile verilen sabitlerin farklı tipte olması.
- int ve char veri türleri dışında değişken tanımlamaya çalışmak (double, string)

If-Else ve Switch-Case Karşılaştırması

1. İşlevsellik:

- C programlama dilinde
  - switch-case deyimi sadece char ve int veri türlerini destekler. Bu durum if-else deyimine göre dezavantajdır.
  - switch-case deyimlerinde aralık kontrolü (x > 1000) yapılamaması yine bir dezavantajdır.

2. Performans:

- Her iki karşılaştırma deyiminin kullanımı farklı derleyicilerde ve farklı programlama dillerinde farklı performans sonuçları üretebilir.
- Genelde switch-case kullanımı daha performanslı çalışır.

3. Okunabilirlik:

- switch-case deyimlerinin kod okunabilirliği her zaman daha yüksektir.

```
/**
* Code Readability
*/
if (readable()) {
    be_happy();
} else {
    refactor();
}
```

ÖDEV

- Ödev 1:

  Kullanıcıdan 2 sayı girmesini ve yaptırmak istediği matematiksel işlemin işaretini girmesini isteyiniz ve istenen işlem sonucunu ekrana yazdırınız (If-Else yapısıyla).

  Örnek ekran çıktısı:

  Birinici sayiyi giriniz: 5

  İkinci sayiyi giriniz: 6

  Yapilacak islemi seciniz (+, -, *, /): +

  Islem Sonucunuz: 5+6=11

- Ödev 2:

  Switch-case yapısıyla kullanıcının girdiği harflere göre 4 takımın ismini ekrana yazdırınız.

  b,B=>Besiktas f,F =>Fenerbahce g,G =>Galatasaray k,K =>Konyaspor



BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 6 – Döngüler

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

06/04/2021

Konya

Döngü Deyimleri

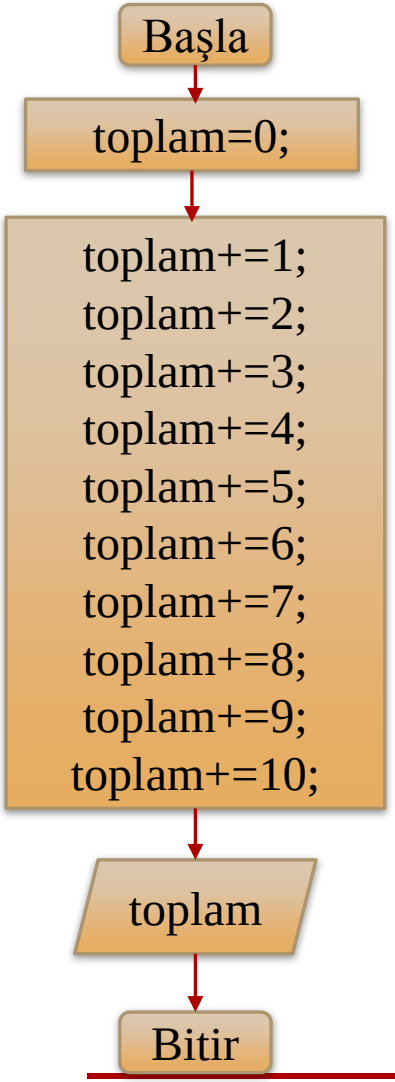
Döngü deyimleri, belirli bir kod parçasının tanımlanmış bir koşul altında tekrarlı bir şekilde çalıştırılmasını sağlar. Kod uzunluğunu azaltarak etkinliği de artırırlar.

Döngünün sonlanabilmesi için, bir aşamasında mutlaka şart ifadesi geçersiz olmalıdır. Aksi takdirde sonsuz döngü oluşur.

Koşulu başta sınayanlar: **for**, **while**
Koşulu sonda sınayanlar: **do while**

* **for** ve **while**' da çevrim içine hiç girilmeyebilir, ancak **do while**' da çevrim en az bir kez yürütülür.

Örnek: 1'den 10'a kadar sayıların toplamını döngü kullanmadan bulduran bir kod yazınız.

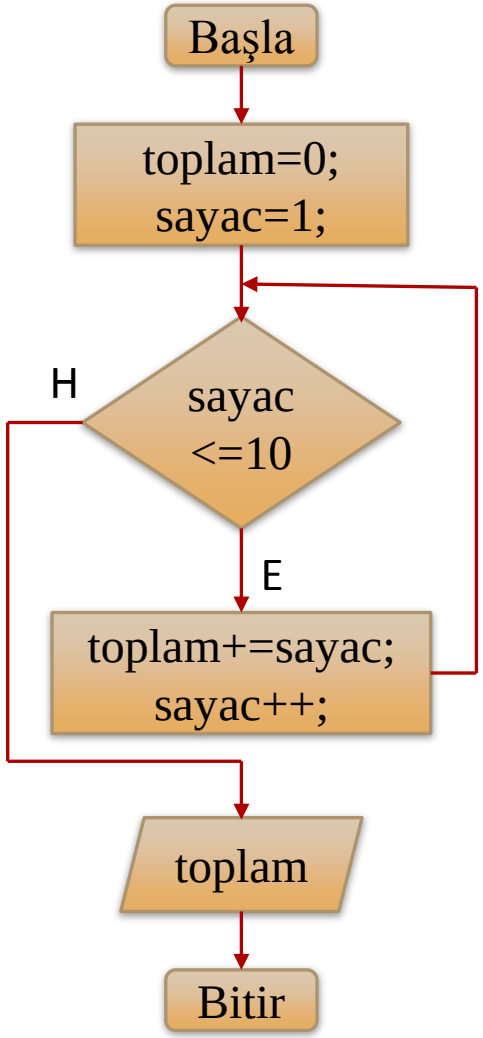



```
#include<stdio.h>
int main() {  int toplam=0;

toplam=toplam+1;   /* toplam+=1; */
toplam=toplam+2;   /* toplam+=2; */
toplam=toplam+3;
toplam=toplam+4;
toplam=toplam+5;
toplam=toplam+6;
toplam=toplam+7;
toplam=toplam+8;
toplam=toplam+9;
toplam=toplam+10;
printf("Sayıların toplamı: %d\n", toplam);

return 0; }
```

Örnek: 1'den 10'a kadar sayıların toplamını döngü kullanarak bulduran bir kod yazınız.




```
#include<stdio.h>
int main() {  int toplam=0; int sayac=1;

while (sayac<=10)
{  toplam=toplam+sayac;   /* toplam+=sayac; */
   sayac++;   }

printf("Sayıların toplamı: %d\n", toplam);
return 0; }
```

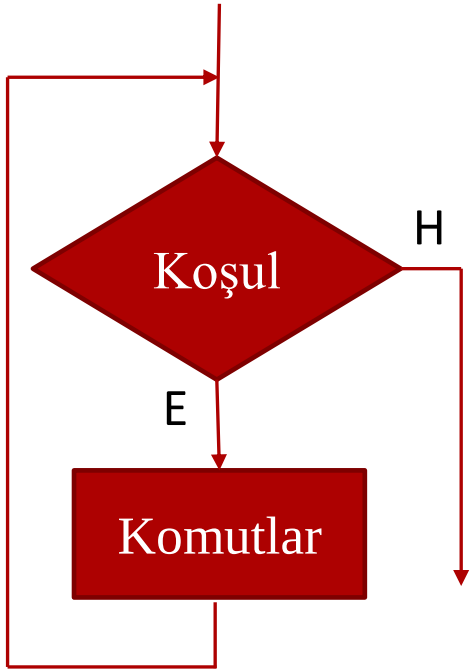
```
Sayilarin toplami: 55

--------------------------------
```

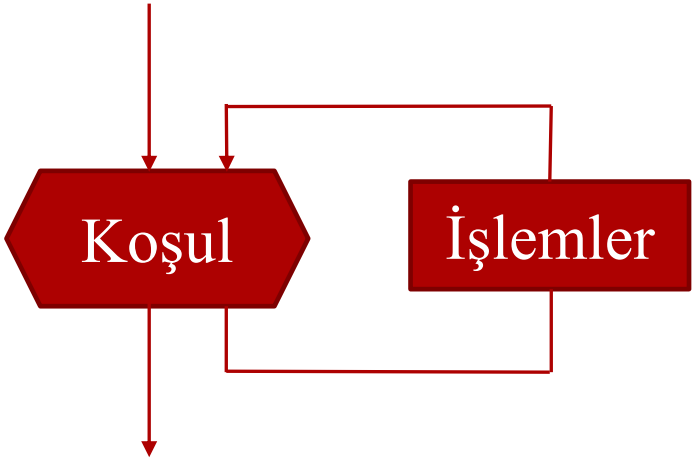
while Döngüsü

- İçine yazılan komut veya komut kümesini, belli bir şart sağlandığı sürece tekraren çalıştırır.

```
while(koşul)
{
İşlem-1;
….
İşlem-N;
}
```



- İki ya da daha fazla koşulun mantıksal operatörlerle birleştirilerek verilmesi de mümkündür.

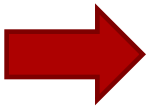


- Döngü şartı başlangıçta test edilir ve şartın sağlanması halinde döngüye girilir. Döngü içinde tek komut çalıştırılacaksa süslü parantezler kullanılmayabilir. Ancak kaç komut olursa olsun parantez kullanmayı alışkanlık haline getirmek daha doğru bir yaklaşım olacaktır. Bu yönüyle **if** yapısına benzer.

```
while(koşul)
Tek-Komut;
```

- Kullanılan sayaç değişkenine mutlaka başlangıçta bir ilk değer ataması yapılmalıdır.

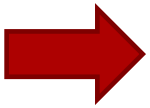
```
main() {
int sayac;
  while(sayac<100) {
   printf(‘‘%d’’,sayac);
sayaç++; }    }
```



Ekrana sayı değeri yazdırılıp yazdırılmayacağı ya da hangi sayıdan başlayarak 100’e kadar sayıların yazdırılacağı belirsizdir.

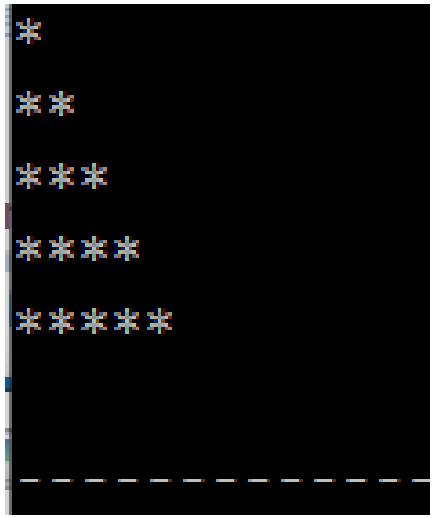
- Döngü içerisinde koşul değişkeninin artırılması gerektiği unutulmamalıdır.

```
main() {
int k=1;
   while(k<2) {
    printf(‘‘%d’’,k);
}    }
```



Koşul değişkeni k döngü içerisinde artırılmadığı için ekrana sonsuza kadar 1 yazar.

- **Örnek:** Ekranda aşağıdaki çıktıyı oluşturmak için **while** döngüsü kullanarak bir C kodu yazınız.

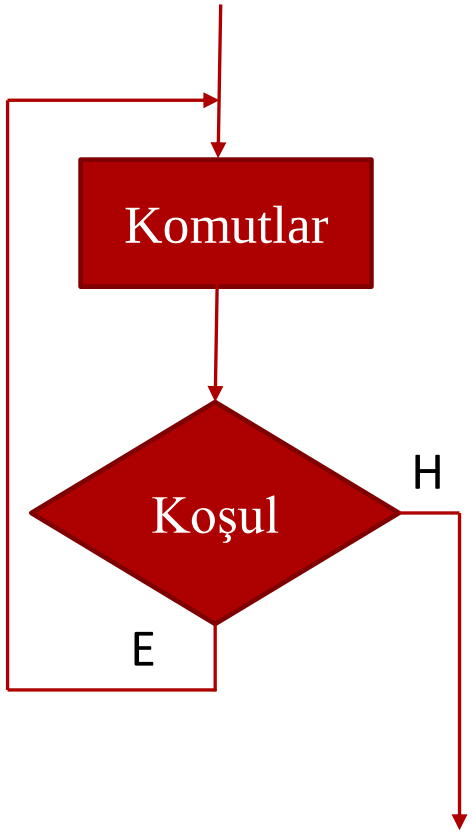



```
*
**
***
****
*****
```

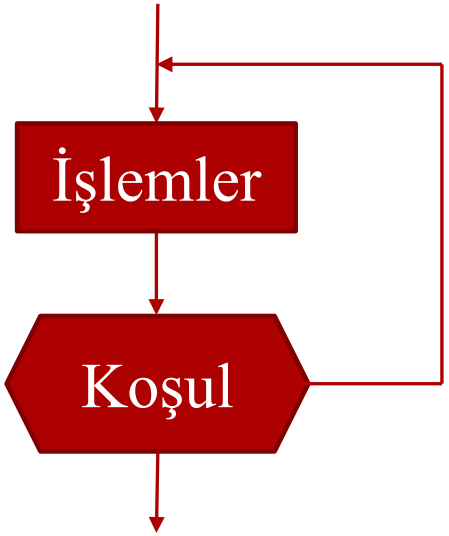
```c
#include<stdio.h>
int main()
{
    int i=0, j;
    while(i<5)
    {
        j = 0;
        while(j<i+1)
        {
            printf(" * ");
            j++;
        }
        printf("\n");
        i++;
    }
    return 0;
}
```

do … while Döngüsü

- **while**' dan farkı, koşulun döngünün sonunda sınanmasıdır. Yani koşula bakılmaksızın çevrime girilir, tekrarlanması koşulun doğruluğuna bağlıdır.
- Dolayısıyla ilk çevrim koşulsuz, sonrakiler koşullu yürütülür.
- İki ya da daha fazla koşulun mantıksal operatörlerle birleştirilerek verilmesi de mümkündür.



```
do
{
   İşlem-1;
   ....
   İşlem-N;
} while(koşul);
```



- Programda bir kod parçası yürütüldükten sonra tekrarlanması istenip istenmediği kullanıcıya sorularak gelen yanıta göre tekrar sağlanacak durumlarda kullanılabilir.

- Döngü sonunda while ifadesinden sonra mutlaka **;** eklenmelidir.

Örnek: Klavyeden giren iki sayıyı toplayan ve kullanıcının isteğine göre işlemi tekrarlayan bir C kodu yazınız.

```
#include<stdio.h>
main() {
int a,b;  char kr;
do { printf("Bir sayi giriniz:\n");
       scanf("%d",&a);
    printf("Bir sayi daha giriniz:\n");
       scanf("%d",&b);
       printf("%d+%d=%d\n",a,b,a+b);
       printf("Baska islem yapmak istiyor musunuz? (E/H)\n");
       kr=getch();
    } while(kr!='h' && kr!='H');
}
```

for Döngüsü

- Çevrime girmeden koşulun sınandığı bir diğer döngü deyimidir.

- **while**' dan farkı, sayacın döngü deyiminin içinde yer alması ve yine döngü deyimi içinde artırılmasıdır.

```
for (ilk_deger;koşul;artim)
{
  İşlem-1;
  ....
  İşlem-N;
}
```

- Parantez içindeki ifadeler **;** ile ayrılmalıdır.

- for deyiminden sonra ; konmaz.

```
for (i=0; i<=15; i++)
```

for Döngüsü

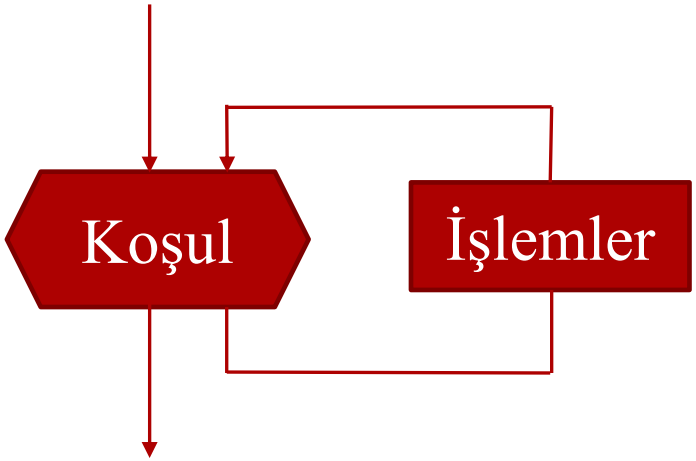
```
for( int i = 1 ; i <= 5 ; i++ )
    {
      printf("%d",i);
    }
```

Program çıktısı: 12345

while Döngüsü

```
int i = 1;
while( i <= 5 )
    {
      printf("%d",i);
      i++;
    }
```

Program çıktısı: 12345





- **for** döngüsünde noktalı virgülle ayrılan kısımlar içinde birden fazla komut varsa, bu komutların virgül ile ayrılması gerekir.

```
#include<stdio.h>
main() {
int a,b;
for (a=4, b=6; a*b<=100; a++, b+=2)
    {  printf(‘‘a değişkeninin şu anki değeri: %d\n’’,a);
       printf(‘‘b değişkeninin şu anki değeri: %d\n’’,b);
       printf(‘‘Çarpma işlemi sonucu:%d\n’’,a*b);
    } return 0;
          }
```

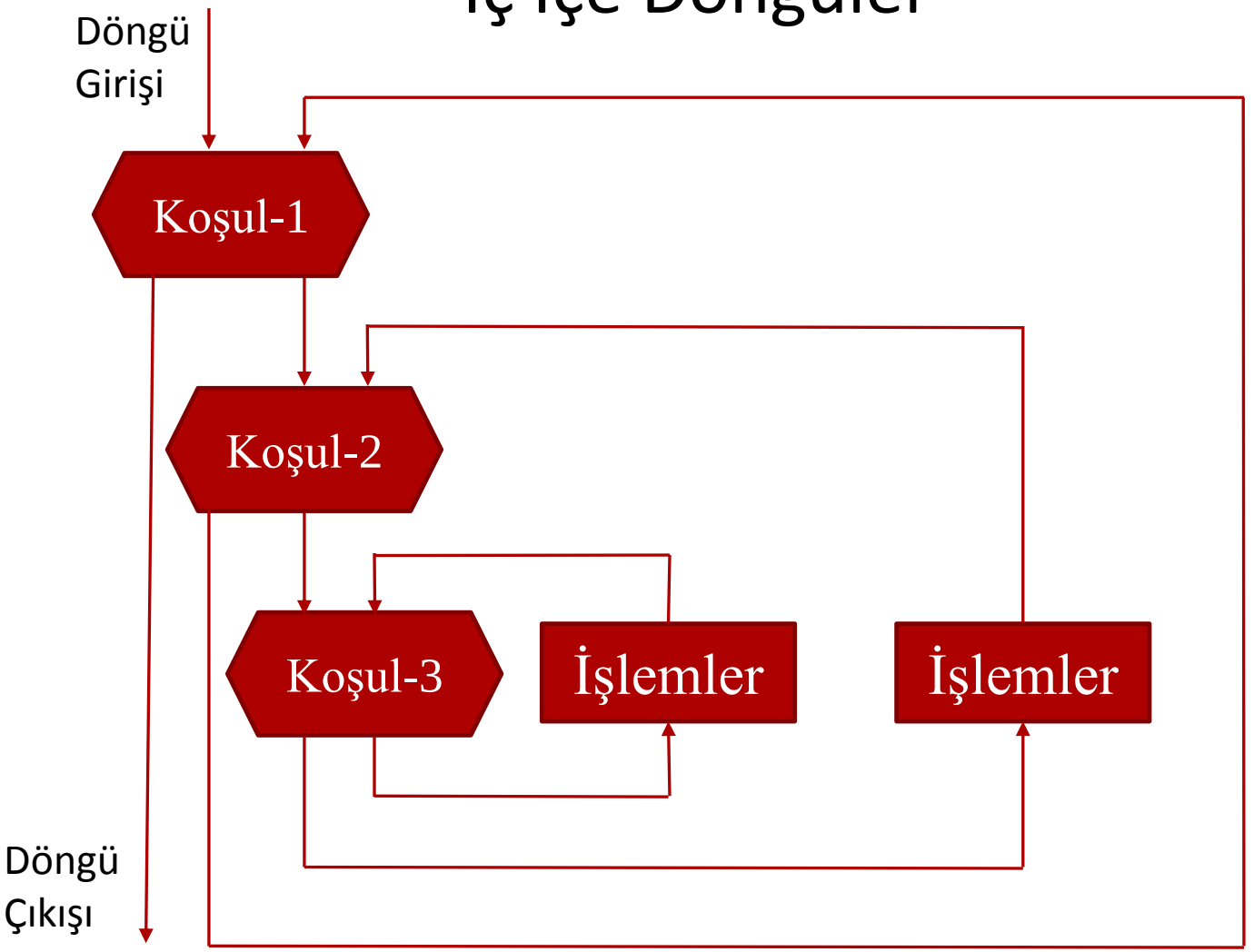
- Döngü ifadesi içinde başlangıç, koşul ve artım/azaltım parametrelerinin herhangi biri kullanılmayabilir. Ancak herhangi biri olmadığında döngünün nasıl yürütüleceği iyi anlaşılmalıdır.

```
for (; ;)
   printf(‘‘ Sonsuz döngüye girdiniz.\n ’’);
```

```
…
for(k=1; k<10;)
printf(‘‘%f\n ’’, sqrt(k++));
…
```

```
…
x=0;
for(; x<5; x+=2)
printf(‘‘%d\n ’’, x);
…
```

İç İçe Döngüler



Örnek: Çarpım tablosunu yazdıran program kodu

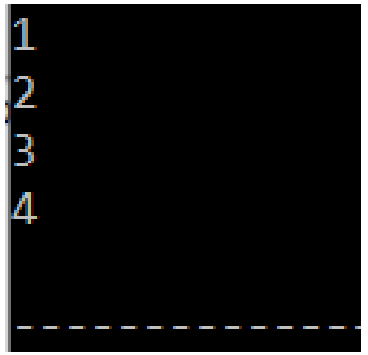
```
#include<stdio.h>
  main(){
  int num,i,j;
       printf("Please enter an integer number: ");
       scanf("%d", &num);
  for(i=1; i<=num; i++){
           for(j=1; j<=10; j++)
             { printf("%d * %d = %d\t", i,j, i*j);  }
   printf("\n"); }
  return(0);    }
```

Parantez eklenmeyebilir.

break Deyimi

Döngü içerisinde belli şartlar oluştuğunda döngüyü sonlandırmak için kullanılır.

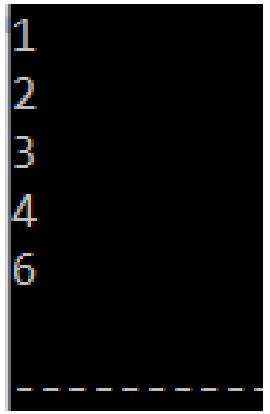
```
#include<stdio.h>
main() {
int i;
      for(i=1; i<7; i++)
        {     if(i==5) { break; }
              printf("%d \n", i);
        }
return 0;   }
```



continue Deyimi

Derleyici bu deyimle karşılaştığında, döngünün bulunduğu iterasyonu sonlandırır ve bir sonraki iterasyona geçer. Başka bir deyişle, döngünün o iterasyonunda continue deyiminden sonraki komutlar yürütülmez.

```
#include<stdio.h>
main() {
int i;
      for(i=1; i<7; i++)
        {     if(i==5) { continue; }
              printf("%d \n", i);
        }
return 0;   }
```



ÖDEV:

Sizlerle paylaşılan “c-dongu-ornek.pdf” dosyası içindeki örneklerin her birini önce cevaplara bakmadan kendiniz Dev-C++ üzerinde gerçekleştirmeye çalışınız. Daha sonra yazdığınız kodu bu dosyadaki cevaplarla karşılaştırınız. Hangisinin daha etkin olduğunu yorumlayınız.

BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 7 – Diziler ve Matrisler

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

13/04/2021

Konya

Diziler (Arrays)

- Bugüne kadar anlatılan konularda verileri depolamak için çeşitli değişkenler kullanıldı.
- Bazı programlarda aynı tip ve çok fazla sayıda değişkeni depolamak gerekebilir.

Örnek:

Bir elektrik sinyalinin voltaj değerini 5'er saniye aralıklarla 500 saniye boyunca kaydetmek.

- Böyle bir program yazmak için 100 adet double tipinde değişken tanımlamamız gerekirdi.
- Diziler bu noktada işimizi kolaylaştırmaktadır.

$$a = \{a_0, a_1, a_2, \ldots, a_{99}\}$$

Tanım ve Özellikler

Dizinin tanımı:

- Bir dizi belirli sayıda ve benzer tipteki değişkenlere tek bir isim ve çeşitli indisler yardımıyla erişim sağlayan özel bir veri yapısıdır.

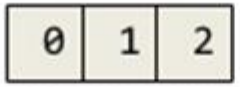
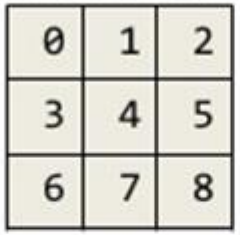
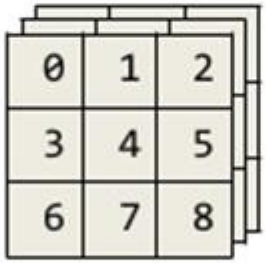
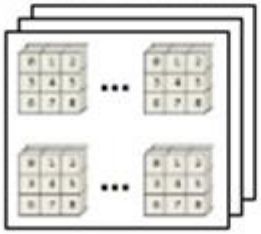
Özellikleri:

- Bir dizi aynı tip verilere sahiptir.
- Bellekte art arda saklanırlar.

$$int \quad a[\,] = \quad \{1123,\ 1125,\ 1234,\ 1256,\ 1321\};$$

Dizilerin Özellikleri

- Tek boyutlu ve çok boyutlu olabilirler.

Boyutlar	Örnek	Adı
1		Vektör
2		Matris
3		3 Boyutlu Dizi
N		N Boyutlu Dizi

Dizilerin Özellikleri

- Bir dizi çok sayıda değişken barındırdığından, bunları birbirinden ayırt etmek için indis adı verilen bilgiler kullanılır.
- Bilgisayar programları basit editörlerle yazılır. O editörler a_i simgesindeki gibi alt simge koyamaz. Üstelik, derleyiciler de alt simgeyi anlamaz. O nedenle, alt simge işlevini görmek üzere, indisler köşeli parantez içine yazılır.
- Dolayısıyla, bir dizinin öğeleri aşağıdaki gibi yazılır:

$$a = \{a[0], a[1], a[2], \ldots, a[99]\}$$

Dizilerin Özellikleri

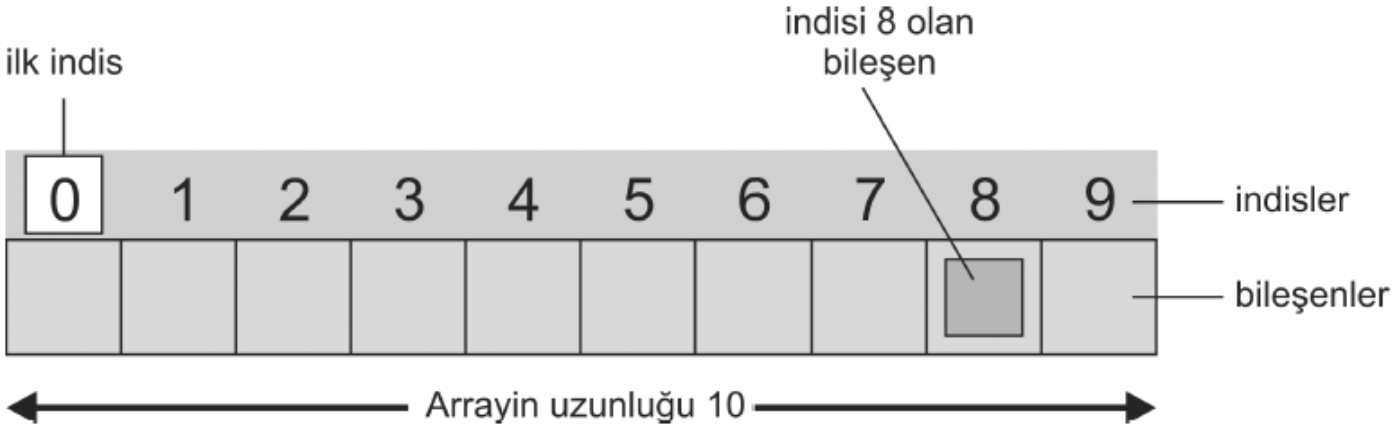
- [ ] operatörünün iki işlevi vardır:

1) array bildirimi $int \;\; a[n];$

2) arrayin bileşenlerinin sıra numarasını (indis) gösterir.

- Arrayde indisler daima tam sayıdır ve 0'dan başlar.

n bileşeni olan arrayin indisleri ($i = 0, 1, 2, \ldots n - 1$) olur.



Dizilerin Özellikleri

- Diziler tanımlanırken;

Dizinin adı, dizinin boyutu, dizi elemanlarının hangi tipte olacağı belirtilir.

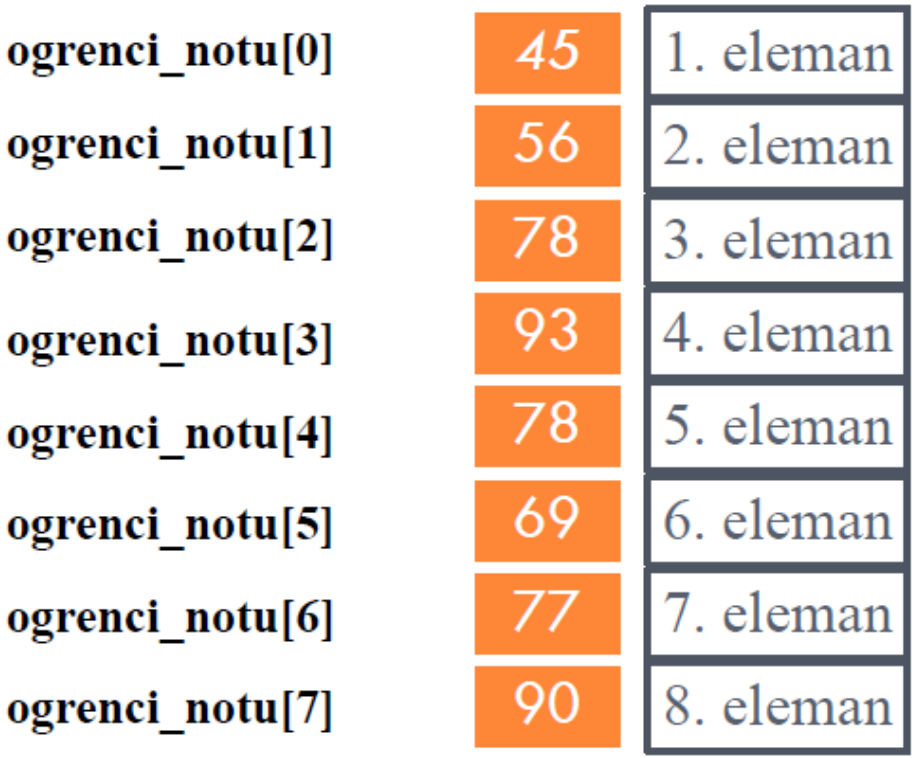
veritipi diziAdi[elemanSayisi];

$$int \quad a[n];$$

- Örnek:

double türündeki 8 adet öğrenci notunu bellekte tutmak için aşağıdaki gibi bir dizi tanımlayabiliriz:

double ogrenci_notu[8];



Dizilerin Özellikleri

- Örnek:

double türündeki 8 adet öğrenci notunu bellekte tutmak:

Dizileri kullanmasaydık;

```
double ogrenci_notu1;
double ogrenci_notu2;
double ogrenci_notu3;
double ogrenci_notu4;
double ogrenci_notu5;
double ogrenci_notu6;
double ogrenci_notu7;
double ogrenci_notu8;
```

Dizilerin Özellikleri

- Array bileşenlerinin kullanımı:

```
printf( " Sonuc:%f ", ogrenci_notu[0] + ogrenci_notu[1] );
```

- Dizinin yedinci elemanının değerinin 2'ye bölünüp x değişkenine atanması:

```
x=ogrenci_notu[6]/2;
```



- Dizinin yedinci elemanı: ogrenci_notu[6]
- Dizinin yedi numaralı elemanı: ogrenci_notu[7]

Dizilerde Atama ve Döngü Kurma

- Bir dizinin uzunluğu belirtilmeden de başlangıç değeri atamak mümkündür.

Örnek:

- int a[]={100, 200, 300, 400};
- float v[]={9.8, 11.0, 7.5, 0.0, 12.5};
- Derleyici bu şekilde bir atama ile karşılaştığında, küme parantezi içindeki eleman sayısını hesaplar ve dizinin o uzunlukta açıldığını varsayar.

- Yukarıdaki örnekte, a dizisinin 4, v dizisinin 5 elemanlı olduğu varsayılır
- int sayilar[20]={};
  - Tüm dizi elemanlarına 0 değeri atanır…

Diziler (Arrays)

- Örnek:

```c
#include<stdio.h>

int main()
{
    int i=0;
    int sayilar[20];
    for (i=0; i<20; i++ )
        {
        printf("Dizinin %d. elemani: %d'dir. \n", i+1, sayilar [i]);
        }


}
```

```
Dizinin 1. elemani: 4202608'dir.
Dizinin 2. elemani: 0'dir.
Dizinin 3. elemani: 164'dir.
Dizinin 4. elemani: 0'dir.
Dizinin 5. elemani: 4616768'dir.
Dizinin 6. elemani: 0'dir.
Dizinin 7. elemani: 1'dir.
Dizinin 8. elemani: 0'dir.
Dizinin 9. elemani: -1'dir.
Dizinin 10. elemani: -1'dir.
Dizinin 11. elemani: 164'dir.
Dizinin 12. elemani: 0'dir.
Dizinin 13. elemani: 1'dir.
Dizinin 14. elemani: 0'dir.
Dizinin 15. elemani: 4203641'dir.
Dizinin 16. elemani: 0'dir.
Dizinin 17. elemani: 3'dir.
Dizinin 18. elemani: 0'dir.
Dizinin 19. elemani: 164'dir.
Dizinin 20. elemani: 0'dir.
```

```c
#include<stdio.h>

int main()
{
    int i=0;
    int sayilar[20]={};
    for (i=0; i<20; i++ )
        {
        printf("Dizinin %d. elemani: %d'dir. \n", i+1, sayilar [i]);
        }


}
```

```
Dizinin 1. elemani: 0'dir.
Dizinin 2. elemani: 0'dir.
Dizinin 3. elemani: 0'dir.
Dizinin 4. elemani: 0'dir.
Dizinin 5. elemani: 0'dir.
Dizinin 6. elemani: 0'dir.
Dizinin 7. elemani: 0'dir.
Dizinin 8. elemani: 0'dir.
Dizinin 9. elemani: 0'dir.
Dizinin 10. elemani: 0'dir.
Dizinin 11. elemani: 0'dir.
Dizinin 12. elemani: 0'dir.
Dizinin 13. elemani: 0'dir.
Dizinin 14. elemani: 0'dir.
Dizinin 15. elemani: 0'dir.
Dizinin 16. elemani: 0'dir.
Dizinin 17. elemani: 0'dir.
Dizinin 18. elemani: 0'dir.
Dizinin 19. elemani: 0'dir.
Dizinin 20. elemani: 0'dir.

--------------------------------
Process exited after 0.01189 s
```

Dizilerde Atama ve Döngü Kurma

- Dizilere başlangıç değeri atarken, tüm elemanlara değer vermeden de atama yapmak mümkündür.

Örnek:

- int sayilar[20]={1, 2, 3};
  - Dizinin ilk 3 elemanına 1, 2 ve 3 değerleri atanır. 4'ten itibaren olan dizi elemanlarına 0 değeri atanır…
- Sayısal tipteki dizi elemanlarına 0 değeri, metin tipindeki dizi elemanlarına NULL değeri atanır.

Dizilerde Atama ve Döngü Kurma

```
#include<stdio.h>
#include<stdlib.h>       //Dizideki her bir elemanın değerinin 3 ile çarpılması
int main ()
{
        int ornek_dizi [5]={1,2,3,4,5};              //Diziyi oluşturma
        int i;
        for (i=0; i<5; i++ )                 //Dizi elemanlarına değer atama
                {
                ornek_dizi [i]=ornek_dizi [i]*3;
                }
        for (i=0; i<5; i++ )
                {                            //Dizi elemanlarını yazdırma
                printf("Dizinin %d. elemani: %d'dir. \n", i+1, ornek_dizi [i]);
                }
        return(0);
}
```

```
Dizinin 1. elemani: 3'dir.
Dizinin 2. elemani: 6'dir.
Dizinin 3. elemani: 9'dir.
Dizinin 4. elemani: 12'dir.
Dizinin 5. elemani: 15'dir.

--------------------------------
```

Dizilerde Atama Örnekleri

```
int x[5]={};     //5 elemanlı sayısal x dizisi oluşturma
int i=2;

x[0]=20;         //Geçerli atama
x[2.3]=5;        //Geçersiz atama (Yanlış)
x[2*i–3]=3;      //Geçerli atama, x[1] dizi elemanına 3 değerini atar
x[i++];          //Önce x[2] dizi elemanına erişilir daha sonra i
                 //değişkenine 3 değeri atanır
x[(int)x[1]];    //x[3] dizi elemanına erişilir
```

Dizi Elemanlarını Kullanıcıdan Okumak

- Klavyeden maksimum 10 tane sayısal değer girilecektir.
- Girilen sayılar bir dizide saklanacaktır.
- Sayı girme işlemi 0 girilene kadar devam edecektir.
- 0 değeri girildiğinde 0 sayısı hariç girilen diğer tüm değerler diziden okunarak ekrana yazdırılacaktır.

```c
#include<stdio.h>
#include<stdlib.h>
int main ()                     //Kullanıcıdan sayısal değer okuma
{
    int ornek_dizi [10];        //Diziyi oluşturma
    int i, j;
    for (i=0; i<10; i++ )       //Dizi elemanlarına kullanıcıdan değer alma
        {
        printf ("%d. sayiyi giriniz", i+1);
        scanf("%d", &ornek_dizi[i]);
        if(ornek_dizi[i]==0)
            break;
            j=i;
        }
    for (i=0; i<=j; i++ )       //Dizi elemanlarını yazdırma
        {
        printf("%d \n", ornek_dizi[i]);
        }
    return(0);
```

```
C:\Users\Kemal\Desktop\G
1. sayiyi giriniz4
2. sayiyi giriniz6
3. sayiyi giriniz8
4. sayiyi giriniz2
5. sayiyi giriniz3
6. sayiyi giriniz4
7. sayiyi giriniz0
4
6
8
2
3
4

--------------------
Process exited after
Press any key to cont
```

Dizilerle ilgili Önemli Hatırlatmalar

- Dizi boyunca döngü kullanırken dizi indisi asla 0'ın altına inmemeli ve her zaman dizideki toplam eleman sayısından az olmalıdır (büyüklük-1).

- Döngü devam şartının bu aralığın dışındaki elemanlara ulaşılmasını engellediğinden emin olmamız gereklidir.

- Dizi sınırlarının dışındaki elemanları kullanmanın yaratacağı hatalar (genelde ciddi hatalardır) sistemden sisteme farklılık gösterir.

Dizilerin Bazı Dezavantajları

- Dizinin boyu değiştirilemez:

Dizinin boyunu (boyutunu, bileşenlerinin sayısını) ya bildiriminde belirtiriz ya da bileşenlerine ilk değerlerini atayarak belirlemiş oluruz. Her durumda, dizinin boyu başlangıçta kesin belirlenmiş olur.

- Dizinin bileşen değerleri aynı veri tipindendir:

Bazı durumlarda farklı veri tiplerinden oluşan veri koleksiyonlarını işlemek gerekebilir. O zaman, bileşenleri aynı veri tipinden olan dizi yapısı işimize yaramaz. Java, C#, Python, Ruby gibi yeni diller farklı veri tiplerinden oluşan koleksiyonlarla iş yapmayı sağlayan yapılar getirmişlerdir.

Çok Boyutlu Diziler

- Bir dizi birden çok boyuta sahip olabilir. Örneğin iki boyutlu y dizisi şöyle tanımlanabilir:

```
int y[5][10];
```

- İki boyutlu diziler **matris** olarak adlandırılır.
- İlk boyuta **satır**, ikinci boyuta **sütün** denir. y matrisinin eleman sayısı 5x10=50'dir.

Çok Boyutlu Diziler

Dizi Çeşidi	Genel Bildirimi	Örnek
Tek boyutlu diziler (Vektörler)	**tip** dizi_adı[eleman_sayısı]	**int** veri[10];
İki boyutlu diziler (Matrisler)	**tip** dizi_adı[satır_sayısı][sutun_sayısı]	`float mat[5][4];`
Çok boyutlu diziler	**tip** dizi_adı[boyut_1][boyut_2]...[boyut_n]	`double x[2][4][2];`

Çok Boyutlu Diziler

- Çok boyutlu diziler tek boyuta indirgenerek bellekte tutulurlar.
- Tek indisli dizilerde olduğu gibi, çok indisli dizilere de başlangıç değeri vermek mümkündür.
- Örneğin 3 satır ve 4 sütunlu (3x4=12 elemanlı) bir x matrisinin elemanları şöyle tanımlanabilir:
- int x[3][4]={11, 34, 42, 60, 72, 99, 10, 50, 80, 66, 21, 38};

- int x[3][4]={11, 34, 42, 60, /*1.satırelemanları*/
  72,99,10,50, /*2.satırelemanları*/
  80,66,21,38}; /*3.satırelemanları*/

Çok Boyutlu Diziler

```c
#include<stdio.h>
#include<stdlib.h>
int main ()                          //İki Boyutlu Dizi Oluşturma ve Yazdırma
{
    int i, j;
    int x[3][4]={11,34,42,60,                //Diziyi oluşturma
                 72,99,10,50,
    80,66,21,38};
    for (i=0; i<3; i++ )                     //Diziyi yazdırma
    {
        for (j=0; j<4; j++ )
        {
        printf ("%d ", x[i][j]);
        }
        printf ("\n");
    }
    return(0);
}
```

```
11 34 42 60
72 99 10 50
80 66 21 38

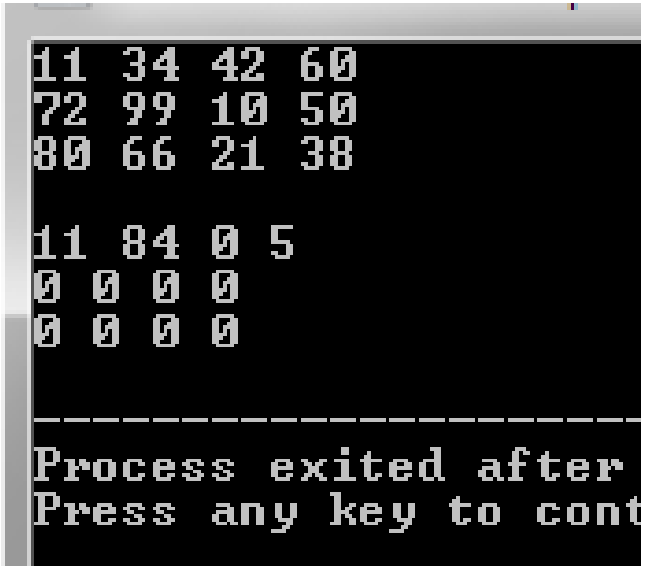
--------------
Process exited
Press any key t
```

Çok Boyutlu Diziler

- int x[3][4]={11, 34, 42, 60, 72, 99, 10, 50, 80, 66, 21, 38};
- int y[3][4]={}; => 0000
  0000
  0000

- y[0][0]=y[0][0]+x[0][0];
- y[0][1]=x[0][2]*2;
- y[0][3]=5;

- for (i=0; i<3; i++) //y dizisini yazdırma
- {
- for (j=0; j<4; j++)
- {
- printf ("%d ", y[i][j]);
- }
- printf ("\n");
- }



ÖDEV:

Bir dersi alan 10 öğrencinin vize ve final notlarını dizilerde saklayabilecek bir program tasarlayın. Notların girişini kullanıcıdan isteyerek dizilere kaydedin.

Bu notlara göre vize notu %40, final notu %60 olacak şekilde her bir öğrencinin yıl sonu puanını, harf karşılığını ve sınıf ortalamasını ekrana yazdırın.

KAYNAKLAR:

- Programlama Sanatı, Algoritmalar, C Dili Uyarlaması, Dr. Rifat ÇÖLKESEN, Papatya Yayıncılık
- Her Yönüyle C, Tevfik KIZILÖREN, Kodlab
- C Programlama Dili, Dr. Rifat ÇÖLKESEN, Papatya Yayıncılık
- Celal Bayar Üniversitesi, Hasan Ferdi Turgutlu Teknoloji Fakültesi, YZM1105 Ders Notu



BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 8 – Sıralama ve Arama

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

20/04/2021

Konya

Diziler ve matrislerin temel kullanım nedeni sıralama ve arama gibi bazı problemlere değişkenlere göre çok daha basit çözüm sağlamalarıdır. Bir dizinin kaç tane elemandan oluştuğu önemli olmamakla birlikte bu işlemler kolaylıkla yapılabilmektedir.

Öte yandan örneğin bir sayı dizisinin sıralanması işleminde sıralanacak sayılar bir dizi üzerinde değil de farklı değişkenler üzerinde tutuluyor ise işlem karmaşıklaşacak hatta belki imkansız duruma dönüşecektir.

Dizilerde Arama ve Sıralama

- Arama ve sıralama algoritmaları programlama ve yazılım geliştirme dünyasında hem akademik hem de endüstri açısından önemli bir yere sahiptirler.

- Özellikle büyük veri kaynakları ile çalışırken, aradığınız veriye en hızlı şekilde ulaşmanız doğru algoritmayı kullanmanıza bağlıdır.



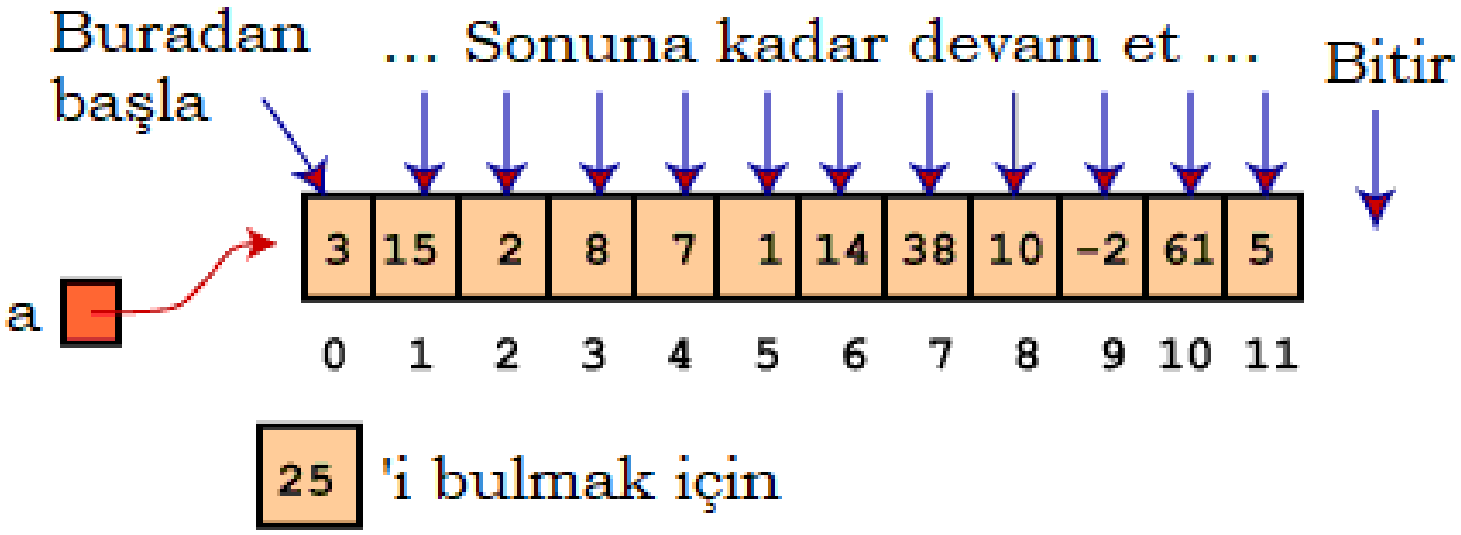
Dizilerde Arama

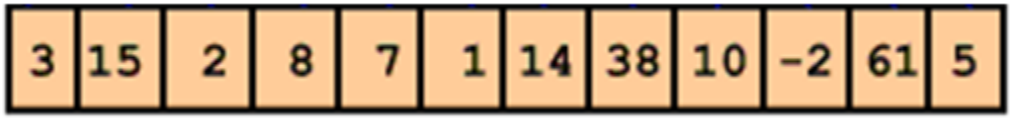
- Bir dizinin, belli bir arama değerine eşit olan bir değer içerip içermediğine karar vermek gerektiğinde kullanılır.
- Dizinin belirli bir elemanını bulma sürecine arama denir.
- Ders kapsamında iki tane arama tekniği üzerinde durulacaktır:
  - Doğrusal Arama (Linear / Sequential Search)
  - İkili Arama (Binary Search)

Doğrusal Arama

- Sıralı/sıralı olmayan bir dizinin ilk elemanından başlanarak dizinin her elemanı, arama değeriyle karşılaştırılır.
- Aranan değer bulunduğunda ilgili dizi elemanının indisi değer olarak döndürülür.
- Dizi elemanları herhangi bir şekilde sıralanmadığından değer ilk ya da son elemanda bulunabilir.
- Dolayısıyla, program ortalama olarak, arama değeriyle dizinin elemanlarının yarısını karşılaştırmalıdır.

Doğrusal Arama






```c
#include<stdio.h>

int main() {
    int aranan, i, bulundu=0;
    int dizi[12]={3, 15, 2, 8, 7, 1, 14, 38, 10, -2, 61, 5};
printf("Dizi icinde aramak istediginiz sayiyi giriniz:\n");
scanf ("%d", &aranan);

for(i=0; i<12; i++) {
    if(dizi[i]==aranan) {
    bulundu = 1;
    break;              }
   }
if(bulundu) /* if(bulundu==1) */
    printf("Aranan sayi, dizinin %d.elemanidir.", i+1);
else
    printf("Aranan eleman bu dizide yoktur.\n");
return 0; }
```

İkili Arama

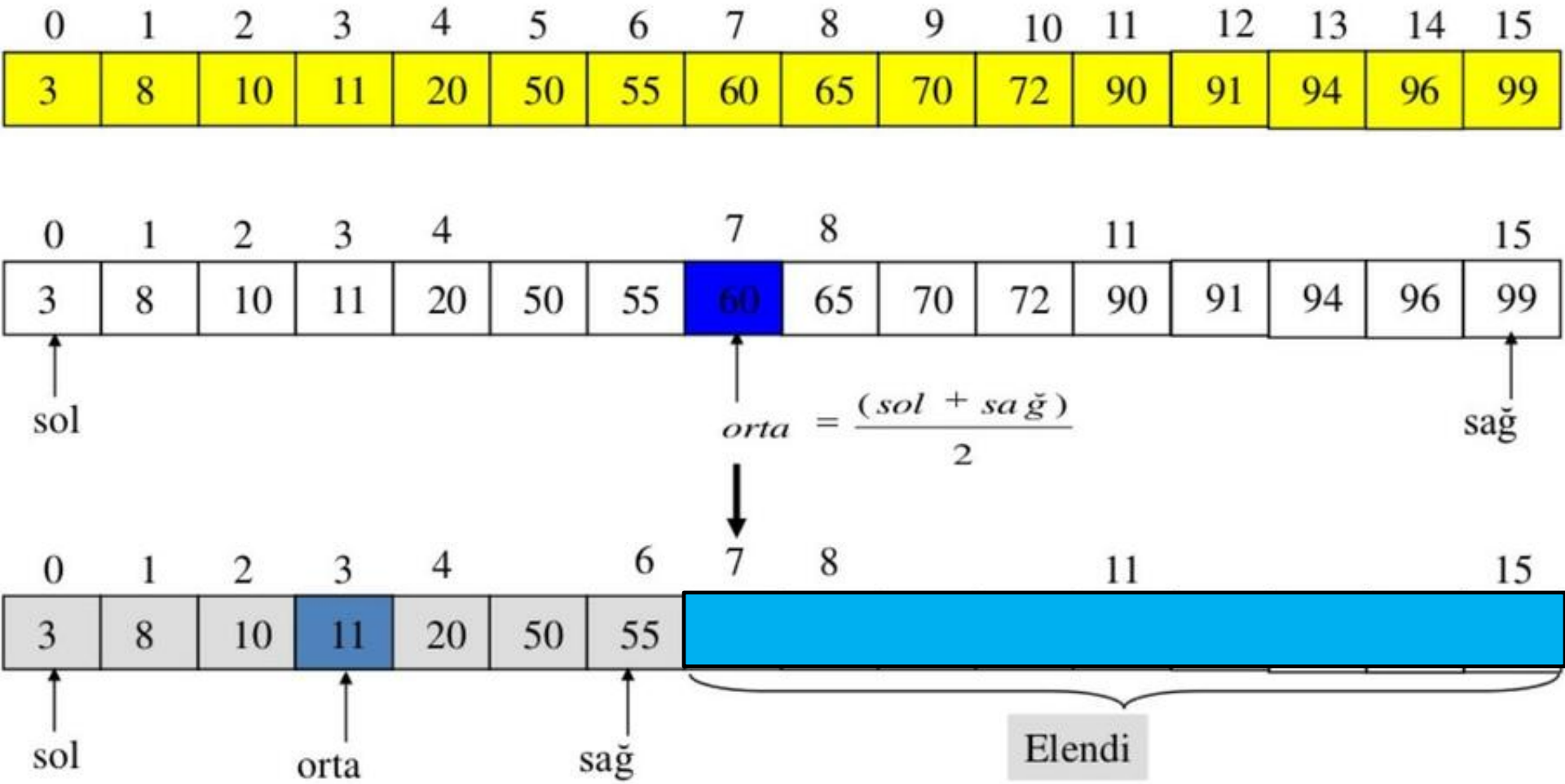
Arama yapılacak dizinin **sıralı bir dizi** olması gerekir. Sıralı değilse, ikili arama yapabilmek için öncelikle dizinin herhangi bir sıralama algoritması ile sıralanması gerekir.

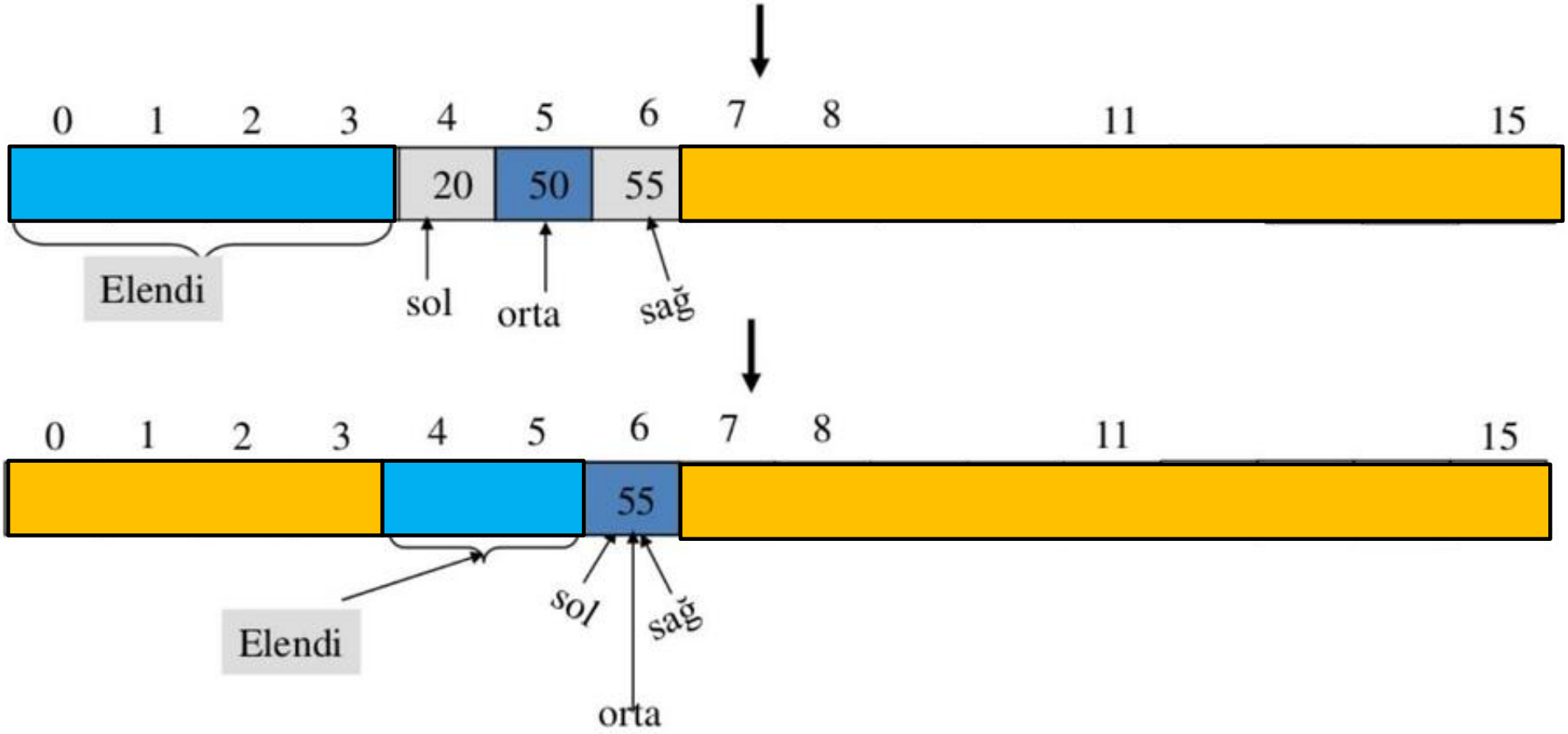
Bu teknikteki her bir adımda, aranan değerin, dizinin orta değerine eşit olup olmadığı kontrol edilir.

- Eşit olmaması durumunda aranan değerin orta değer tarafından ikiye ayrılan kısımlardan hangisinde olduğu kontrol edilir.

- Aranan değeri içeren kısım bir sonraki adımda arama yapılacak dizi olur ve bu sayede arama yapılan listedeki eleman sayısı her adımda yarıya düşer.

ÖRNEK: Aşağıdaki dizide 55 sayısını arayalım.




0 1 2 3 4 5 6 7 8 11 15
20
50
55
Elendi
sol
orta
sağ
0 1 2 3 4 5 6 7 8 11 15
55
Elendi
sol
sağ
orta

```c
#include <stdio.h>
int main()
{
    int array[16] = {3,8,10,11,20,50,55,60,65,70,72,90,91,94,96,99};
    int sol = 0;
    int sag = 15;
    int flag = 0;
    int s;
printf("Dizi icinde aramak istediginiz sayiyi giriniz:\n");
scanf ("%d", &s);

    while(sol <= sag){
        int indis = (sol+sag )/2;
        if(array[indis] == s){
            flag = 1;
            printf("Bulundu: %d \n",indis);
            break;      }
        else if(array[indis] < s){
            sol   = indis+1;    }
        else{
            sag = indis-1;      }
    }
    if(flag == 0){
        printf("Bulunamadi!\n");
    }

    return 0;
}
```

Dizilerde Sıralama

Diziler üzerinde yapılan işlemlerde döngülere en çok ihtiyaç duyulan işlemlerden biri de sıralama işlemidir.

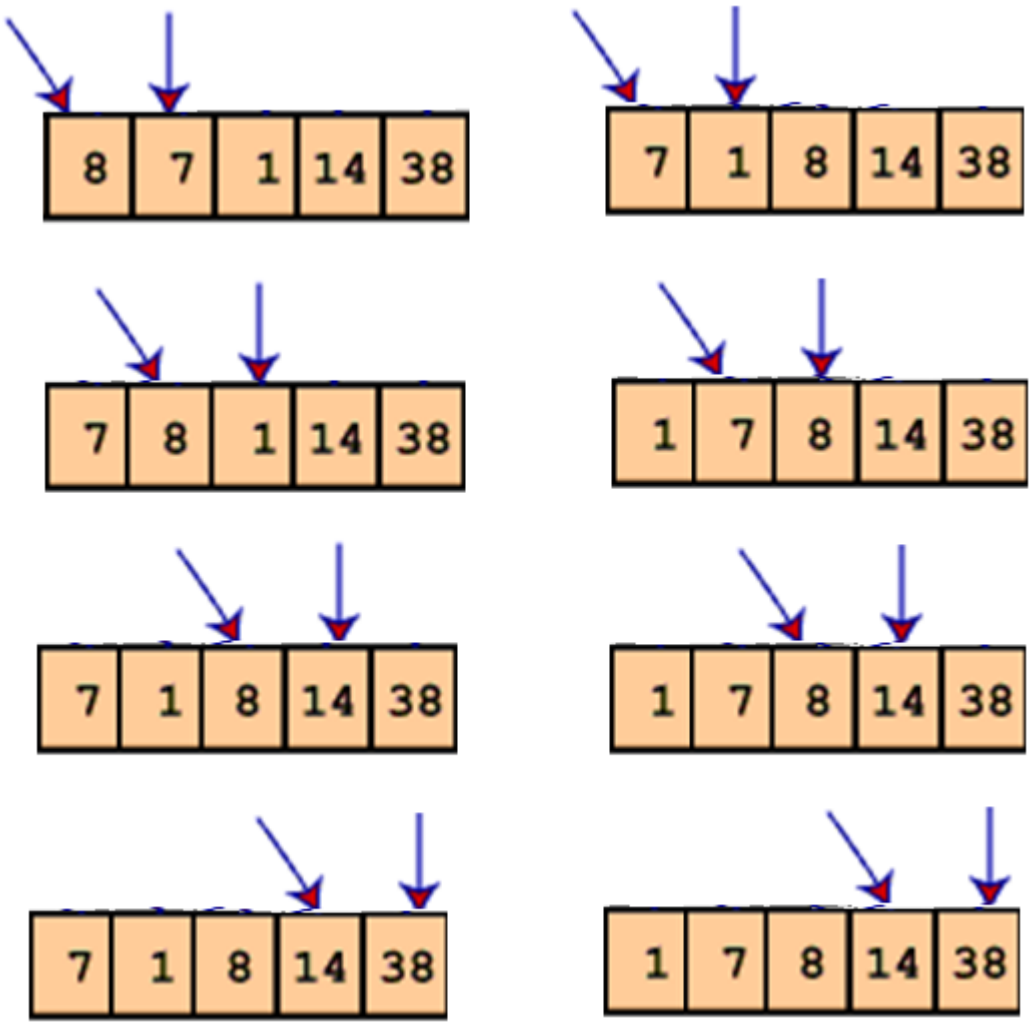
Sıralama algoritmalarında genellikle dizinin elemanlarını gezmek için bir döngü ve bu döngünün her iterasyonunda da dizinin ilgili elemanlarını karşılaştırmak ya da yer değiştirmek için ikinci bir döngü kullanılır.

Ders kapsamında, sıralama algoritmalarından **kabarcık sıralama (bubble sort)** ve **hızlı sıralama (quick sort)** algoritmaları üzerinde durulacaktır.

Kabarcık Sıralama

Kabarcık denmesinin sebebi bu algoritmada en büyük sayının en sona atılmasıdır. Her iterasyonda en büyük sayı en sona atılır ve iterasyonlar tamamlandığında sıralama gerçekleşmiş olur.

Her adımda dizinin ardışık iki elemanını karşılaştırır. Eğer ilk eleman ikinci elemandan büyükse yer değiştirme yapılır. Her iterasyonda en büyük sayı sona gelir, iterasyonlar tamamlandığında algoritma tamamlanmış olur.


8 7 1 14 38
7 1 8 14 38
7 8 1 14 38
1 7 8 14 38
7 1 8 14 38
1 7 8 14 38
7 1 8 14 38
1 7 8 14 38

```c
#include <stdio.h>
#include <stdlib.h>
/* Kabarcık Sıralama (Bubble Sort) */
int main(){

    int son, gecici;
    int dizi[100];
    int i,j;
    printf("Girilecek sayi adedi:\n ");
    scanf("%d", &son);

    for(i=0; i<son; i++){
        printf("%d. Sayiyi giriniz: ", i+1);
        scanf("%d", &dizi[i]);    }

    for(i=0; i<son; i++)
        printf("%d ", dizi[i]);
    printf("\n\n");

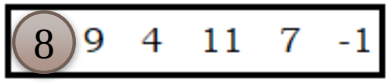
    for(i=0; i<son; i++){
        for(j=0; j<son-1-i; j++){
            if(dizi[j] > dizi[j+1]){
                gecici = dizi[j];
                dizi[j] = dizi[j+1];
                dizi[j+1] = gecici;
            }
        }
    }

    for(i=0; i<son; i++)
        printf("%d ", dizi[i]);
        printf("\n");
    return 0;
}
```

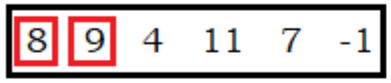
Hızlı Sıralama

Hızlı sıralama işlemi, C standart kütüphanesinde «**qsort()**» adıyla hazır bir fonksiyon olarak mevcuttur. Bu fonksiyona sıralama yapılacak dizi, sıralanacak elemanların başlangıç indisi (sol) ve bitiş indisi (sağ) gönderilmelidir. Böl ve yönet mantığına dayanır.

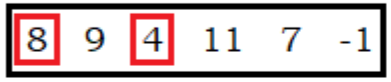
Dizinin ilk elemanı pivot seçilir, sağındaki elemanlar ise pivottan büyük ya da küçük olma durumuna göre sağa ya da sola yazılır. İşlem bitince soldaki ve sağdaki gruplar içinde sıralama devam eder.



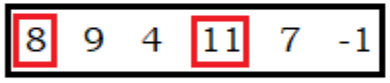
8



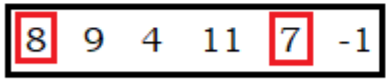
8
9



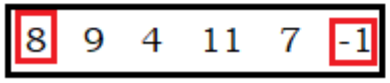
8
4 9



8
4 9 11



8
4 7 9 11



8
4 7 -1 9 11



Pivot ile karşılaştırılan sayı pivottan küçükse pivotun soluna, büyükse sağına yazılır.

8

4 7 -1 9 11

4 7 9 11

4 7 -1

4
-1 7

-1 4 7 8 9 11

Matris Toplama

$$\begin{bmatrix} 1 & 3 \\ 1 & 2 \end{bmatrix} + \begin{bmatrix} 7 & 5 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 8 & 8 \\ 3 & 3 \end{bmatrix}$$

```
for(i=0; i<2; i++){
        for(j=0; j<2; j++){
                C[i][j] = A[i][j] + B[i][j];
                printf("%d ", C[i][j]);
        }
        printf("\n");   }
```

Matris Çarpma

$$\begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix} \cdot \begin{bmatrix} 4 & 3 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$c_{11} = 1.4 + 2.1 = 6$$
$$c_{12} = 1.3 + 2.2 = 7$$

```
for(i=0; i<2; i++){
        for(j=0; j<2; j++){
                for(k=0; k<2; k++){
                        C[i][j] += A[i][k] * B[k][j];
                }
                printf("%d ", C[i][j]);   }

                printf("\n"); }
```

ÖDEV - 1:

Türkiye’de geçtiğimiz hafta tespit edilen günlük Covid-19 vakalarının sayısını bir diziye aktararak en az vaka sayısının kaçıncı gün tespit edildiğini ekrana yazan bir C kodu yazınız.

ÖDEV – 2:

Aşağıda Konya iline ait MGM' nin 5 günlük hava tahminleri ve geçmiş yılların ortalama en düşük ve en yüksek sıcaklık değerleri verilmiştir. Matris işlemlerinden yararlanarak, tahmini en düşük ve en yüksek sıcaklık değerlerinin mevsim ortalamalarının ne kadar altında/üstünde olduğunu hesaplayıp bir matris halinde ekrana yazdıran bir C kodu yazınız.

TARİH	TAHMİN EDİLEN			GEÇMİŞTE GERÇEKLEŞEN	
	Hadise	Sıcaklık (°C)		Ortalama Sıcaklık (°C)	
		En Düşük	En Yüksek	En Düşük	En Yüksek
20 Nisan		13	26	4,5	16,7
21 Nisan		7	23	4,8	16,5
22 Nisan		8	23	4,8	17
23 Nisan		9	28	4,4	17,5
24 Nisan		11	30	5,3	18,2

Kaynaklar

- **Programlama Sanatı, Algoritmalar, C Dili Uyarlaması**
  Dr. Rifat ÇÖLKESEN, Papatya Yayıncılık
- **Her Yönüyle C**
  Tevfik KIZILÖREN, Kodlab
- https://www.mgm.gov.tr/tahmin/il-ve-ilceler.aspx?il=KONYA

BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 9– Fonksiyonlar

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

18/05/2021

Konya

Fonksiyonlar

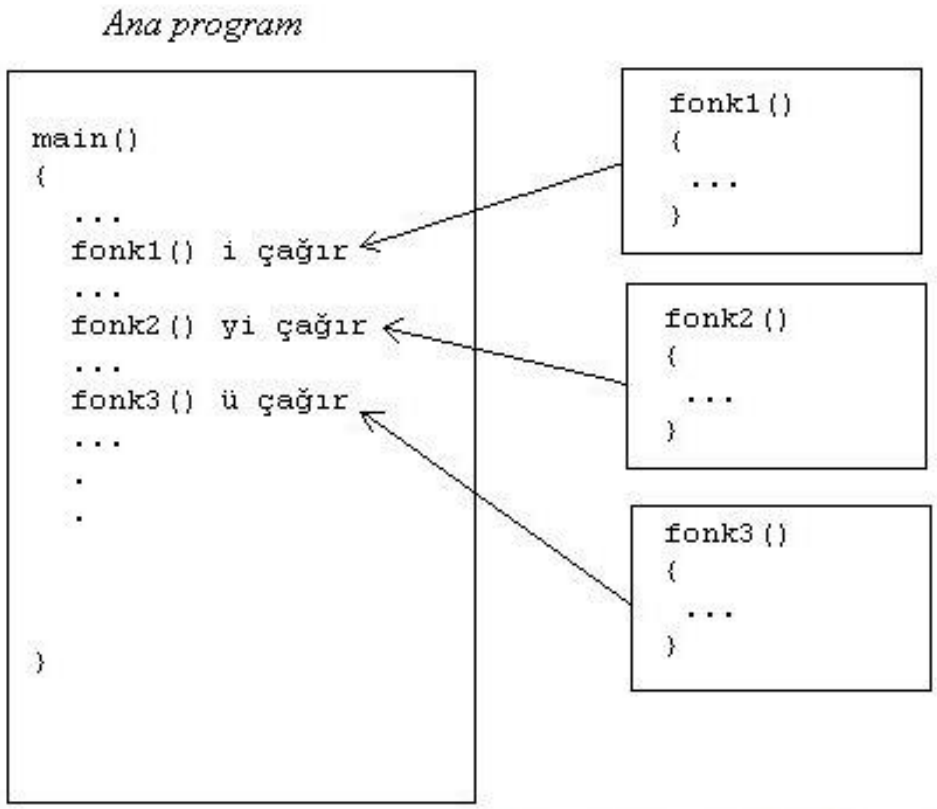
- C dilindeki fonksiyonları, girdisi ve çıktısı olan kapalı bir kutu gibi düşünebiliriz.
- Fonksiyonlar girdileri alır, bunlara göre işlemleri gerçekleştirirler ve isteğe bağlı olarak sonuç üretirler.
- Biz de fonksiyonların ürettiği bu çıktıları kullanırız.
- printf, scanf, sqrt, vb… birer fonksiyon örneğidir.
- Her bir fonksiyonu kendi işini yapan bir altprogram gibi düşünebiliriz.
- main() fonksiyonu da, program çalıştırıldığında otomatik olarak çağrılan bir fonksiyondur.

Neden Fonksiyonlara İhtiyaç Var?

- Geniş programları yazarken,

küçük parçaları ya da her biri orijinal programdan daha kolay kullanılabilecek modülleri (daha önceden hazırlanmış program parçacıkları) birleştirmek kolaylık sağlar.

- Karmaşık yapılı programları sadeleştirir ve bu programları modüler bir hale getirir.
- Bazen standart kütüphanelerde tanımlı fonksiyonlar bütün ihtiyaçlarımızı karşılayamazlar.
- Fonksiyonlar, programcıların tekrarlanan kodlar yazmalarını önlerler.

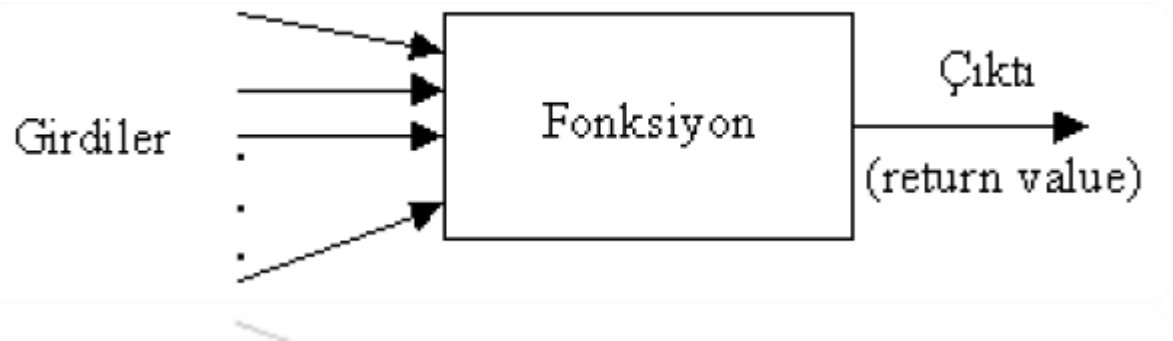


Fonksiyonlar

- Bir fonksiyon çağrıldığında;
  - Fonksiyon kendisine parametre olarak verilen veriler üzerinde çeşitli işlemler gerçekleştirir.
  - İstenildiği taktirde bu işlemlerin sonucunu çağrıldığı yere geri döndürür (geri yollar).

Örnek: toplamaYap fonksiyonu

- Girdiler: 2 ve 6 ise çıktı: 8
- Girdiler: 10 ve 15 ise çıktı: 25



Fonksiyonun Yapısı

- C'de fonksiyon bir veya daha fazla işlem satırından oluşan kodların bir kod bloğu şeklinde yapılandırılması ile oluşturulur. 3 Temel Parçadan oluşur.
- **Fonksiyon Yapısı:**

```
dönüşVeriTipi fonksiyonAdı (argümanTürü);                    //Fonksiyon bildirimi

main() {                                                     //main Ana Program
        fonksiyonAdı ( argümanlar);                          //Fonksiyonun çağrılması
}

dönüşVeriTipi fonksiyonAdı (argümanTürü ve adları)           //Fonksiyon gövdesi
{
        yerel bildirimler;
        işlem satırları
        …
        işlem satırları
        return ifade;
}
```

Fonksiyonun Yapısı

```
dönüşVeriTipi fonksiyonAdı (argümanTürü ve adları)          //Fonksiyon gövdesi
{
        yerel bildirimler;          // Yerel bildirimler/değişkenler sadece içinde
        işlem satırları             // bulundukları fonksiyonda kullanılabilir
        …
        işlem satırları
        return ifade;
}
```

- **veri-tipi :** Fonksiyonun geri verdiği veri tipini gösterir.
- **fonksiyon-adı :** Fonksiyon adını gösterir.
- **argümanlar :** Fonksiyona geçirilen verileri gösterir. Argüman yerine Parametre veya Girdi ifadesi de kullanılmaktadır.
- **işlem satırı :** Fonksiyon içindeki işlem satırlarını gösterir.
- **return :** Verileri geri döndürmeye yarar. Son satırda kullanılması şart değildir.
- **ifade** : Değişken, sabit ve işlemciler kullanılabilir. Elde edilen veri türü, fonksiyonun geri döndürdüğü veri türü ile aynı olmalıdır.

Fonksiyonun Yapısı-Örnek

- **Örnek 1 Toplama Yap Fonksiyonu:**

ders9.1.c

```c
#include <stdio.h>

int toplamaYap(int,int);          //Fonksiyon bildirimi


int main()                        // main Ana Program
{
    int a,b,toplam;
    printf ("Birinci sayiyi giriniz\n");
    scanf ("%d",&a);
    printf ("Ikinci sayiyi giriniz\n");
    scanf ("%d",&b);

    toplam=toplamaYap(a,b);       //Fonksiyonun çağrılması
    printf("Iki sayinin toplami=%d",toplam);

    return 0;
}

int toplamaYap(int sayi1,int sayi2)      //Fonksiyon gövdesi
{
    int sonuc;
    sonuc=sayi1+sayi2;
    return sonuc;
}
```

```
C:\Users\Kemal\Desktop\Google Drive\
Birinci sayiyi giriniz
2
Ikinci sayiyi giriniz
8
Iki sayinin toplami=10
--------------------------------
Process exited after 5.004 se
Press any key to continue . .
```

Return ve Veri Tipi İfadesi

- Bir fonksiyonun geri verdiği veri tipi **dizi** dışında herhangi bir tür olabilir.
  - Fonksiyon dizi tipinde bir veri geri döndüremez, ancak farklı yöntemlerle bu işlemi gerçekleştirilebiliriz.
- veri-tipi **ifadesi**:
  - Eğer bir fonksiyonun adının başında veri türü tanımlanmazsa, fonksiyon int bir değer geri verir.
  - Eğer bir fonksiyonun int değer dışında bir veri türü geri vermesi isteniyorsa, fonksiyon adının başında mutlaka bir veri türü tanımlanması gerekir.

Return ve Veri Tipi İfadesi

- Örnekler:

```
return (a+b/c);          /*parantez kullanmak zorunlu değil*/
return 10;               /*değişken kullanmak mecbur değil*/
return topla(a,b)/2;     /*önce topla fonksiyonu çalışır*/
```

- Programın çözüm mantığına göre bir fonksiyon içerisinde birden çok geri dönüş değeri kullanılabilir.
- Fakat, ilk karşılaşılan return deyiminden sonra fonksiyon sonlanır ve çağrılan yere bu değer gönderilir

Fonksiyon Bildirimi

- Bir veri türü geri veren bir fonksiyon kullanılacağı zaman, bu fonksiyonu kullanmadan önce, fonksiyonun geri vereceği veri türünün, fonksiyona geçirilecek argümanların sayısının ve türlerinin programın başında belirlenmesi gerekir. Bu işleme **Fonksiyon bildirimi** adı verilir.
- Fonksiyon bildiriminde geri döndürülecek veri türü belirtilmediği takdirde, bazı derleyiciler bu fonksiyonun geri vereceği değeri int olarak kabul eder.
- Eğer fonksiyonun yapısı main() fonksiyonundan önce tanımlanırsa, ayrıca prototip yöntemiyle bir bildirim yapılmasına ihtiyaç duyulmaz.

Fonksiyon Bildirimi- Örnek

- **Örnek 2 Toplama Yap Fonksiyonu (float değişkeni kullanılarak):**

```
1   #include <stdio.h>
2
3   //float toplamaYap(float,float);          //Fonksiyon bildirimi
4
5
6   int main()                          // main Ana Program
7   {
8       float a,b,toplam;
9       printf ("Birinci sayiyi giriniz\n");
10      scanf ("%f",&a);
11      printf ("Ikinci sayiyi giriniz\n");
12      scanf ("%f",&b);
13
14      toplam=toplamaYap(a,b);      //Fonksiyonun çağrılması
15      printf("Iki sayinin toplami=%f",toplam);
16
17      return 0;
18  }
19
20  float toplamaYap(float sayi1,float sayi2)       //Fonksiyon gövdesi
21  {
22      float sonuc;
23      sonuc=sayi1+sayi2;
24      return sonuc;
25  }
```

klar | Derleme Mesajları | Hata ayıkla | Arama sonuçları | Kapat

	Mesaj
ıal\Deskto...	[Error] conflicting types for 'toplamaYap'
ıal\Deskto...	[Note] previous implicit declaration of 'toplamaYap' was here

Void (Boş) Fonksiyon

- Bir fonksiyon için geri dönen veri türü ve parametre tanımlanmayacaksa, bu ifadelerin yerine **void** ifadesini kullanarak bu durumu derleyiciye bildirebiliriz.
- main() fonksiyonundan önce de void ifadesini kullanılabilir.
- Parantez içine void yazılmaması, yani boş bırakılması da aynı anlama gelir.

- void fonksiyonAdı (void)

```
{
  işlem satırları
}
```

Fonksiyon Bildirimleri

Örnek	Açıklama
int islem();	Tam sayı değer dönen ve parametre girdisi içermeyen bir fonksiyon
int islem(void);	Tam sayı değer dönen ve parametre girdisi içermeyen bir fonksiyon
int islem(int x);	Tam sayı değer dönen ve tam sayı türünde parametre girdisi olan bir fonksiyon
void islem();	Değer dönmeyen ve parametre girdisi olmayan bir fonksiyon
void islem(int x);	Değer dönmeyen ve tam sayı türünde parametre girdisi olan bir fonksiyon

Void (Boş) Fonksiyon Örnek

- Örnek 3:

ders9.3.c

```c
#include <stdio.h>

float toplamaYap(float,float);              //Fonksiyon bildirimleri
void herkesiSelamla();

int main()                                  // main Ana Program
{
    float a,b,toplam;
    printf ("Birinci sayiyi giriniz\n");
    scanf ("%f",&a);
    printf ("Ikinci sayiyi giriniz\n");
    scanf ("%f",&b);

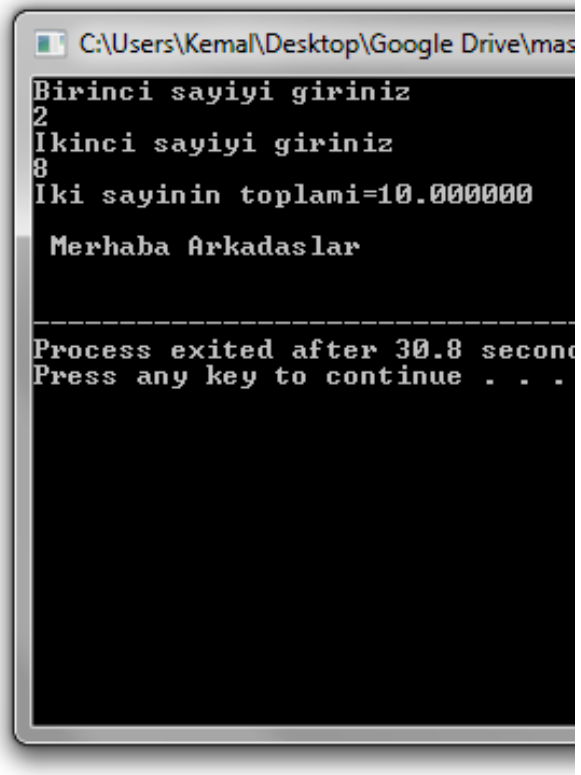
    toplam=toplamaYap(a,b);                 //1. Fonksiyonun çağrılması
    printf("Iki sayinin toplami=%f",toplam);

    herkesiSelamla();                       //2. Fonksiyonun çağrılması

    return 0;
}

float toplamaYap(float sayi1,float sayi2)   //1. Fonksiyon gövdesi
{
    float sonuc;
    sonuc=sayi1+sayi2;
    return sonuc;
}

void herkesiSelamla()                       //2. Fonksiyon gövdesi
{
    printf("\n\n Merhaba Arkadaslar \n\n");
}
```



İç içe Fonksiyonlar

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    mesaj();
    printf(" Hos geldiniz...\n");
    return 0;
}

void mesaj()
{
    printf("Algoritma");
    mesaj2();
}

void mesaj2()
{
    printf(" ve Programlama");
}
```



Fonksiyonlara Parametre Aktarımı

- Fonksiyonlara parametre aktarımı iki şekilde yapılabilir:

  1) Değer ile çağırma (Calling by Value)

  Bu yöntemde değişkenlerin değerleri çağrılan fonksiyonun içine gönderilir. Bu değerler çağrılan fonksiyon içindeki değişkenlere atanarak işlemler yapılır. Fonksiyondaki işlemler tamamlandığında ana fonksiyondaki değişkenlerde herhangi bir değişiklik olmaz.

  2) Adres ile çağırma (Calling by Reference)

  Değişkenlerin hafızada tutulduğu adresler çağrılan fonksiyona gönderilir. Fonksiyondaki işlemler sonucunda ilgili adreslerdeki değerler güncelleneceğinden değişkenlerin değeri de değişmiş olur.

Fonksiyonlara Parametre Aktarımı

Değer ile çağırma (Calling by Value): Örnek 4

ders9.4.c

```c
#include <stdio.h>

float ortalama(int, int, int);  //fonksiyon bildirimi

main()
{
    int m,n,p;
    float k;
    printf("\nBirinci sayiyi giriniz:");
    scanf("%d",&m);
    printf("\nIkinci sayiyi giriniz:");
    scanf("%d",&n);
    printf("\nUcuncu sayiyi giriniz:");
    scanf("%d",&p);
    k=ortalama(m,n,p);          //fonksiyonun çağrılması ve fonksiyondan
                                //geri gelecek sonuç değerinin k değişkenine atanması

    printf("\nHesaplanan Ortalama:%f",k);
}
float ortalama(int a, int b,int c)  //fonksiyonun tanımlanması
{
    float g;
    g=(a+b+c)/3.0;
    return g;
}
```

```
C:\Users\Kemal\Desktop\Google Drive\masaustu\

Birinci sayiyi giriniz:3

Ikinci sayiyi giriniz:5

Ucuncu sayiyi giriniz:6

Hesaplanan Ortalama:4.666667
--------------------------------
Process exited after 4.917 seconds w
Press any key to continue . . .
```

Fonksiyonlara Parametre Aktarımı

Adres ile çağırma (Calling by Reference): Örnek 5

```c
#include <stdio.h>;

int yerdegistir(int*, int*);

main()
{
    int x,y;
    printf("Bir sayi giriniz:");
    scanf("%d",&x);
    y=23;
    printf("x=%d    y=%d \n",x,y);  //x ve y'nin ilk hali
    yerdegistir(&x,&y);
    printf("x=%d    y=%d \n",x,y);  //x ve y'nin son hali
}
int yerdegistir(int *a, int *b)
{
    int g;
    g=*a;   //a'nın işaret ettiği yerdeki değer g'ye atanıyor
    *a=*b;  //b'nin işaret ettiği yerdeki değer a'ya atanıyor
    *b=g;   //g'de tutulan değer(a'nın ilk değeri) b'ye atanıyor

}
```

```
Bir sayi giriniz:3
x=3     y=23
x=23    y=3

--------------------------------
Process exited after 3.9 sec
Press any key to continue .
```

Fonksiyonlara Parametre Aktarımı

Karma Kullanım: Örnek 6

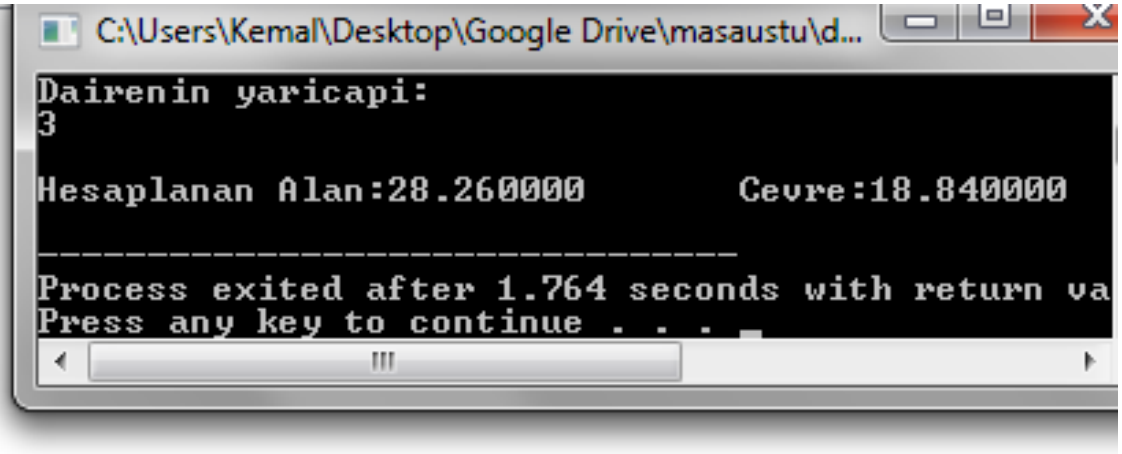
```c
#include <stdio.h>;

void daire(float, float*, float*);

main()
{
    float yaricap, alan, cevre;
    puts("Dairenin yaricapi:");
    scanf("%f",&yaricap);

    daire(yaricap, &alan, &cevre);
    printf("\nHesaplanan Alan:%f    Cevre:%f\n",alan,cevre);
}
void daire(float r, float *al, float*cev)
{
    *al=3.14*r*r;
    *cev=2*3.14*r;
    r=r+1;

}
```



Fonksiyonlara Parametre Aktarımı

Karma Kullanım (yarı çap değerinin değişmemesi): Örnek 7

```c
#include <stdio.h>;

void daire(float, float*, float*);

main()
{
    float yaricap, alan, cevre;
    puts("Dairenin yaricapi:");
    scanf("%f",&yaricap);

    daire(yaricap, &alan, &cevre);
    printf("\nHesaplanan Alan:%f    Cevre:%f\n",alan,cevre);

    printf("\nYaricapin son degeri:%f \n",yaricap);
}
void daire(float r, float *al, float*cev)
{
    *al=3.14*r*r;
    *cev=2*3.14*r;
    r=r+1;

}
```

```
C:\Users\Kemal\Desktop\Google Drive\masaustu\ders icin ornek dok
Dairenin yaricapi:
3

Hesaplanan Alan:28.260000       Cevre:18.840000

Yaricapin son degeri:3.000000

--------------------------------
Process exited after 8.772 seconds with return v
Press any key to continue . . .
```

Fonksiyonlara Parametre Aktarımı

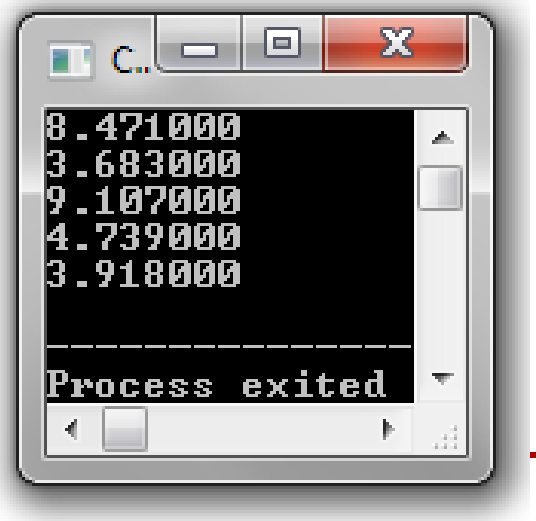
- Fonksiyon kullanımı planlanırken değer ile veya adresi ile çağırmanın hangisinin kullanılacağı fonksiyonun yapacağı işleme ve çağırana döndüreceği parametrelerin sayısına göre seçilmelidir.
- Çağırana tek bir sonuç döndürülecekse değer ile çağırma kullanılarak return ile değer döndürülmesi uygun olabilir.
- Ancak birden fazla değer döndürmek gerekiyorsa return kullanımı yetersiz kalabileceğinden dolayı adres ile çağırma yöntemi kullanılmalı ve değişkenlerin değerleri güncellenerek fonksiyondaki işlemlerin sonuçları çağıran programa döndürülmelidir.

Dizilerin Fonksiyonlarda Kullanımı

- Diziler de diğer değişkenler gibi fonksiyonlara gönderilip işleme tabi tutulabilir. Örnek 8:

ders9.8.c

```
1   #include <stdio.h>;
2
3   void dizi_yazdir(float x[], int n );
4
5   int main()
6   {
7       float dizi[5]={8.471, 3.683, 9.107, 4.739, 3.918};
8       dizi_yazdir(dizi, 5);
9       return 0;
10  }
11
12  void dizi_yazdir(float x[], int n)
13  {
14      int i;
15      for(i=0;i<n;i++)
16      {
17          printf("%f \n",x[i]);
18      }
19
20  }
```



Özyinelemeli (Recursive) Fonksiyonlar

- Belirli bir dönüş kriteri gelene kadar kendi kendini çağıran fonksiyonlara özyinelemeli (recursive) fonksiyon denir.

```c
#include <stdlib.h>

int faktoryel_hesapla(int sayi);

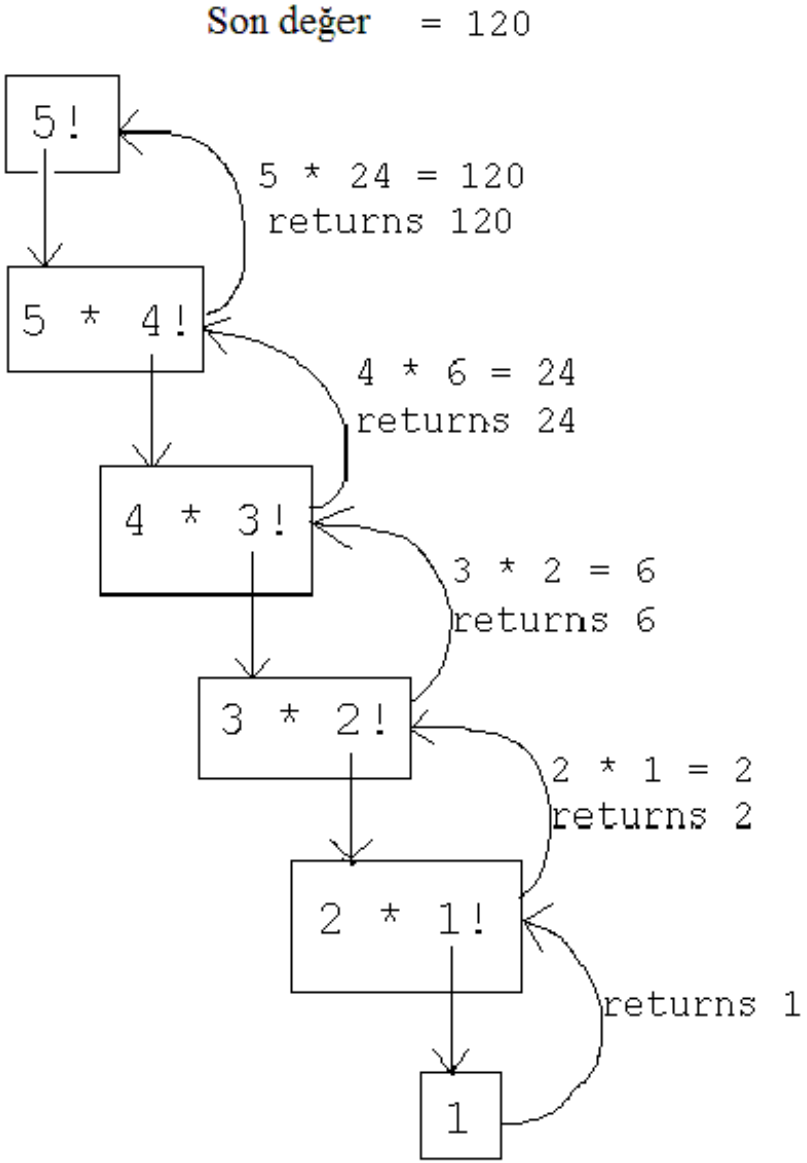
int main()
{
    int sayi;
    printf("Faktoryel hesabi icin sayi giriniz:");
    scanf("%d", &sayi);

    printf("Faktoryel sonucu: %d \n", faktoryel_hesapla(sayi));
    return 0;
}

int faktoryel_hesapla(int sayi)
{
    if (sayi <= 1)
        return 1;
    else
        return (sayi * faktoryel_hesapla(sayi-1)); //yineleme kismi
}
```

```
Faktoryel hesabi icin sayi giriniz:5
Faktoryel sonucu: 120
```

Özyinelemeli (Recursive) Fonksiyonlar



Özyineleme vs Tekrar

- Tekrar ve yinelemenin ikisi de döngü içerir.
- Tekrar özellikle döngü yapısını kullanırken, yineleme döngüyü fonksiyon çağrılarının tekrarında kullanır.
- Tekrar ve yinelemenin ikisi de bir sonlandırma testi içerirler.
- Yineleme temel bir durumla karşılaşıldığında, tekrar ise döngü devam koşulu yanlış hale geldiğinde sona erer.
- Yineleme bir çok negatif özelliğe sahiptir.
- Yineleme, mekanizmayı sürekli çağırarak fonksiyon çağrılarının artmasına sebep olur. Bu, işlemci için fazladan yük demektir.

Özyineleme vs Tekrar

- Ayrıca her yineleme çağrısı, fonksiyonun başka bir kopyasının oluşmasına sebep olur, bu da hafızayı fazladan işgal etmek demektir.
- Tekrarlı döngüler sayesinde, fonksiyonların sürekli olarak çağrılması ve fazladan hafıza kullanılması engellenir.
- Yinelemeli olarak çözülen her problem, tekrarlı bir biçimde çözülebilir. Yineleme yaklaşımı genelde problemi daha iyi yansıttığı ve daha kolay anlaşılan ve hataları kolay ayıklanan programlar yazılmasını sağlattığı için tekrar yaklaşımına göre bazen tercih edilebilir.
- Yinelemeli çözümleri seçmenin başka bir sebebi de tekrarlı çözümün kolaylıkla bulunamayışıdır.

Fonksiyon Kullanım Hataları

- Fonksiyon tanımlamalarında geri dönüş değerini unutmak.
- Geri dönüş tipi void olarak bildirilmiş bir fonksiyonun bir değer geri döndürmesi bir yazım hatasıdır.
- Aynı tipte fonksiyon parametrelerini ***double*** *x*, ***double*** *y* yerine ***double*** *x*, *y* olarak bildirmek. double x, y biçiminde parametre bildirmek, ***y parametresinin tipinin int olmasına*** sebep olur. Çünkü belirtilmeyen parametre tipi bazı derleyicilerde otomatik olarak int tipinde varsayılır.

Fonksiyon Kullanım Hataları

- Parametre (Argüman) listesini yazdığımız parantezlerin dışına noktalı virgül koymak yazım hatasıdır.
- Bir fonksiyon parametresini daha sonradan fonksiyon içinde yerel bir değişken olarak kullanmak bir yazım hatasıdır.
- Bir fonksiyon içinde başka bir fonksiyon tanımlamak yazım hatasıdır.
- Fonksiyon bildiriminin (prototipinin) sonuna noktalı virgül koymamak bir yazım hatasıdır.

Tavsiyeler

- Birden fazla fonksiyon kullanılan programlarda, **main** fonksiyonu programın esas görevini yerine getiren fonksiyonların çağırıcısı olarak kullanılmalıdır.
- Her fonksiyon, iyi olarak tanımlanmış tek bir işi yapacak şekilde sınırlandırılmalıdır ve fonksiyon ismi, fonksiyonun görevini etkili bir biçimde açıklamalıdır. Bu, özetlemeyi ve yazılımın yeniden kullanılabilirliğini sağlatır.
- Eğer fonksiyonun görevini açıklayacak etkili bir isim bulamıyorsanız muhtemelen yazdığınız fonksiyon birden fazla görevi yerine getirmeye çalışmaktadır. Bu tarzda fonksiyonları daha küçük fonksiyonlara bölmek en iyi yoldur.

Tavsiyeler

- Bir fonksiyon genellikle bir sayfadan daha uzun olmamalıdır. Küçük fonksiyonlar yazılımın yeniden kullanılabilmesini sağlatır.
- Programlar, küçük fonksiyonların bir araya getirilmesiyle yazılmalıdır. Bu, programların daha kolay yazılması, değiştirilmesi ve hatalarının giderilmesini sağlar.
- Çok fazla sayıda parametreye ihtiyaç duyan fonksiyonlar birden fazla görevi yerine getiriyor olabilir. Böyle fonksiyonları ayrı görevleri gerçekleştiren daha küçük fonksiyonlara bölmek gerekir.
- Fonksiyonun başlığı mümkünse bir satıra sığmalıdır.

Ödev

- Hazırlayacağınız program kullanıcıdan 5 öğrencinin bir dersten aldıkları vize ve final notlarını isteyecektir. Bu notlara göre yıl sonu notunu hesaplayıp harf karşılıklarını ekrana bastıracaksınız.
- 2 farklı yöntem kullanacaksınız:
  - 1. Yöntem: Kullanıcıdan istediğiniz vize ve final notları girildiğinde bunları harfe dönüştüren bir fonksiyona gönderip ekrana harf karşılıklarını basacaksınız daha sonra kullanıcıdan diğer öğrencinin notunu girmesini isteyeceksiniz.
  - 2. yöntem: Kullanıcıdan tüm öğrencilerin vize notlarını peş peşe girmesini isteyecek ve bunları bir dizide saklayacaksınız. Final için de aynı işlemi uygulayacaksınız. Sonra bu iki dizideki değerleri harfe dönüştürme fonksiyonuna gönderip her öğrencinin harf notunu peş peşe ekrana bastıracaksınız

Kaynaklar

- Programlama Sanatı, Algoritmalar, C Dili Uyarlaması, Dr. Rifat ÇÖLKESEN, Papatya Yayıncılık
- Her Yönüyle C, Tevfik KIZILÖREN, Kodlab
- C Programlama Dili, Dr. Rifat ÇÖLKESEN, Papatya Yayıncılık
- Celal Bayar Üniversitesi, Hasan Ferdi Turgutlu Teknoloji Fakültesi, YZM1105 Ders Notu
- https://www.bilgigunlugum.net/prog/cprog/c_fonksiyon (Erişim Tarihi: 13.03.2020)

BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 10 – İşaretçiler

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

25/05/2021

Konya

C dili bir değişkenin değerine erişmek için iki farklı yöntem sağlar:

- **Direkt erişim:** Şimdiye kadar yazdığımız kodlardaki gibi, bir değişkenin adını kullanarak o değişkenin değerine erişmeye denir.

- **Dolaylı erişim:** İşaretçiler aracılığıyla değişkenin bellekteki adresi üzerinden ilgili değere erişime denir.

İşaretçiler

- Bellekteki her byte'ın hexadecimal sistemde ifade edilen bir adresi vardır. **İşaretçi**, bellek alanındaki **adresler**in saklandığı bir değişkendir. İşaret değişkenlerine referans değişken de denir.

- İşaretçilere veriler değil, verilerin bellekte saklı olduğu gözlerin başlangıç adresi atanır.

- C dilinde özellikle programın çalıştırıldığı bilgisayarın iç belleğinin verimli bir şekilde kullanılmasını sağlayan **dinamik bellek** uygulamalarında ve katarlarla (string) çalışırken etkin bir şekilde kullanılmaktadır.

Örnek Verirsek;

`okul_no = 453` değişkeni için;

```
Tipi     = int
Adi      = okul_no
Değeri   = 453
Adresi   = 1005
```

1-A dersliğine ait 1005 adresindeki sıra yeri sabittir. Derse değişik öğrencilerin gelmesi durumunda değişen sadece öğrenci numaralarıdır.

1-A DERSLİĞİ

1001, 123	1011, 789	1021, 823
1002, 752	1012, 111	1022, 901
1003, 696	1013, 222	1023, 903
1004, 678	1014, 333	1024, 907
1005, 453	1015, bos	1025, boş
1006, 287	1016, 899	1026, boş
1007, 900	1017, 890	1027, 278
1008, 876	1018, bos	1028, boş
1009, boş	1019, boş	1029, boş
1010, boş	1020, boş	1030, boş

İşaretçi (pointer) bildirimi

İşaretçi değişkeni tanımlanırken, aşağıdaki ifade kullanılır:

tip *degisken_adi;

İşaretçi değişkeni, ilgili adrese hangi tip değişken saklanacak olursa olsun, bellekte aynı uzunlukta yer işgal eder. Ne kadar yer işgal edeceği (32 bit, 64 bit gibi), derleyicinin bellek modeline ya da kullanılan mikroişlemcinin adres uzunluğuna bağlı olabilir.

int *p; → Tamsayı saklamak için bir adres
float *q; → Gerçel sayı saklamak için bir adres
char *r; → Karakter saklamak için bir adres

- İşaretçiye bir değişkenin **adresini** atamak için **&** operatörü kullanılır:

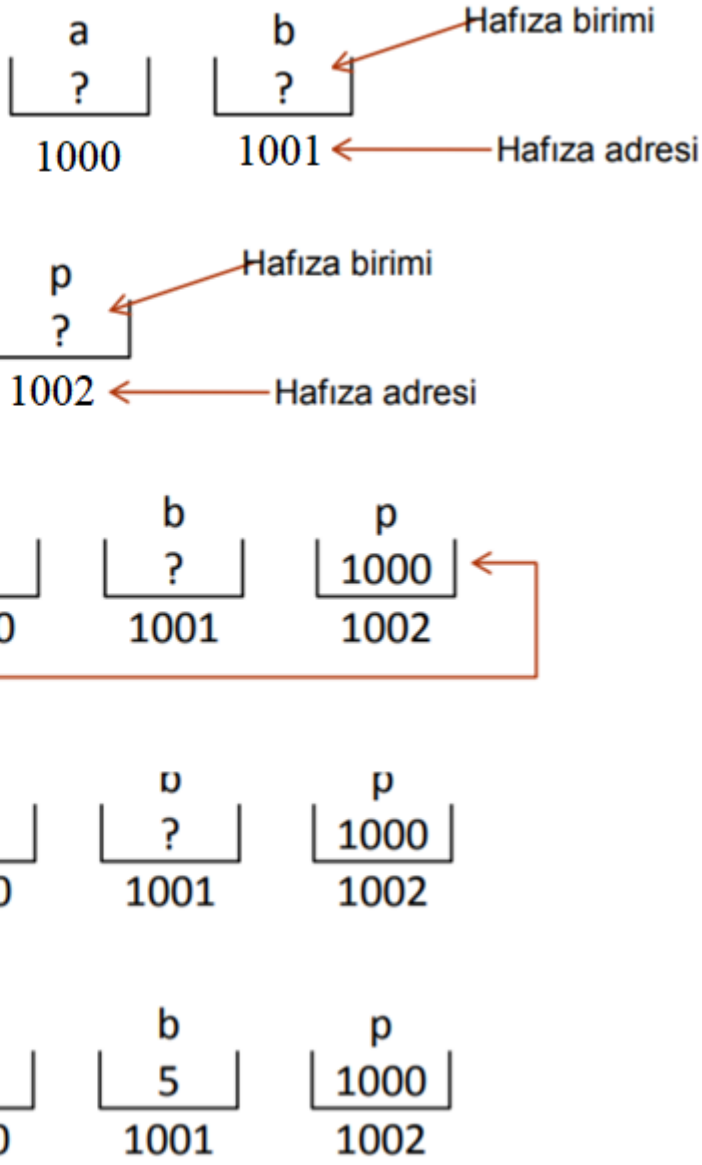
```
int *p;
float *q;
char *r;
```

p = &t ; q = &f ; r = &kr ;

- Yani **&** operatörü bir değişkenin önüne konmuşsa, o değişkenin değeri/içeriğiyle değil, adresiyle ilgileniliyor anlamına gelir. Öte yandan bir işaretçi adının önüne * operatörü konmuşsa; işaretçinin tuttuğu adresle değil, işaret ettiği yerdeki veri ile ilgileniliyor demektir.

* → İşaretçi bildirimi
→ Değere erişim operatörü

```
int a, b;
int *p;
p = &a;
*p = 5;
b = *p;
```



Bazı doğru / yanlış tanımlamalar

```
int c, *pc;

/* pc' nin işaret ettiği adresteki değer c olsun istiyorsak: */

pc = c;      // Yanlış! pc adres, ancak c bir adres değil.

*pc = c;   // Doğru! *pc, pc adresindeki değerdir ve c de bir
              değerdir (adres değil).

/* pc, c' nin adresini işaret etsin istiyorsak: */

*pc = &c; // Yanlış! *pc, pc adresindeki değer, ancak &c bir adres.

pc = &c;  // Doğru! pc bir adres ve &c de bir adres.
```

Aynı işlevde

C dilinde oluşturduğumuz her tip hafızada belirli byte boyutunda yer kaplar.

- **32** bit işlemcili bilgisayarlarda;

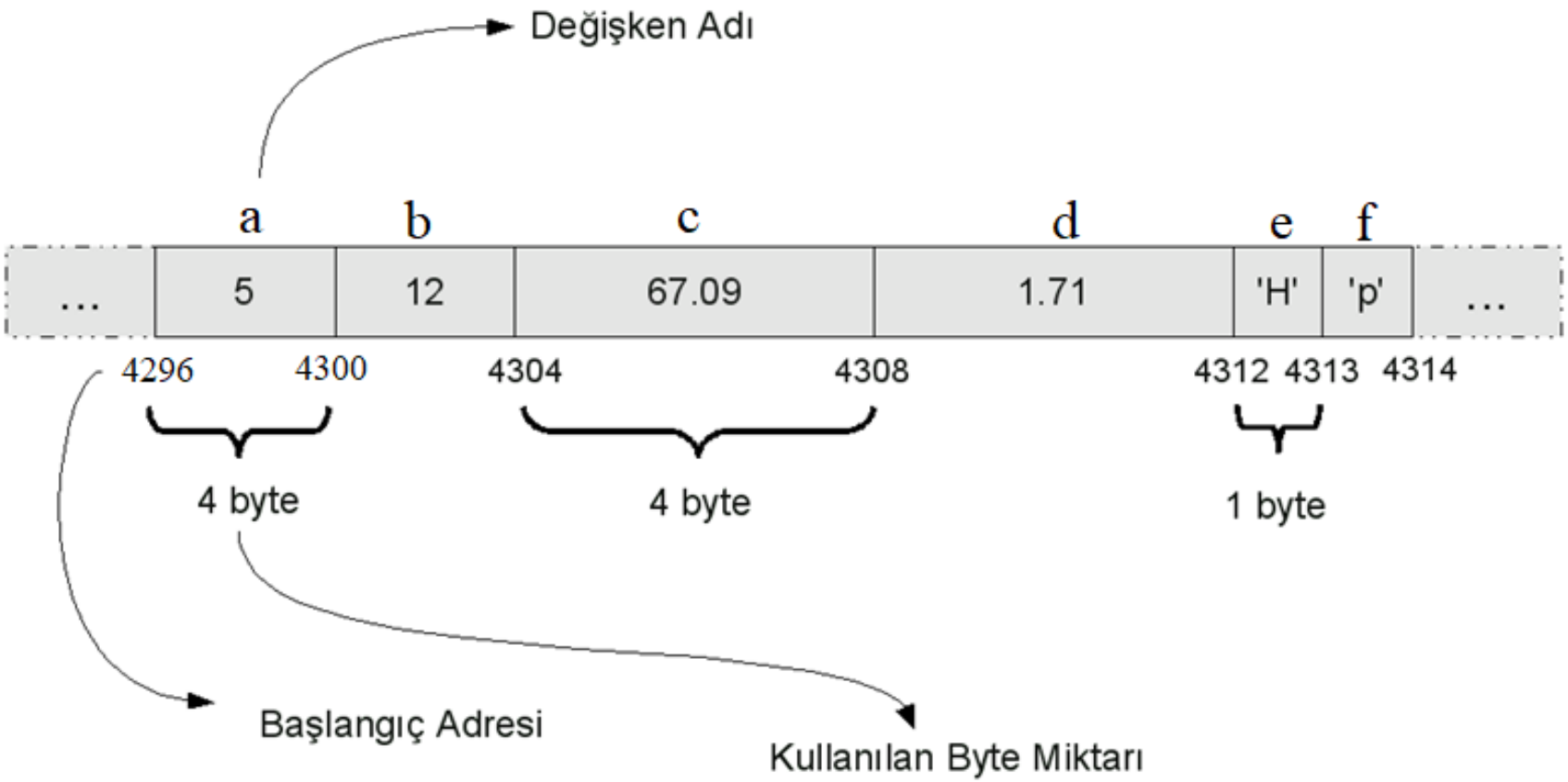
```
char   = 1 byte
int    = 4 byte
float  = 4 byte
double = 8 byte
```

- **64** bit sistemlerde ise, değerler **iki katı** olarak değişir.

Pointerlar ise tipe bakmaksızın her zaman;

- **32** bit sistemlerde **4 byte**,
- **64** bit sistemlerde **8 byte** yer kaplar.

```
int a = 5;
int b = 12;
float c = 67.09;
float d = 1.71;
char e = 'H';
char f = 'p';
```



```
#include<stdio.h>
main() {
        int deg, *isaret;
        deg = 999;
        isaret = &deg ;
printf("degiskenin adresi: %p\n", isaret);
printf("degiskenin degeri: %d\n", *isaret);
printf("degiskenin adresi: %p\n", &deg);
printf("degiskenin degeri: %d\n", deg);
        return 0;  }
```

Adres	Bellek
FAFF	…
FB00	
FB01	999
FB02	
FB03	…

```
degiskenin adresi: 000000000062FE14
degiskenin degeri: 999
degiskenin adresi: 000000000062FE14
degiskenin degeri: 999

--------------------------------
Process exited after 0.04861 seconds with return value 0
Press any key to continue . . .
```

```c
#include<stdio.h>
main() {
    int sayi = 15;
    printf("sayi degiskeninin degeri: %d\n", sayi);
    printf("sayi degiskeninin bellek adresi: %p\n", &sayi);

    int *p = NULL;
    printf("isaretci degiskeninin tanimlama esansinda degeri: %p\n", p);

    p = &sayi;
    printf("isaretci degiskeninin yeni degeri: %p\n", p);
    printf("isaretci degiskeninin tuttugu bellek adresindeki deger: %d\n", *p);

    *p = 16;
    printf("isaretci degiskeninin tuttugu bellek adresindeki yeni deger: %d\n", *p);
    printf("sayi degiskeninin yeni degeri: %d\n", sayi);

    return 0;  }
```

```
sayi degiskeninin degeri: 15
sayi degiskeninin bellek adresi: 000000000062FE14
isaretci degiskeninin tanimlama esansinda degeri: 0000000000000000
isaretci degiskeninin yeni degeri: 000000000062FE14
isaretci degiskeninin tuttugu bellek adresindeki deger: 15
isaretci degiskeninin tuttugu bellek adresindeki yeni deger: 16
sayi degiskeninin yeni degeri: 16

--------------------------------
Process exited after 0.04044 seconds with return value 0
Press any key to continue . . .
```

```c
#include<stdio.h>
main() {
    int sayi = 100;
    int *ref;
    ref = &sayi ;
    int sayi2 = *ref;
    *ref = 150;
printf("sayi degiskeninin degeri: %d\n", sayi);
printf("sayi2 degiskeninin degeri: %d\n", sayi2);
    return 0;  }
```

```
sayi degiskeninin degeri: 150
sayi2 degiskeninin degeri: 100

--------------------------------
Process exited after 0.03967 seconds with return value 0
Press any key to continue . . .
```

İşaretçi aritmetiği

- İşaretçiler kullanılırken, işaret edilen adres temel alınıp o adresten önceki ya da sonraki adreslere erişilmesi istendiğinde aritmetik işlemlerden yararlanılabilir.

- İşaretçiler üzerinde yalnız bir tam sayı ile **toplama**, bir tam sayı **çıkarma**, **artırma** ve **azaltma** operatörleri kullanılabilir (+, ++, -, --).

- İşaretçilerin bu operatörlere verdiği tepki, her bir veri tipi için farklıdır. Örneğin 32 bit işlemcili bilgisayarlarda int * tipindeki bir işaretçiye ++ operatörü uygulanırsa adres değeri *4 byte*, char * tipine uygulanırsa *1 byte* artacaktır.

```c
#include<stdio.h>
main() {
    int tsayi = 17;
    double osayi = 21.3;
    char kr = 'C';

    int *intPtr;
    double *doublePtr;
    char *charPtr;

    intPtr = &tsayi ;
    doublePtr = &osayi;
    charPtr = &kr;

printf("Tam sayi isaretcisinin ilk degeri: %p\n", intPtr);
printf("Ondalik sayi isaretcisinin ilk degeri: %p\n", doublePtr);
printf("Karakter isaretcisinin ilk degeri: %p\n", charPtr);
printf("------------------------------\n");

    intPtr ++;
    doublePtr --;
    charPtr ++;

printf("Tam sayi isaretcisinin son degeri: %p\n", intPtr);
printf("Ondalik sayi isaretcisinin son degeri: %p\n", doublePtr);
printf("Karakter isaretcisinin son degeri: %p\n", charPtr);
printf("------------------------------\n");

    return 0;  }
```

```
Tam sayi isaretcisinin ilk degeri: 000000000062FE04
Ondalik sayi isaretcisinin ilk degeri: 000000000062FDF8
Karakter isaretcisinin ilk degeri: 000000000062FDF7
------------------------------
Tam sayi isaretcisinin son degeri: 000000000062FE08
Ondalik sayi isaretcisinin son degeri: 000000000062FDF0
Karakter isaretcisinin son degeri: 000000000062FDF8
------------------------------
```

- *ip++; /* İşaretçinin gösterdiği bellek adresini artırır. */

(*ip)++; /*İşaretçinin gösterdiği adresteki değişken değerini artırır. */

- İki işaretçi direkt toplanamaz ve çarpılamaz. Mantıksal olarak iki işaretçinin toplanması ya da çarpılmasının bir anlamı yoktur.

- İşaretçi değişkenler üzerinde karşılaştırma operatörleri

(<, >, <=, >=, ==, !=) kullanılabilir.

  - İki işaretçi değeri birbirine eşitse, aynı bellek alanındaki değere işaret ediyorlar demektir.
  - Bir işaretçinin değeri bir diğer işaretçinin değerinden küçükse, bellekte daha önce gelen bir alandaki değeri işaret ediyor demektir. Dizilerde önceki/sonraki elemanın belirlenmesi, sınır aşımı kontrolü gibi uygulamalarda kullanılabilir.

İşaretçiler & Diziler

- İşaretçilerin kullanım amaçlarından biri de dizilerin daha kolay ve verimli şekilde kullanılmasını sağlamaktır.
- Dizideki elemanların tipleriyle uyumlu bir işaretçiye dizinin ilk elemanının adresi kopyalandığında, bu işaretçi aracılığıyla dizinin tüm elemanlarına erişilebilir.
- Dizinin adı, o dizinin bellekteki başlangıç adresine karşılık gelir.
- Dizinin bellekteki başlangıç adresi, dizinin ilk elemanının bellek adresidir.

```c
#include<stdio.h>
main() {
    int dizi[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int *ptr = &dizi[0];
    int *ptr2 = dizi;
    int i;

printf("Dizinin 1. elemani: %d\n", dizi[0]);
printf("Dizinin 1. elemani: %d\n", *ptr);
printf("Dizinin 1. elemani: %d\n", *ptr2);
printf("------------------------------\n");
printf("Dizinin 5. elemani: %d\n", dizi[4]);
printf("Dizinin 5. elemani: %d\n", *(ptr+4));
printf("Dizinin 5. elemani: %d\n", *(ptr2+4));
printf("------------------------------\n");
printf("Dizinin 9. elemani: %d\n", dizi[8]);
printf("Dizinin 9. elemani: %d\n", *(ptr+8));
printf("Dizinin 9. elemani: %d\n", *(ptr2+8));

    for(i=0; i<10; i++) {
printf("Dizi[%d] -> Adresi: %p, Degeri: %d\n", i, ptr+i, *(ptr+i));   }
    return 0;  }
```

```
Dizinin 1. elemani: 1
Dizinin 1. elemani: 1
Dizinin 1. elemani: 1
------------------------------
Dizinin 5. elemani: 5
Dizinin 5. elemani: 5
Dizinin 5. elemani: 5
------------------------------
Dizinin 9. elemani: 9
Dizinin 9. elemani: 9
Dizinin 9. elemani: 9
Dizi[0] -> Adresi: 000000000062FDE0, Degeri: 1
Dizi[1] -> Adresi: 000000000062FDE4, Degeri: 2
Dizi[2] -> Adresi: 000000000062FDE8, Degeri: 3
Dizi[3] -> Adresi: 000000000062FDEC, Degeri: 4
Dizi[4] -> Adresi: 000000000062FDF0, Degeri: 5
Dizi[5] -> Adresi: 000000000062FDF4, Degeri: 6
Dizi[6] -> Adresi: 000000000062FDF8, Degeri: 7
Dizi[7] -> Adresi: 000000000062FDFC, Degeri: 8
Dizi[8] -> Adresi: 000000000062FE00, Degeri: 9
Dizi[9] -> Adresi: 000000000062FE04, Degeri: 10
```

Adresler arasında 4’ er fark olduğuna dikkat !!!

Bir A[] dizisinin elemanlarının toplamını bulan bir fonksiyon yazmamız gerekirse;

```
int topla1(int A[], int n)
{
    int *p, toplam=0, sayac;
    p=&A[0];

    for(sayac=0; sayac<n; sayac++)
        toplam+=*(p+sayac);

    return toplam;
}
```

```
int topla2(int A[], int n)
{
    int *p, toplam=0;

    for(p=A; p<&A[n]; p++)
        toplam+=*p;

    return toplam;
}
```

- Matrisler, bilindiği gibi iki boyutlu dizilerdir. Matrisler üzerinde işaretçi atamaları yapmak da mümkündür:

```
int A[10][20], *p, *q, *r;

p = A;
p = &A[0][0];

q = &A[3][4];
r = &A[9][19];
```

Aynı adres ?

A[i][j]; → *(p+(i*20+j)) → Aynı eleman ?

Dinamik bellek kullanımı nedir ?

Dizileri önceki haftalarda bahsettiğimiz klasik yöntemlerle tanımlarsak, derleyici bu dizi için gerekli bellek alanını bildirildiği fonksiyon etkin olduğu sürece tutacak/meşgul edecek ve bu adresteki değer kullanılmasa bile belleğin bu kısmını kullanılamaz hale getirecektir.

Eğer bu alan seyrek kullanılıyorsa ya da kullanılmadığı zamanlarda saklanan değerin bir önemi yoksa; bu alan kullanılmadığı zamanlarda serbest bırakılarak başka amaçlarla kullanılabilir. Bunun için belleğin dinamik kullanımı gereklidir.

Dinamik bellek kullanımı; gerekli bellek alanlarının program ya da fonksiyon başında ayrılması yerine, gerektiği anda istenmesi ve kullanılmadığında serbest bırakılması/iade edilmesidir. Boş bellek alanı istemek ya da bir alanı serbest bırakmak için standart kütüphanede bazı fonksiyonlar mevcuttur.

Dinamik belleğin AVANTAJları

- C dilinin bu kadar yaygın olmasının önemli sebeplerinden biri, işletim sistemi kaynaklarına en alt (detay) seviyede erişim imkanı sağlamasıdır. Bunda işaretçilerin payı büyük olup, kullanmayı bilene oldukça **esnek** imkanlar sunar.

- Bellek **daha verimli** bir şekilde kullanılır ve **daha hızlı** programlar yazılır.

- Dizilerin yönetiminde kullanılması **performans**ı artırır.

- Fonksiyonların birden fazla değer döndürmesine olanak verir.

- Klasik yöntemle diziler tanımlanırken boyutları baştan tanımlanır ve değiştirilemezken, dinamik bellek kullanımıyla ihtiyaç halinde fazladan alan sağlanabilir.

Dinamik belleğin DEZAVANTAJları

- Bellek adreslerine erişim işaretçi değişkenlerinin kullanımını gerektirir. Bu, nispeten öğrenmesi zor bir yöntem olduğundan, programlamaya yeni başlayanlar için işleri karmaşık bir hale getirebilir.

- Bellek adreslerini kullanıcı erişimine açmak, olası programlama hatalarının ihtimalini artırır.

- Belleğin tüm hücrelerine erişim imkanı sağladığı için işaretçilerin yanlış kullanımı, sadece yazılan programın değil, tüm sistemin çökmesine dahi yol açabilir.

Dinamik Diziler

Diziler, programın başında kaç boyutlu olduğu verilerek bildirilirse, derleyici o dizi için gerekli bellek alanını tanımlı dizi için saklar ve dizi üzerindeki işlemler tamamlanmış ve artık bu bellek alanına dizi için ihtiyaç kalmamış olsa bile program içinde başka amaçla kullanmaz.

Ancak dinamik bellek kullanımı söz konusu ise, program yürütülürken dizi için gerekli bellek alanı talep edilir ve işi bittiğinde iade edilebilir.

Bu amaçla standart kütüphanede;

calloc(), ***malloc()***, ***realloc()*** ve ***free()***

fonksiyonları tanımlanmıştır.

```c
#include<stdio.h>

main()  {

    int *p, i, j, enkucuk, enbuyuk;
    p = (int *) malloc(20*sizeof(int));
        for(i=0; i<20; i++) {
        *(p+i) = rand();
        printf("%d\n", *(p+i)); }

        enkucuk = *p;
        for(j=0; j<20; j++)
            if(*(p+j)<enkucuk)
                enkucuk = *(p+j);
        printf("\n En kucuk eleman: %d' dir.\n", enkucuk);

        enbuyuk = *p;
        for(j=0; j<20; j++)
            if(*(p+j)>enbuyuk)
                enbuyuk = *(p+j);
        printf("\n En buyuk eleman: %d' dir.\n", enbuyuk);
    free(p);

    return 0;
}
```

Bellekten istenen alanı ayırır ve o alanın başlangıç adresini gönderir.

Bellekten daha önce istenen alanı boşaltıp iade eder.

```
41
18467
6334
26500
19169
15724
11478
29358
26962
24464
5705
28145
23281
16827
9961
491
2995
11942
4827
5436

 En kucuk eleman: 41' dir.

 En buyuk eleman: 29358' dir.
```

İşaretçi Tipli Fonksiyonlar

Fonksiyonlar tanımlanırken işaretçi tipli değişkenleri olabileceği gibi, fonksiyonun kendi tipi de işaretçi olabilir.

Yani fonksiyon, çağırıldığı yere bir değer yerine adres de gönderebilir.

Standart kütüphanede katarlar üzerinde işlem yapmayı sağlayan işaretçi tipli fonksiyonlar da vardır ve bu fonksiyonlar uygulamaların etkinliğini artırır.

Katarlarla ilgili işaretçi uygulamalarına bir sonraki dersimizde yer verilecektir.

```c
#include<stdio.h>

main() {
    int dizi[] = {11, 15, 37, 2, 43, -2, 7};
    int *q;
    q = EnKucugunAdresi(dizi, 7);
    printf("En kucuk eleman: %d\n", *q);
    printf("En kucuk elemanin adresi: %p\n", q);
    return 0;   }

    int EnKucugunAdresi(int A[], int n) {
        int EnKucuk, *p, i;
        EnKucuk = A[0];
        p = &A[0];

        for(i=1; i<n; i++)
            if (A[i] < EnKucuk) {
                EnKucuk = A[i];
                p = &A[i];}
        return p;
    }
```

```
En kucuk eleman: -2
En kucuk elemanin adresi: 000000000062FE04
```

ÖRNEK: İşaretçi kullanan ortalama değer bulma fonksiyonu

```c
#include<stdio.h>
#define NOTSAY 10

NotOku(int *, int);
double OrtBul(int *, int);

main() {
    int notlar[NOTSAY];
    int *p = notlar; /* int *p = &notlar[0]
                        olarak da yazılabilirdi. */
    NotOku(p, NOTSAY);
    double Ort = OrtBul(p, NOTSAY);

    printf("\n Girilen sayilarin ortalamasi: %.2f\n", Ort);

    return 0; }

NotOku(int *p, int sayi) {
    int i;
        for(i=0; i<sayi; i++) {
            printf("\n%2d. sayiyi giriniz:", i+1);
            scanf("%d", p+i);
        }
}
```

```c
double OrtBul(int *p, int sayi) {
    int i;
    double toplam = 0;

    for(i=0; i<sayi; i++)
        toplam+= *(p+i);
    return (double) toplam / (double) sayi;
}
```

ÖRNEK: İşaretçi tipli faktöriyel hesaplama fonksiyonu

```c
#include<stdio.h>

int Faktoriyel(int);

main() {
    int sayi = 0;

    printf("\n Faktoriyel Hesaplama Fonk. Adresi: %p\n", Faktoriyel);
    printf("\n Faktoriyeli Hesaplanacak Sayi:");
    scanf("%d", &sayi);

int faktor = Faktoriyel(sayi);
printf("\n Sonuc: %d", faktor);

}

    int Faktoriyel(int sayi) {
        int sonuc = 1, i;

        if( sayi<=0 )
        return sonuc;

        for(i=sayi; i>=1; i--) {
            sonuc = sonuc*i; }
        return sonuc;
    }
```

ÖDEV:

Verilen ek pdf dosyasında bir sayının karesinin hesaplandığı Program 3.7‘ye benzer şekilde işaretçilerden yararlanarak, kullanıcı tarafından girilen bir sayının %2,5’unu hesaplayan ve sonucu ekranda gösteren bir program yazınız. Yazacağınız program, tek bir hesaplama yapıp sonlanmamalı, kullanıcının istediği kadar tekrarda hesaplama yapabilmesine olanak sağlamalıdır.

KAYNAKLAR

- Her Yönüyle C, Tevfik KIZILÖREN, Kodlab
- İşte C Programlama Dili, Dr. Rifat ÇÖLKESEN, Papatya Yayıncılık
- http://www.baskent.edu.tr/~tkaracay/etudio/ders/prg/c/pointers.pdf
- http://www.alikeskin.org/algoritma/Pointer.pdf

BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 11 – Sözce(String)

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

1/06/2021

Konya

Sözce (String)

- Bazı Kaynaklarda Karakter Dizileri ya da Katar isimleriyle de tanımlanmaktadır.
- Sözcelerin normal dizilerden farkı yoktur. Sadece elemanlar char tipinde saklanır.
- Bazı programlama dillerinde sözceleri tutmak için özel veri türleri (string, vb.) bulunmaktadır (Java, C# vs…).
- Ancak C programlama dilinde böyle bir veri türü olmadığı için karakterlerden oluşan bir boyutlu diziler kullanılır.
- Karakter dizilerine özel olarak, karakter dizilerinin sonuna sonlandırıcı karakter olarak adlandırılan bir simge eklenir.
- Sonlandırıcı karakter:
  - Dizinin bittiği yeri gösterir.
  - ASCII tablosunun sıfır numaralı ('\0') karakteridir.

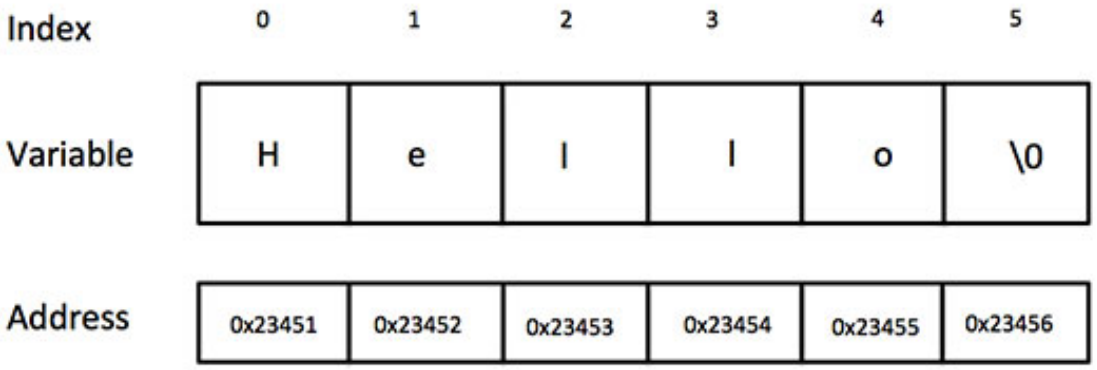
Sözce (String)

- Karakter dizilerine 2 şekilde başlangıç değeri verilebilir:

  (1) char selam[6]="Hello";

  (2) char selam[6]={'H', 'e', 'l', 'l', 'o', '\0'};

- İkinci tanımlamada sonlandırıcı karakter (NULL) programcı tarafından konmalıdır.
- Birinci tanımlamada ise buna gerek yoktur. Çünkü, sonlandırıcı karakter bu atamayla, derleyici tarafından eklenir.



Sözce (String)

- Örnek 1

```c
#include<stdio.h>
main() {
    char selam [6]="Hello";
    printf("Merhaba Mesaji: %s\n", selam );
    return 0;  }
```

```
Merhaba Mesaji: Hello

--------------------------------
Process exited after 0.00986
Press any key to continue .
```

```c
#include<stdio.h>
main() {
    char selam[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
    printf("Merhaba Mesaji 2: %s\n", selam );
    return 0;  }
```

```
Merhaba Mesaji 2: Hello

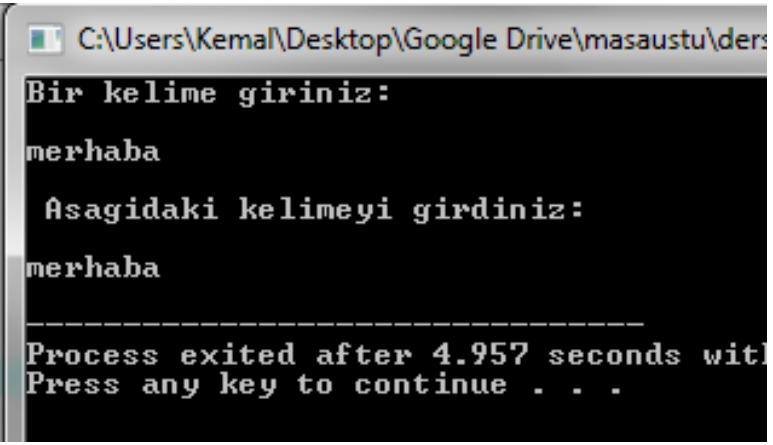
--------------------------------
Process exited after 0.01056
Press any key to continue .
```

puts() ve gets() Fonksiyonları

- Bir karakter dizisini klavyeden okumak için C'nin standart gets() fonksiyonu kullanılır.
- puts() fonksiyonu da kendisine argüman olarak verilen karakter dizisini ekrana yazdırır.
- Bu fonksiyonlar herhangi bir indeks tanımlamadan karakter dizilerinin okunmasını sağlar.
- Okuduğu karakter dizisinin sonuna satır sonu işaretini değil, NULL değerini yerleştirir.
- Örnek 2:

ders11.2.c

```
1   #include<stdio.h>
2   main() {
3       char kelime[10];
4       puts("Bir kelime giriniz:\n");
5       gets(kelime);
6       puts("\n Asagidaki kelimeyi girdiniz:\n");
7       puts(kelime);
8       return 0;
9       }
10
```

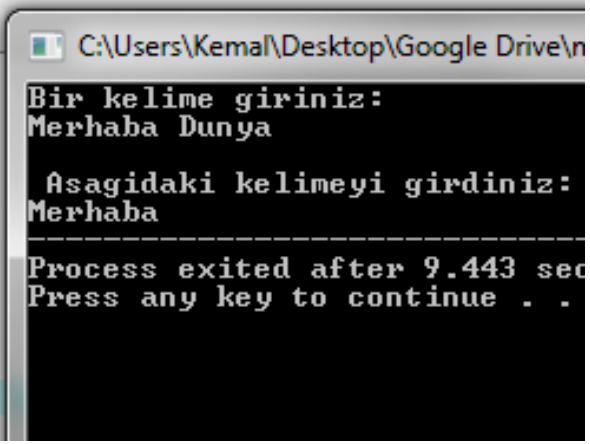


Sözce (String)

- Örnek 3:
- scanf sadece tek kelimeyi alır. printf sözcenin tamamını yazar
- puts yazdırmayı bitirdikten sonra bir alt satıra geçer (\n'e gerek yok)

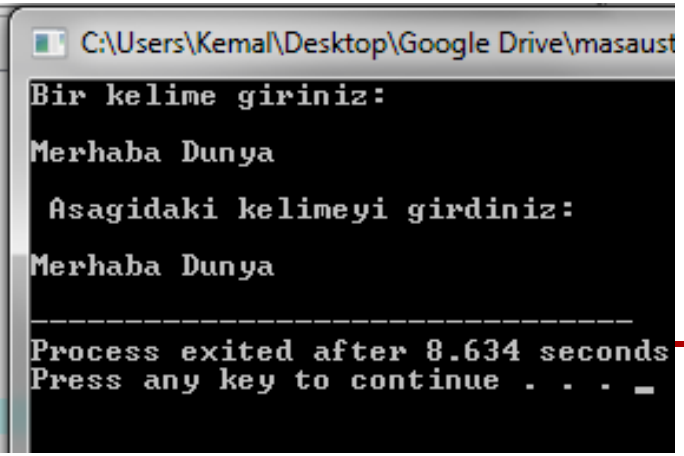
ders11.3.c

```
1   #include<stdio.h>
2   main() {
3       char kelime[20];
4       printf("Bir kelime giriniz:\n");
5       scanf("%s", kelime);
6       printf("\n Asagidaki kelimeyi girdiniz:\n");
7       printf("%s", kelime);
8       return 0;
9       }
10
```



ders11.2.c

```
1   #include<stdio.h>
2   main() {
3       char kelime[20];
4       puts("Bir kelime giriniz:\n");
5       gets(kelime);
6       puts("\n Asagidaki kelimeyi girdiniz:\n");
7       puts(kelime);
8       return 0;
9       }
10
```



Sözce (String) Özel Fonksiyonları

- strlen() fonksiyonu: Sözcenin uzunluğunu döndürür.
- Dönüş tipi unsigned long (işaretsiz tamsayı)'dır.
- Sözce sonunda bulunan NULL karakterini hesaba katmaz.
- Örneğin, karakter dizisi "abc" değerlerini içeriyorsa, strlen() fonksiyonu bu uzunluk olarak "3" değerini döndürür.
- Bu tip özel fonksiyonları kullanmak için <string.h> kütüphanesi programa eklenmelidir.

strlen() Örnek

- Örnek 4:

ders11.4.c

```
1   #include<stdio.h>
2   #include<string.h>
3   main() {
4       char dizi[50];
5       int dizi_uzunluk,i=0;
6       printf("Bir metin giriniz:\n");
7       gets(dizi);
8       printf("\n Tek tek karakterler yazdiriliyor:\n");
9       dizi_uzunluk=strlen(dizi);
10      for(i=0;i<dizi_uzunluk;i++)
11          {printf("%c\n",dizi[i]);
12          }
13      printf("Tum Karakterler Pespese Yazdiriliyor:\n");
14      printf("%s", dizi);
15      return 0;
16      }
17
```

C:\Users\Kemal\Desktop\Google Drive\masaustu\ders icin ornek

```
Bir metin giriniz:
Merhaba Dunyali

 Tek tek karakterler yazdiriliyor:
M
e
r
h
a
b
a

D
u
n
y
a
l
i
Tum Karakterler Pespese Yazdiriliyor:
Merhaba Dunyali
--------------------------------
Process exited after 9.078 seconds with return
```

strcpy() fonksiyonu

- strcpy() fonksiyonu: Bir sözceyi başka bir sözceye kopyalamaya yarar.
- Örnek 5:

```c
#include<stdio.h>
#include<string.h>
main() {
    char dizi1[50]="Merhaba";
    char dizi2[50];

    strcpy(dizi2, dizi1);

    printf("dizi1: %s\n",dizi1);
    printf("dizi2: %s\n",dizi2);

    return 0;
    }
```

```
dizi1: Merhaba
dizi2: Merhaba

--------------------------------
Process exited after 0.2757 seconds with return val
Press any key to continue . . .
```

strcat() fonksiyonu

- strcat() fonksiyonu: Bir sözceyi başka bir sözcenin sonuna ekleyip bunları birleştirmeye yarar.
- Örnek 6:

ders11.6.c

```
1   #include<stdio.h>
2   #include<string.h>
3   main() {
4       char dizi1[50]="Merhaba";
5       char dizi2[50]="Dunya";
6
7       strcat(dizi1, dizi2);
8
9       printf("\n%s",dizi1);
10
11      return 0;
12      }
13
```

```
C:\Users\Kemal\Desktop\Google Drive\masaustu\ders icin c

MerhabaDunya
--------------------------------
Process exited after 0.05682 seconds with
Press any key to continue . . .
```

strcmp() fonksiyonu

- strcmp() fonksiyonu: İki sözceyi alfabetik olarak karşılaştırmaya yarar.
- Eğer kendisine gelen ilk argüman alfabetik olarak 2. argümandan önce geliyorsa, 0'dan küçük bir değer döndürür.
- Eğer ilk argüman alfabetik olarak diğerinden sonra geliyorsa 0'dan büyük bir değer döndürür.
- Eğer iki sözcedeki elemanlar eşitse, 0 değerini döndürür
- Örnek 7:

```c
#include<stdio.h>
#include<string.h>
main() {
    char cevap1[50]="Ankara";
    char cevap2[50];
    int sonuc;

    printf("\nTurkiyenin baskenti neresidir?\n");
    gets(cevap2);
    sonuc=strcmp(cevap1,cevap2);
    if (sonuc==0)
        printf("Tebrikler! Dogru Cevap...\n");
    else
        printf("Malesef Yanlis Cevap...\n");

    return 0;
    }
```

```
Turkiyenin baskenti neresidir?
Ankara
Tebrikler! Dogru Cevap...

--------------------------------
Process exited after 5.607 seconds with
Press any key to continue . . .
```

strcmp() fonksiyonu

```c
#include<stdio.h>
#include<string.h>
main() {
    char cevap1[50]="Ankara";
    char cevap2[50];
    int sonuc;

    printf("\nTurkiyenin baskenti neresidir?\n");
    gets(cevap2);
    sonuc=strcmp(cevap1,cevap2);
    if (sonuc==0)
        printf("Tebrikler! Dogru Cevap...\n");
    else
        printf("Malesef Yanlis Cevap...\n");

    printf ("%d",sonuc);
    return 0;
    }
```

```
Turkiyenin baskenti neresidir?
Konya
Malesef Yanlis Cevap...
-1
--------------------------------
Process exited after 10.9 seconds
Press any key to continue . . .
```

```c
#include<stdio.h>
#include<string.h>
main() {
    char cevap1[50]="Ankara";
    char cevap2[50];
    int sonuc;

    printf("\nTurkiyenin baskenti neresidir?\n");
    gets(cevap2);
    sonuc=strcmp(cevap1,cevap2);
    if (sonuc==0)
        printf("Tebrikler! Dogru Cevap...\n");
    else
        printf("Malesef Yanlis Cevap...\n");

    printf ("%d",sonuc);
    return 0;
    }
```

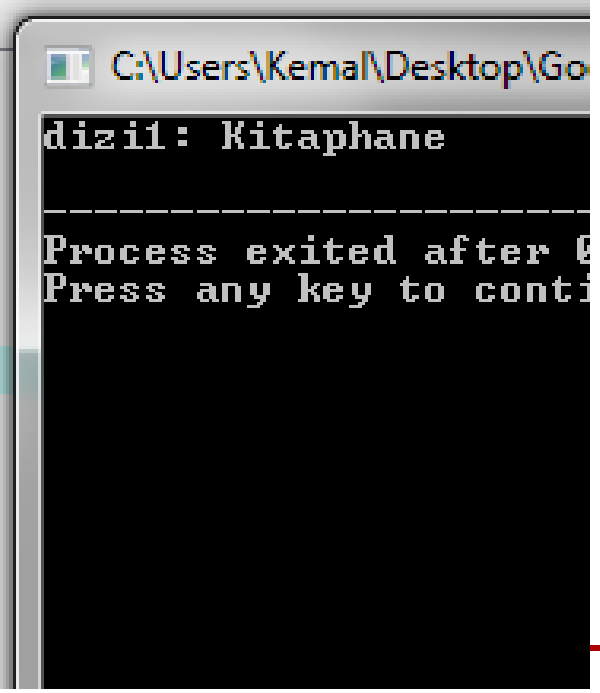
```
Turkiyenin baskenti neresidir?
Abbb
Malesef Yanlis Cevap...
1
--------------------------------
Process exited after 9.026 seco
Press any key to continue . . .
```

strncpy() fonksiyonu

- strncpy() fonksiyonu: İkinci sözcenin ilk N sayıdaki karakterini Birinci sözcenin ilk N sayıdaki karakterinin yerine kopyalamaya yarar.
- Fonksiyon içindeki argümanlar yazılırken sözce1, sözce2 ve son olarak da kaçıncı karaktere kadar kopyalamanın yapılacağı sayı girilmelidir.
- Örnek 8:

ders11.8.c

```
1   #include<stdio.h>
2   #include<string.h>
3   main() {
4       char dizi1[50]="Kutuphane";
5       char dizi2[50]="Kitapci";
6
7       strncpy(dizi1, dizi2, 5);
8
9       printf("dizi1: %s\n",dizi1);
10
11      return 0;
12      }
13
```



Bazı Özel Fonksiyonlar

- strncat() fonksiyonu:
  - Bu fonksiyon argüman olarak gelen ikinci sözcenin, argüman olarak gelen ilk N sayıdaki karakterini kopyalayıp, argüman olarak gelen birinci sözcenin sonuna ekler.

- strncmp() fonksiyonu:
  - Argüman olarak aldığı iki sözcenin tamamını karşılaştırmak yerine sadece ilk N karakterini karşılaştırır.
  - Sonuç özellikleri strcmp() fonksiyonu ile aynıdır.

Sözce Kullanımında Yapılan Bazı Hatalar

- Dizi notasyonu kullanarak tanımladığınız bir karakter dizisine sadece tanımlama esnasında değer atanabilir. Örneğin aşağıdaki gibi bir kullanım hatalıdır:

```
char kelime[20];
kelime="Ankara"; //Yanlış kullanım
```

- Önce sözce tanımlanmış ve sonraki satırda değer ataması yapılmaya çalışılmıştır.
- Bir C dizisi tanımlandığında belli bir bellek alanı ayrılır ve dizi adı bu bellek alanının başlangıç adresini tutar. Dizinin adıyla ulaşılan bellek adresi sabittir ve başka bir bellek adresiyle değiştirilemez. 2. satır yürütülürken "Ankara" değeri bellekte ayrı bir alanda oluşturulur ve bu atama komutuyla birlikte, bu değerin adresinin, sözcenin adresiyle değiştirilmesi söylenmiş olur fakat bu sözcenin adresi başta belirlendiği için değiştirilmesi mümkün değildir.
- Doğru olan:

```
char kelime[20]="Ankara"; //Doğru
```

Sözce Kullanımında Yapılan Bazı Hatalar

- ```
  char sehir[20] ="Ankara";
  char kelime[20];
  kelime=sehir;       //Yanlış kullanım
  ```

- C programlama dilinde değeri atanmış bir sözcenin değerini değiştirmek için = operatörünü kullanamazsınız.
- Bunun için strcpy fonksiyonu kullanılmalıdır.
- Doğru olan:

```
char kelime[20];
strcpy (kelime, "Ankara");              //Doğru
```
```

ÖDEV:

Bir sözcedeki boşluklar haricindeki tüm farklı karakterleri bulan ve bunları sıralı bir şekilde ekrana yazdıran bir fonksiyon hazırlayınız.

Örnek Girdi: "merhaba dunya"

Çıktı: a, b, d, e, h, m, n, r, u, y,

KAYNAKLAR

- Her Yönüyle C, Tevfik KIZILÖREN, Kodlab
- İşte C Programlama Dili, Dr. Rifat ÇÖLKESEN, Papatya Yayıncılık
- Celal Bayar Üniversitesi, Hasan Ferdi Turgutlu Teknoloji Fakültesi, YZM1105 Ders Notu

BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 12 – Matematiksel Fonksiyonlar

Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

08/06/2021

Konya

Matematiksel Fonksiyonlar

Kod yazarken program içinde matematiksel bir işlem yapma ihtiyacı doğduğunda, kullanılan programlama dilinin hazır kütüphane fonksiyonlarını öncelikli tercih etmek daha akıllıca bir yaklaşım olacaktır.

Eğer problemin doğrudan ya da dolaylı olarak çözümünde hazır fonksiyonlar yetersiz kalıyorsa, söz konusu problemi çözmek için programcı tarafından kod yazılabilir.

Matematik fonksiyonlarının türü *double* olup, kullanılabilmeleri için genellikle **math.h** başlık dosyasının program içerisine dahil edilmesi gerekir.

#include<math.h>

Matematiksel Fonksiyonlar

- math.h kütüphanesi içinde EDOM, HUGE_VAL, gibi bazı simgesel sabit bildirimleri de yapılmıştır.

- Bunlardan EDOM, fonksiyona gelen parametreler fonksiyona tanımlanmış sınırların dışında kaldığında kullanıcıyı uyarmak için kullanılır (Negatif bir sayının karekökünün hesaplanmaya çalışılması gibi.).

- HUGE_VAL ise fonksiyonda hesaplanan sonucun double tipinde gösterilemeyecek kadar büyük ya da küçük olduğunu kullanıcıya bildirir.

Karmaşık Sayılar

- C dilinde, karmaşık sayıları tanımlamak için *int*, *float* gibi özel bir veri tipi yoktur.

- Karmaşık sayılarla çalışabilmek için **complex.h** kütüphanesinde bazı tanımlamalar yapılmıştır, dolayısıyla bu başlık dosyasının programa dahil edilmesi gerekir.

```
#include<complex.h>
```

- **z=a+jb** şeklinde tanımlanan kompleks sayıların **a** ve **b** katsayıları, sırasıyla sayının reel ve imajiner bileşenlerini temsil etmekle birlikte, birer *double* ya da *long double* veri olarak tanımlanabilirler.

```
double _Complex a = 3+5*I ;
```

```c
#include<stdio.h>
#include<complex.h>
main() {
    double _Complex a=3+5*I;
    double _Complex b=3+4*I;
    printf("%lf\n", creal(a)*cimag(b));
    printf("%lf\n", cimag(a)+creal(b));
    return 0;  }
```

```
12.000000
8.000000

--------------------------------
Process exited after 0.04058 seconds with return value 0
Press any key to continue . . .
```

Trigonometrik Fonksiyonlar

Fonksiyon Adı	İşlevi / Amacı
sin(x)	Sinüs / Radyan olarak verilen x açısının sinüsünü hesaplar.
cos(x)	Kosinüs / Radyan olarak verilen x açısının kosinüsünü hesaplar.
tan(x)	Tanjant / Radyan olarak verilen x açısının tanjantını hesaplar. Açı (pi/2)'nin tek katlarına çok yakınsa hata verebilir.
asin(x)	Ters sinüs / Verilen x değerinin ters sinüsünü hesaplar. x, (-1,1) arasında değilse EDOM hatası üretilir. (sonuç -pi/2 ile pi/2 arasında)
acos(x)	Ters kosinüs / Verilen x değerinin ters kosinüsünü hesaplar. x, (-1,1) arasında değilse EDOM hatası üretilir. (sonuç 0 ile pi arasında)
atan(x)	Ters tanjant / Verilen x değerinin ters tanjantını hesaplar. (sonuç –pi/2 ile pi/2 arasında)
sinh(x)	Hiperbolik sinüs / Radyan olarak verilen x açısının hiperbolik sinüsünü hesaplar.
cosh(x)	Hiperbolik kosinüs / Radyan olarak verilen x açısının hiperbolik kosinüsünü hesaplar.

```c
#include<stdio.h>
#include<math.h>
main() {
    float x, y, PI = 22.0/7.0;

    printf("\n\n sin(%.2f)=%.2f",x,sin(x));

    x = 45;
    y = x * (PI/180);

    printf("\n\n sin(%.2f)=%.2f",x,sin(y));
    printf("\n\n cos(%.2f)=%.2f",x,cos(y));
    printf("\n\n tan(%.2f)=%.2f",x,tan(y));

    printf("\n\n *** Program Sonu *** \n\n");
    return 0;  }
```

```
sin(0.00)=0.00

sin(45.00)=0.71

cos(45.00)=0.71

tan(45.00)=1.00

*** Program Sonu ***
```

Yuvarlatma Fonksiyonları

- **ceil(x)**: x değerinden büyük en küçük tam sayıyı bulur ve çağırana gönderir.

  Örneğin: x=7.3 ise 8, x=-7.3 ise -7 gönderir.

- **floor(x)**: x' ten küçük en büyük tam sayıyı bulur ve çağırana gönderir.

  Örneğin: x=7.3 ise 7, x=-7.3 ise -8 gönderir.

Logaritmik Fonksiyonlar

- **exp(x)**: e'nin x. kuvvetini hesaplar ve çağırana gönderir. Üstten taşma olursa HUGE_VAL, alttan taşma olursa 0 gönderir.

- **log(x)**: x'in doğal logaritmasını (ln) hesaplar ve çağırana gönderir. x eksi sayı ise EDOM gönderir.

- **log10(x)**: x'in 10 tabanında logaritmasını hesaplar ve çağırana gönderir. x eksi sayı ise EDOM gönderir.

Üstel Fonksiyonlar

- **pow(x,y)**: x'in y. kuvvetini hesaplar ve çağırana gönderir. Eğer x eksi bir sayı ve y bir tam sayı değilse ya da x sıfırsa ve y sıfırdan küçük ya da eşitse EDOM üretilir.

- **ldexp(x,n)**: 2'nin n. kuvvetiyle x'in çarpımını hesaplar ve çağırana gönderir.

Diğer Bazı Fonksiyonlar

- **abs(x):** x'in mutlak değerini gönderir.
- **fabs(x):** Gerçel sayıların mutlak değerini bulmak için kullanılır.
- **labs(x):** Uzun tamsayının mutlak değerini gönderir.
- **sqrt(x)**: x'in karekökünü hesaplayıp gönderir. x negatifse EDOM üretilir.
- **rand()**: Rastgele sayı üretmekte kullanılır.
- **div_t div(x,y):** x'i y'ye böler ve bölme sonucunu div_t içerisinde iki elemana yerleştirir.
- **ldiv_t ldiv(x,y):** div() fonksiyonunun uzun tam sayıları bölmek için kullanılan versiyonudur.
- **fmod(x,y):** x'in y'ye bölümünden kalanı gönderir.
- **modf(x, *y)**: x gerçel sayısının tam ve kesirli kısımlarını ayırır, tam kısmını y işaretçisinin gösterdiği adrese yerleştirip kesirli kısmını çağırana gönderir.

```c
#include<stdio.h>
#include<math.h>
main() {
    float x=10.0, y=-5.0, n=2, z=25;

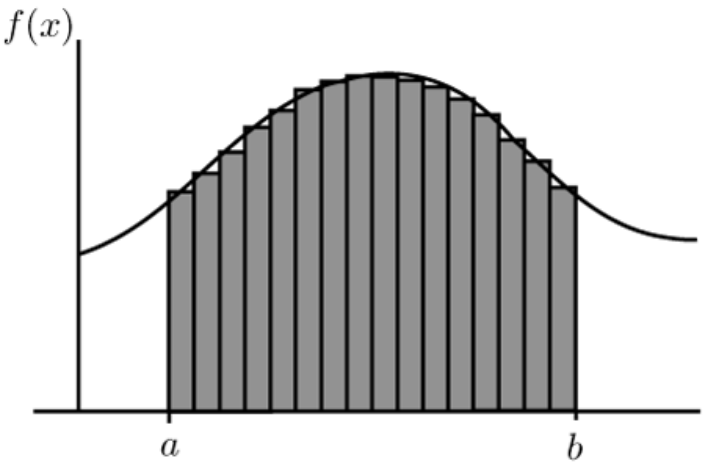
    printf("log10(%.1f)=%.2f \n",x,log10(x));
    printf("log(%.1f)=%.2f \n",x,log(x));
    printf("%.1f 'nin mutlak degeri: %.2f \n",y,fabs(y));
    printf("%.1f uzeri %.1f 'nin degeri: %.2f \n",y,n, pow(y,n));
    printf("%.1f 'nin karekok degeri: %.2f \n",z,sqrt(z));

    return 0;  }
```

```
log10(10.0)=1.00
log(10.0)=2.30
-5.0 'nin mutlak degeri: 5.00
-5.0 uzeri 2.0 'nin degeri: 25.00
25.0 'nin karekok degeri: 5.00
```

İntegral Hesabı

- İntegral, temelde alan hesabı için kullanılır. Sürekli bir fonksiyonun belirli aralıkta hesaplanan integralinin değeri, söz konusu fonksiyonun grafiğinin x ekseniyle arasında kalan alanıdır. Yani şekildeki f(x) fonksiyonunun a'dan b'ye integrali; yaklaşık olarak a ve b arasındaki eşit genişlikli dikdörtgenlerin alanlarının toplamından elde edilebilir.



Buradaki dikdörtgen genişliği duyarlılığı ifade etmekle birlikte, dikdörtgen sayısı arttıkça gerçek sonuca daha çok yaklaşılmakta ancak bu ekstra hesaplamalar gerektirmektedir.

```c
#include<stdio.h>

#define F(a) ((a*a)+4)
main() {
    float x, x_alt, x_ust;
    float top=0.0, h, integral;
    int dilim=1000;

    x_alt=1;
    x_ust=4;


    h= (x_ust-x_alt)/dilim;
    x=x_alt;

    while(x<x_ust) {
        top+=F(x);
        x+=h;
    }
    integral=h*top;

    printf("Integral (%.1f ve %.1f sinirlari arasinda yaklasik): %.2f\n",x_alt,x_ust,integral);
    return 0;  }
```

```
Integral (1.0 ve 4.0 sinirlari arasinda yaklasik): 32.98

--------------------------------
Process exited after 0.08965 seconds with return value 0
Press any key to continue . . .
```

KAYNAKLAR

- Dr. Rıfat ÇÖLKESEN, Programlama Sanatı, Papatya Yayıncılık, Cilt 1, Kasım 2004.

BİLGİSAYAR PROGRAMLAMA 1

Ders Notu 13 – Dosya İşlemleri

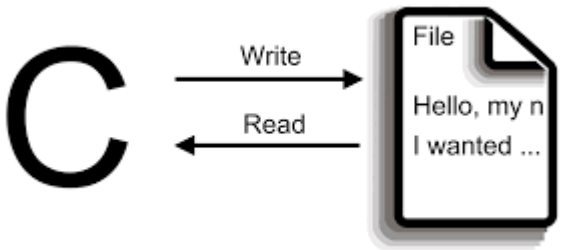
Konya Teknik Üniversitesi
Elektrik – Elektronik Mühendisliği Bölümü

15/06/2021

Konya

Dosya İşlemleri

- Değişkenlerden farklı olarak bir dosya, programın yürütülmesinden önce ve/veya sonra da var olmaktadır.
- Dosyalar sayesinde başka programlar da birbirleriyle haberleşebilirler, bunlar arasında veri aktarımı sağlanır.
- Dosya boyutlarının sabit diskin izin verdiği ölçüde sınırsız genişlikte olması da bunları kullanmamız için önemli nedenlerdendir.



Dosya Türleri

- İşletim sistemleri dosya türlerinin **Metin Dosyası** ve **İkili(Binary) Dosyalar** olarak iki farklı biçimde saklanmasına izin verirler.
- Metin dosyaları sadece ASCII karakterlerinden oluşabilir.
- Bundan dolayı ASCII dosyaları şeklinde de adlandırılabilirler.
- Her ASCII karakteri 1 bayt olduğundan metin dosyasında kaç karakter varsa boyutu da o kadar bayttır.
- Metin dosyaları sadece ileri yönlüdür, mesela 3 satırdan oluşan bir metin dosyasının 2. satırına metin ekleyemezsiniz, en sona ekleyebilirsiniz.
- İkili dosyalar baytlar halinde okunur ve yazılırlar. 8 bitlik 1 ve 0'lar.
- Özel editörler ile okunur ve yazılırlar.
- Örneğin Microsoft Word'ün .doc ve .docx uzantılı dosyaları.
- En önemli avantajı dosya üzerinde ileri ve geri gidilebilmesidir.

FILE

- C programlama dilinde giriş ve çıkışlar Akış(stream) adı verilen yapı üzerinden gerçekleştirilir.
- Bir dosya yazılıp okunmadan önce o dosya için bir akış oluşturulmalıdır.
- Bu akışı oluşturup takip edebilmek için FILE yapısı kullanılır.
- FILE genel kütüphanede tanımı bulunan bir yapıdır.
- Yani özetle bir dosya oluşturulmadan önce FILE tipinde bir işaretçi tanımlanmalıdır.

fopen

- Dosya üzerinde işlem yapmadan önce dosyanın açılması gereklidir.
- fopen fonksiyonu dosyayı açmaya yarar.
- Bu fonksiyon kullanılırken iki argüman alır: Açılacak dosya adı ve erişim modu. Erişim modu, hangi modda açılacağını tanımlar.
- fopen("Yeni Metin Belgesi.txt", "w");

Sembol	Erişim Modu
r	okuma (read)
w	yeniden yazma (write)
a	sona ekleme (add)
r+	okuma ve yazma (dosya zaten varolmalı)
w+	okuma ve yeniden yazma (değiştirme)
a+	okuma ve ekleme
rb	ikili dosyayı okuma amaçlı açar
r+b	ikili dosyayı okuma ve yazma amaçlı açar

fopen

- Eğer açma işlemi başarılıysa fopen fonksiyonu dosya işaretçisi döndürür. Başarısızsa null döndürür.

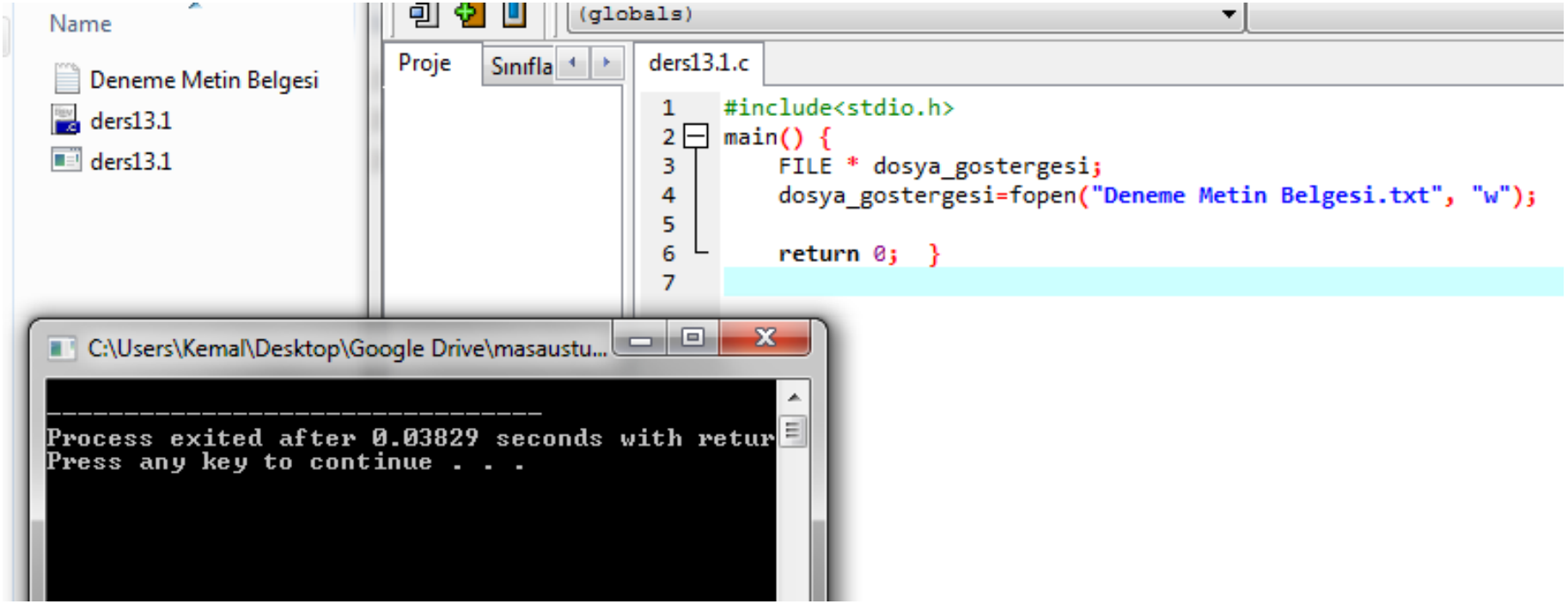
- Dosya işaretçisi tanımlamak ve yeni dosya oluşturmak için;

```
FILE *dosya_gostergesi;
dosya_gostergesi=fopen("Yeni Metin Belgesi.txt", "w");
```

- FILE tipi ve fopen gibi fonksiyonlar stdio.h kütüphanesinde tanımlıdırlar.
- fopen fonksiyonu işletim sistemine dosyanın kullanıldığını bildirir ve işletim sistemi o dosyanın başka programlar tarafından işlenmesine, silinmesine izin vermez

fopen

- Örnek-1:



fclose

- Bir dosyayla işimiz bittiğinde o dosyayı kapatmamız gerekir.
- Bu bellek akışını fclose fonksiyonu ile sonlandırabiliriz.
- Kullanım:

  fclose (FILE *kapatilacak_dosya_adi);

- Eğer dosya başarılı bir şekilde kapatıldıysa 0 değeri döndürür.
- Başarısız olursa sıfırdan farklı bir değer döndürülür.

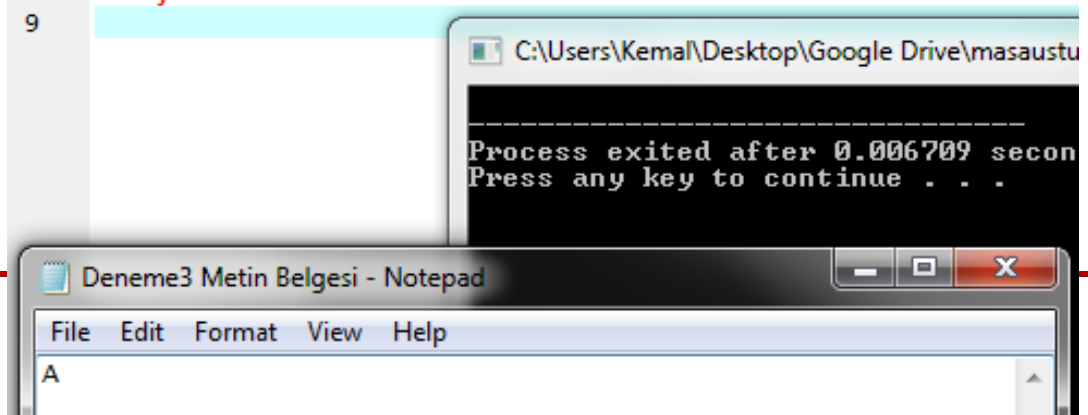
```
FILE *dosya_gostergesi=fopen("Yeni Metin Belgesi.txt", "w");
fclose(dosya_gostergesi)
```

Metin Dosyası İşlemleri

- Okuma ve yazma işlemlerini yapan fonksiyonlar 3 çeşittir:
  - karakter bazında işlem yapanlar: putc, fputc, getc, fgetc
  - biçimlendirilmiş metin bazında: fprintf, fscanf
  - satır bazında : fputs, fgets
- dosya_gostergesi ile işaret edilen bir dosyaya karakter yazdırma işlemi yapmak için;

  putc(‘z’, dosya_gostergesi)
- Örnek-2:

```
1  #include<stdio.h>
2
3  main() {
4      FILE *dosya_gostergesi;
5      dosya_gostergesi=fopen("Deneme3 Metin Belgesi.txt", "w");
6      putc('A',dosya_gostergesi);
7      return 0;
8      }
9
```



putc, fputc

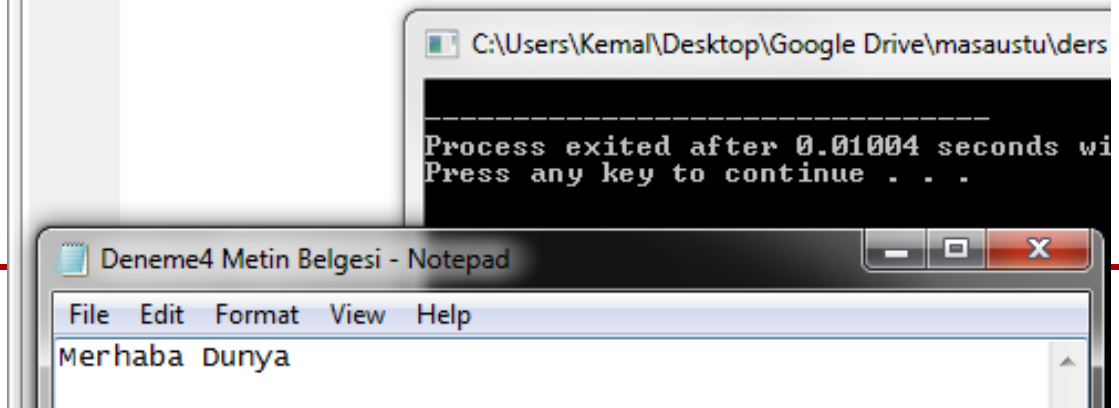
- putc ve fputc birebir aynı işi yapmaktadırlar.
- fputc bir C fonksiyonuyken, putc ise bir makro olarak tanımlanmıştır.
- Performans gerektiren işlemlerde putc daha hızlı çalışır. Ancak makrolarla ilgili hatalar oluşabilmektedir. fputc kullanımı önerilir.
- Örnek-3:

```c
#include<stdio.h>

main() {
    int i;
    char dizi1[]="Merhaba Dunya";
    FILE *dosya_gostergesi;
    dosya_gostergesi=fopen("Deneme4 Metin Belgesi.txt", "w");

    for (i = 0; dizi1[i]!='\0'; i++)
        fputc(dizi1[i], dosya_gostergesi);

    return 0;  }
```



EOF değeri ve feof fonksiyonu

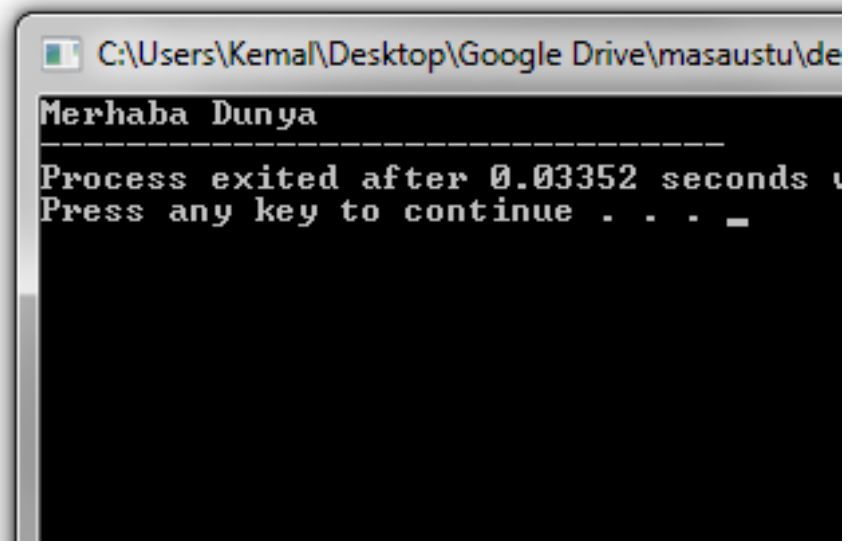
- EOF: end of file
- Bir dosyadan karakterlerin okunması işi yapılırken, dosyanın sonuna gelip gelmediğimizi gösteren bir göstergeye ihtiyaç duyarız.
- getc ve fgetc fonksiyonları dosyadan bir değer okunamadığında bir EOF değeri döndürürler.
- Dosya sonuna ulaşıldığında ya da dosyaya erişim bir sorundan dolayı kesildiğinde de EOF değeri döndürüldüğünden dolayı buna göre daha kullanışlı olan feof fonksiyonu üretilmiştir.
- Sadece dosyada okunacak bir karakter kalmadığında feof fonksiyonu sıfır dışında bir değer döndürür.

getc, fgetc

- Her iki fonksiyon da kendisine parametre olarak gelen dosyadan tek bir karakter okumaya yarar.
- Dosya yolu yazılmazsa programın bulunduğu klasör kastedilmiş olur.
- dosya=fopen("D:\\Deneme\\Deneme4 Metin Belgesi.txt", "r");
- Örnek-4a: EOF

while içinde okunan s değeri, EOF değilse döngü çalışmaya devam eder

```
1   #include<stdio.h>
2
3   main() {
4
5       FILE *dosya;
6       dosya=fopen("Deneme4 Metin Belgesi.txt", "r");
7       char s;
8       while((s=fgetc(dosya))!=EOF)
9       {
10      printf("%c",s);
11      }
12
13      fclose(dosya);
14      return 0;  }
15
```



getc, fgetc

- Örnek-4b:

feof fonksiyonu sıfır dışında bir değer döndürdüğünde dosya sonuna gelinmiştir.

```c
#include<stdio.h>
main() {
    FILE *dosya;
    dosya=fopen("Deneme4 Metin Belgesi.txt", "r");

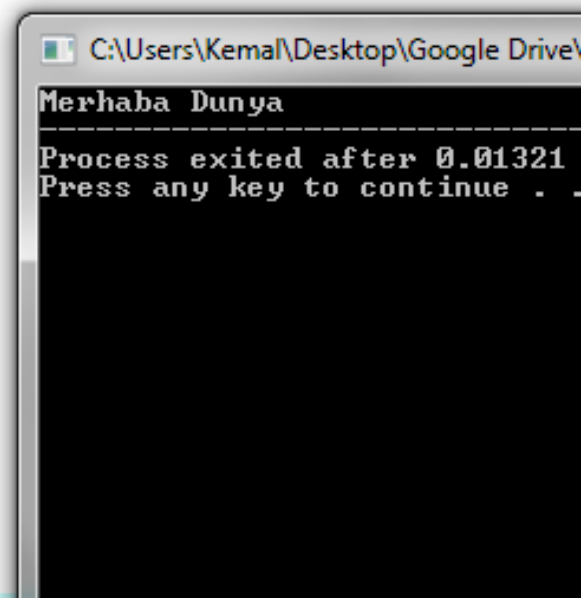
    if (dosya == NULL)    //Dosya açılamazsa çıkış yapılacak
        return 0;

    do
    {
        char c = fgetc(dosya); // Her seferinde tek karakter alacak

        if (feof(dosya))  //Okunacak değer varsa sıfır değeri döndürülür
            break ;

        printf("%c", c);
    }  while(1);

    fclose(dosya);
    return 0;  }
```



fprintf ve fscanf

- Biçimlendirilmiş metin bazında işlem yapan bu iki fonksiyon aslında printf ve scanf gibi çalışır.
- fscanf'in farkı klavyeden okumak yerine dosyadan okumaktır.
- fprintf'in farkı ekrana yazmak yerine dosyaya yazmaktır.
- Belirtilen dosya akışı üzerinden çalışmaktadır.
- Örnek-5:

```c
#include<stdio.h>
main() {
    FILE *dosya1, *dosya2;

    char ch;
    dosya1=fopen("Deneme3 Metin Belgesi.txt", "r");
    dosya2=fopen("Deneme4 Metin Belgesi.txt", "r+");
    if (dosya1 == NULL)   //Dosya açılamazsa çıkış yapılacak
    {printf("dosya1 bulunamadi\n");
    return 0;
    }

    fscanf(dosya1,"%c",&ch); // 1.dosyadaki A karakteri okunuyor

    if (dosya2 == NULL)   //Dosya açılamazsa çıkış yapılacak
    {printf("dosya2 bulunamadi\n");
    return 0;
    }
    fprintf(dosya2,"%c",ch); // 2.dosyanın başına A karakteri yazılıyor

    fclose(dosya1);
    fclose(dosya2);
    return 0;  }
```

```
Process exited after 0.0122
Press any key to continue .
```

Deneme4 Metin Belgesi - Notepad

File Edit Format View Help

Aerhaba Dunya

fputs ve fgets

- Satır bazında işlem yapan bu iki fonksiyon putc ve getc gibi çalışmaktadır fakat sadece bir satır boyunca çalışırlar.
- Maksimum satır uzunluğu bilgisi satırdaki karakter sayısından fazla olsa bile alt satıra geçilmez.
- fgets (kelimeDizisi[i], maxSatirUzunlugu, dosya);
- fputs(kelimeDizisi, dosya1);
- Örnek-6:

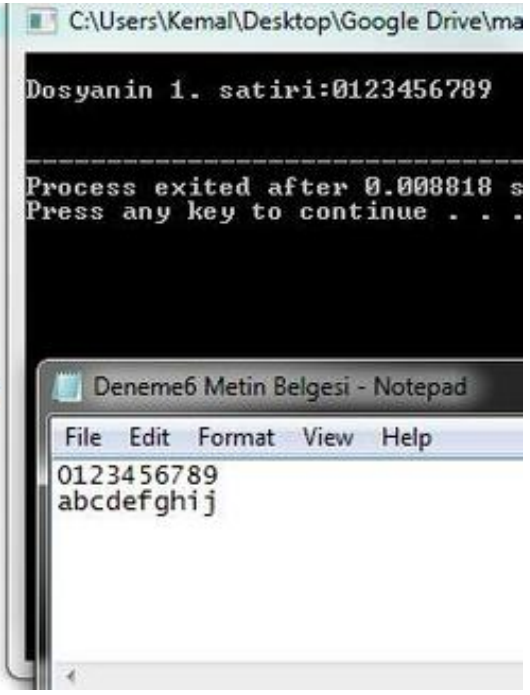
```c
#include<stdio.h>
main() {
    FILE *dosya1, *dosya2;
    char dizi1[20];
    int i;
    dosya1=fopen("Deneme6 Metin Belgesi.txt", "w");

    for(i=0;i<10;i++)
    {
    fprintf(dosya1,"%d",i); // dosyanın 1. satırına 0'dan 9'a kadar sayılar yazdırılıyor
    }
    fprintf(dosya1,"\n");   //Alt satıra geçiliyor
    fputs("abcdefghij", dosya1);// dosyanın 2. satırına a'dan j'ye kadar harfler yazdırılıyor
fclose(dosya1);
    dosya1=fopen("Deneme6 Metin Belgesi.txt", "r");
    fgets(dizi1, 20, dosya1);

    printf("\nDosyanin 1. satiri:%s\n", dizi1);

    fclose(dosya1);

    return 0;  }
```



rewind fonksiyonu

- Görevi imleci dosyanın başına getirmektir.

- Bir dosyada işlem yapılırken akış başlatıldığından işlemleri bu akış denetler. Bu akışa göre de dosya üzerinde okuma veya yazma işlemi yapıldığında, imleç görevin bittiği yerde kalır.

- Başa döndürmek istenilen dosyayı bu fonksiyona göndermek yeterlidir:

- Örnek:

```
rewind(dosya1);
```

fseek fonksiyonu

- Görevi imleci dosyanın belirli bir yerine götürmeye yarar.
- int fseek(FILE *dosya, long ofset, int lokasyon);
- lokasyon için 3 seçenek mevcuttur:
  - SEEK_SET ise yeni konumu dosyanın başlangıcından itibaren ayarlar,
  - SEEK_CUR ise yeni konumu geçerli dosya konumundan itibaren ayarlar,
  - SEEK_END ise yeni konumu dosyanın sonundan itibaren ayarlar.
- Örnek:

fseek (dosya, 10, SEEK_SET) //dosyanın en başından 10 bayt ileri

fseek (dosya, -5, SEEK_CUR) //dosyanın aktif konumdan 5 bayt geri

fseek (dosya, -100, SEEK_END) //dosyanın en sonundan 100 bayt geri

- İkili(binary) modda çalıştığı zaman işlevseldir. Metin modunda açıldığında ofset değeri için sadece 0 değeri kullanılabilir.

ftell fonksiyonu

- ftell fonksiyonu imlecin o an dosyanın neresinde olduğunu söyleyen bir fonksiyondur.

- Akış ikili (binary) modda açıldığı zaman bir anlamı vardır.

- long int uzunluk = ftell (dosya);

ÖDEV:

- **Kullanıcıdan 5x5 lik bir matris girmesini isteyin.**
- **Bunu matris1.txt isimli bir dosyaya yazdırın.**
- **Ardından matris1'in transpozunu alıp matris2.txt isimli dosyaya yazın.**
- **Transpoz işlemini yaparken matris1.txt dosyasından her bir satırı veya karakteri okuyup ondan sonra matris2'ye yazdırın.**

KAYNAKLAR

- Her Yönüyle C, Tevfik KIZILÖREN, Kodlab